

APRENDIZAJE Y CLASIFICACIÓN AUTOMÁTICA CON R



LIBRO GRATUITO

Para estudiantes, docentes y profesionales.



TEORÍA • DATOS REALES • CÓDIGO
GOOGLE COLAB



Aprende con datos reales de México y el mundo.



Ejemplos prácticos en R paso a paso.



Cuadernos completos en Google Colab.



Libro gratuito, abierto y en constante actualización.



TEMAS PRINCIPALES

EDICIÓN INTERACTIVA AMPLIADA

12 capítulos + laboratorios web

- R y Quarto
- Google Colab

• Laboratorios Shinylive

Incluye: regresión, k-NN, árboles, Random Forest, SVM, Naive Bayes, redes neuronales y aprendizaje no supervisado.

• Datos abiertos reales

• Curso en YouTube

• PDF descargable

MI JESÚS GILBERTO
RODRIGUEZ ESCOBEDO

Autor del libro y contenidos
Docente universitario con más de
45 años de experiencia



Cuaderno completo con
todos los capítulos y
código ejecutable

colab.research.google.com



LIBRO EN LÍNEA

[gilbertorodriguez59.github.io/
libro-machine-learning-r/](https://gilbertorodriguez59.github.io/libro-machine-learning-r/)



GITHUB

[github.com/
gilbertorodriguez59/
libro-machine-learning-r](https://github.com/gilbertorodriguez59/libro-machine-learning-r)



YOUTUBE

Videos explicativos, tutoriales
y casos prácticos
[@AprendizajeAutomaticoConR](https://www.youtube.com/@AprendizajeAutomaticoConR)

Tabla de contenidos

Presentación	11
Aprendizaje y Clasificación Automática con R	11
Un enfoque práctico con datos abiertos reales	11
Objetivo general	11
Público al que va dirigido	12
Software utilizado	12
Cuaderno completo de Google Colab	12
Video de presentación del libro	12
Curso completo en YouTube	13
Curso completo en YouTube	13
Lista oficial del curso	13
Materiales complementarios descargables	14
Estructura actual del libro	14
Ruta prevista para los siguientes capítulos	15
Nota para el lector	15
Fuente principal de los datos	15
Licencia, autoría y forma de citar	16
Autor	16
Licencia	16
Cita sugerida	16
Uso educativo	16
Cuaderno completo para Google Colab	17
¿Qué contiene el cuaderno?	17
Cómo utilizarlo	17
Diferencias entre el libro web y el cuaderno	18
Ejecución autónoma en Google Colab	18
Instalación automática de paquetes	18
Referencias bibliográficas en Colab	19
Descarga automática de los datos	19
1 ¿Qué es el aprendizaje automático?	20
1.1 Objetivos del capítulo	20
1.2 Introducción	20
1.3 Programación tradicional contra aprendizaje automático	20
1.4 Variables predictoras y variable respuesta	22
1.5 Tipos principales de aprendizaje automático	22

1.6	Notación básica	22
1.7	Primer ejemplo visual en R	22
1.8	División entre entrenamiento y prueba	23
1.9	Primer clasificador basado en una regla	24
1.10	Materiales complementarios del capítulo	25
1.11	Conclusión	26
2	Preparación de datos reales	27
2.1	Objetivos del capítulo	27
2.2	Introducción	27
2.3	Descargar la base ATUS desde el INEGI	28
2.4	Cómo citar los datos	29
2.5	Cargar paquetes	29
2.6	Localizar el archivo	29
2.7	Leer el archivo CSV	30
2.8	Primer vistazo a los datos	31
2.9	Normalizar los nombres de variables	32
2.10	Seleccionar variables de interés	32
2.11	Crear la variable respuesta	33
2.12	Preparar las variables finales	34
2.13	Guardar la base preparada	35
2.14	Lista de comprobación	35
2.15	Resumen del capítulo	36
2.16	Materiales complementarios del capítulo	36
3	Análisis exploratorio de datos	38
3.1	Objetivos del capítulo	38
3.2	Cargar paquetes y base	38
3.3	Preparar variables	39
3.4	Distribución de la variable respuesta	39
3.5	Accidentes por mes	40
3.6	Accidentes por hora del día	42
3.7	Tipos de accidente más frecuentes	43
3.8	Proporción por tipo de accidente	44
3.9	Materiales complementarios del capítulo	45
3.10	Conclusión	46
4	Regresión lineal simple y múltiple	47
4.1	Objetivos del capítulo	47
4.2	¿Por qué estudiar regresión lineal en un libro de clasificación?	47
4.3	Regresión lineal simple	48
4.4	Primer ejemplo didáctico	48
4.5	Visualizar la relación	49

4.6	Ajustar el modelo simple	49
4.7	Interpretar los coeficientes	50
4.8	Realizar una predicción	51
4.9	Residuos	51
4.10	Visualizar los residuos	52
4.11	Regresión lineal múltiple	53
4.12	Preparar un ejemplo múltiple	53
4.13	Ajustar el modelo múltiple	54
4.14	Interpretar los coeficientes múltiples	54
4.15	Separar entrenamiento y prueba	55
4.16	Métricas de evaluación	56
4.17	Observado frente a predicho	56
4.18	Aplicación con datos ATUS agregados	57
4.19	Construir una base mensual	58
4.20	Ajustar la regresión múltiple con ATUS	59
4.21	Supuestos principales	60
4.22	Diferencias entre regresión lineal y logística	61
4.23	Laboratorio interactivo: regresión lineal	61
4.23.1	Laboratorio disponible en la versión web	61
4.24	Actividades para el lector	61
4.25	Conclusiones	61
4.26	Materiales complementarios del capítulo	62
5	Regresión logística para clasificación	63
5.1	Objetivos del capítulo	63
5.2	Fundamento matemático	63
5.3	Visualización de la función logística	63
5.4	Cargar paquetes y datos	64
5.5	Preparar y dividir datos	65
5.6	Ajustar modelo	65
5.7	Predicción y matriz de confusión	66
5.8	Métricas	67
5.9	Distribución de probabilidades	68
5.10	Materiales complementarios del capítulo	68
5.11	Laboratorio interactivo: regresión logística	69
5.11.1	Laboratorio disponible en la versión web	69
5.12	Conclusión	70
5.13	Referencias fundamentales de la regresión logística	70
6	k vecinos más cercanos, k-NN	71
6.1	Objetivos del capítulo	71
6.2	Idea intuitiva	71
6.3	Distancia euclidiana	72

6.4	Cargar y preparar datos	73
6.5	Modelo k-NN	73
6.6	Comparación de valores de k	74
6.7	Laboratorio interactivo: k-NN	76
6.7.1	Laboratorio disponible en la versión web	76
6.8	Materiales complementarios del capítulo	76
6.8.1	Video del capítulo	76
6.9	Conclusión	77
6.10	Referencias fundamentales de k-NN	77
7	Árboles de decisión para clasificación	78
7.1	Objetivos del capítulo	78
7.2	Introducción	78
7.3	Fundamento matemático	78
7.4	Cargar paquetes y datos	78
7.5	Usar la base completa	79
7.6	División y entrenamiento	79
7.7	Complejidad e importancia	80
7.8	Gráfica opcional del árbol	81
7.9	Predicción y métricas	81
7.10	Laboratorio interactivo: construye una división del árbol	82
7.10.1	Laboratorio disponible en la versión web	82
7.11	Materiales complementarios del capítulo	82
7.11.1	Video del capítulo	82
7.12	Conclusión	83
7.13	Referencias fundamentales de los árboles de decisión	83
8	Random Forest para clasificación	84
8.1	Objetivos del capítulo	84
8.2	Introducción	84
8.3	Fundamento matemático	84
8.4	Cargar paquetes y datos	84
8.5	Muestra de trabajo	85
8.6	División en entrenamiento y prueba	86
8.7	Ajustar Random Forest	87
8.8	Importancia de variables	88
8.9	Predicción y métricas	89
8.10	Distribución de probabilidades predichas	90
8.11	Conclusión	91
8.12	Referencias fundamentales de Random Forest	91
9	Evaluación, validación y comparación de modelos	92
9.1	Objetivos del capítulo	92

9.2	Introducción	92
9.3	Cargar los datos	92
9.4	Preparar una muestra reproducible	93
9.5	Entrenamiento, validación y prueba	94
9.6	División estratificada	94
9.7	Matriz de confusión	95
9.8	Métricas principales	95
9.8.1	Exactitud	95
9.8.2	Sensibilidad	96
9.8.3	Especificidad	96
9.8.4	Precisión	96
9.8.5	Valor F1	96
9.9	Función para calcular métricas	96
9.10	Modelo de referencia: regla mayoritaria	98
9.11	Evaluar una regresión logística	99
9.12	Comparación inicial	100
9.13	Umbral de clasificación	102
9.14	Validación cruzada	103
9.14.1	Crear pliegues estratificados	103
9.14.2	Validación cruzada de la regresión logística	104
9.15	Cómo comparar varios modelos	106
9.16	Selección del modelo final	106
9.17	Actividad guiada	107
9.18	Actividades para el lector	107
9.19	Resumen del capítulo	107
9.20	Referencias fundamentales sobre evaluación	107
10	Máquinas de Vectores de Soporte (SVM)	108
10.1	Objetivos del capítulo	108
10.2	Introducción	108
10.3	Intuición geométrica	109
10.4	Crear un ejemplo didáctico	109
10.5	Visualizar las clases	110
10.6	Instalar y cargar el paquete <code>e1071</code>	111
10.7	Entrenar una SVM lineal	111
10.8	Identificar los vectores de soporte	112
10.9	Dibujar la frontera de decisión	113
10.10	El parámetro de costo	114
10.11	Cuando una línea no es suficiente	115
10.12	Kernel radial	116
10.13	Aplicación con los datos ATUS	116
10.14	Cargar la base preparada	116
10.15	Preparar una muestra reproducible	117

10.16	Dividir los datos de forma estratificada	118
10.17	Calcular pesos para las clases	118
10.18	Ajustar una SVM lineal	119
10.19	Realizar predicciones	120
10.20	Función segura para calcular métricas	120
10.21	Evaluar la SVM lineal	121
10.22	Ajustar una SVM con kernel radial	122
10.23	Evaluar la SVM radial	122
10.24	Comparar los dos modelos	123
10.25	Selección de hiperparámetros con validación cruzada	125
10.26	Ventajas y limitaciones	126
10.26.1	Ventajas	126
10.26.2	Limitaciones	126
10.27	Recomendaciones prácticas	126
10.28	Actividad guiada	126
10.29	Ejercicios	127
10.30	Conclusiones	127
10.31	Referencias fundamentales de SVM	128
11	Clasificador Naive Bayes	129
11.1	Objetivos del capítulo	129
11.2	Introducción	129
11.3	Recordatorio del teorema de Bayes	129
11.4	El supuesto ingenuo de independencia	130
11.5	Ejemplo manual con variables categóricas	130
11.6	Calcular las probabilidades previas	131
11.7	Calcular probabilidades condicionales	131
11.8	Clasificar una observación manualmente	132
11.9	El problema de las probabilidades iguales a cero	133
11.10	Naive Bayes gaussiano	133
11.11	Instalar y cargar paquetes	134
11.12	Cargar la base preparada de ATUS	134
11.13	Preparar una muestra para el modelo	134
11.14	División estratificada en entrenamiento y prueba	135
11.15	Entrenar el modelo Naive Bayes	136
11.16	Obtener predicciones de clase	138
11.17	Obtener probabilidades posteriores	138
11.18	Construir la matriz de confusión	139
11.19	Calcular las métricas	139
11.20	Visualizar las métricas	140
11.21	Inspeccionar las probabilidades aprendidas	141
11.22	Ventajas de Naive Bayes	142
11.23	Limitaciones	142

11.24	Comparación conceptual con otros algoritmos	142
11.25	Actividades para el lector	143
11.26	Conclusiones	143
11.27	Referencias fundamentales de Naive Bayes	144
12	Redes neuronales artificiales	145
12.1	Objetivos de aprendizaje	145
12.2	Introducción	145
12.3	La neurona artificial	146
12.3.1	Interpretación	146
12.4	Ejemplo manual de una neurona	146
12.5	Funciones de activación	147
12.5.1	Función escalón	147
12.5.2	Función sigmoide	147
12.5.3	Tangente hiperbólica	148
12.5.4	ReLU	148
12.5.5	Softmax	148
12.6	El perceptrón	148
12.6.1	Limitación	149
12.7	Estructura de una red multicapa	149
12.7.1	Capa de entrada	149
12.7.2	Capas ocultas	149
12.7.3	Capa de salida	149
12.8	Propagación hacia adelante	149
12.9	Función de pérdida	150
12.9.1	Error cuadrático medio	150
12.9.2	Entropía cruzada binaria	150
12.9.3	Entropía cruzada categórica	150
12.10	Descenso de gradiente	151
12.10.1	Tasa demasiado pequeña	151
12.10.2	Tasa demasiado grande	151
12.11	Retropropagación	151
12.12	Implementación manual de una neurona en R	151
12.13	Laboratorio interactivo: construye una neurona artificial	152
12.13.1	Laboratorio interactivo disponible en la versión web	152
12.14	Perceptrón desde cero en R	152
12.15	Ejemplo de XOR	154
12.16	Preparación de los datos	154
12.17	Normalización	155
12.17.1	Estandarización	155
12.17.2	Mínimo-máximo	155
12.18	Clasificación binaria con neuralnet	156
12.19	Predicción con neuralnet	158

12.20	Métricas binarias	158
12.21	Selección del umbral	159
12.22	Visualizador interactivo de una red neuronal	160
12.22.1	Visualizador disponible en la versión web	160
12.23	Arquitectura de la red	160
12.23.1	Red muy pequeña	160
12.23.2	Red demasiado grande	160
12.24	Épocas, lotes y optimizadores	161
12.24.1	Época	161
12.24.2	Lote	161
12.24.3	Descenso por lotes	161
12.24.4	Descenso estocástico	161
12.24.5	Mini-batch	161
12.24.6	Optimizadores	161
12.25	Sobreajuste	161
12.26	Regularización	162
12.26.1	Regularización L2	162
12.26.2	Regularización L1	162
12.26.3	Dropout	162
12.26.4	Early stopping	162
12.27	Implementación moderna con keras	162
12.28	Curvas de aprendizaje	164
12.29	Clasificación multiclase	164
12.30	Codificación de la respuesta	165
12.31	Inicialización de pesos	165
12.32	Gradientes que desaparecen o explotan	165
12.33	Importancia de la reproducibilidad	166
12.34	Interpretabilidad	166
12.35	Aplicación biomédica	167
12.36	Aplicación ambiental	167
12.37	Comparación con otros algoritmos	168
12.38	Ventajas	168
12.39	Limitaciones	168
12.40	Errores frecuentes	168
12.41	Flujo de trabajo recomendado	169
12.42	Actividad guiada	169
12.43	Ejercicios	170
12.43.1	Ejercicio 1	170
12.43.2	Ejercicio 2	170
12.43.3	Ejercicio 3	170
12.43.4	Ejercicio 4	170
12.43.5	Ejercicio 5	170
12.43.6	Ejercicio 6	170

12.43.7 Ejercicio 7	170
12.43.8 Ejercicio 8	170
12.43.9 Ejercicio 9	170
12.43.10 Ejercicio 10	171
12.44 Preguntas de reflexión	171
12.45 Conclusión	171
13 Agrupamiento con k-means	172
13.1 Objetivos de aprendizaje	172
13.2 Del aprendizaje supervisado al no supervisado	172
13.3 Idea intuitiva de k-means	172
13.4 Función objetivo	173
13.5 Ejemplo manual	173
13.6 Implementación básica en R	173
13.7 ¿Por qué usar <code>nstart</code> ?	175
13.8 Escalamiento de variables	175
13.9 Aplicación con <code>iris</code>	176
13.10 Interpretación de centroides	176
13.11 Método del codo	177
13.12 Coeficiente silhouette	178
13.13 Laboratorio interactivo k-means	178
13.14 Ventajas	178
13.15 Limitaciones	178
13.16 Errores frecuentes	179
13.17 Aplicación con datos reales	179
13.18 Actividad guiada	179
13.19 Ejercicios	179
13.20 Conclusión	180
Apéndices	181
Referencias	181
Reconocimiento de la fuente de datos	182

Presentación

Aprendizaje y Clasificación Automática con R

Un enfoque práctico con datos abiertos reales

Autor: MI Jesús Gilberto Rodríguez Escobedo

Versión: 0.13.2 (desarrollo) **Modalidad:** Libro abierto, práctico e interactivo

Tecnologías: R · Quarto · Machine Learning · Shinylive · Google Colab · YouTube · GitHub Pages

Edición interactiva ampliada

Este libro integra código en R, datos abiertos reales, videos, presentaciones, infografías, Google Colab y laboratorios interactivos disponibles en la versión web.

- Curso en YouTube: <https://www.youtube.com/playlist?list=PLDJYd2v7Kt-Q>
- Libro web: <https://gilbertorodriguez59.github.io/libro-machine-learning-r-dev/>

Este libro introduce el aprendizaje automático (*Machine Learning*) mediante ejemplos claros, didácticos y prácticos desarrollados principalmente en **R**.

La intención no es escribir un tratado matemático extenso, sino construir una guía aplicada para aprender haciendo con datos abiertos reales. En esta versión trabajamos con datos de accidentes de tránsito de México y los usamos como hilo conductor para explicar preparación de datos, análisis exploratorio y modelos de clasificación.

i Enfoque del libro

El libro combina fundamento conceptual, código reproducible en R, explicación breve del código, interpretación didáctica de resultados y visualizaciones profesionales.

Objetivo general

Desarrollar competencias básicas e intermedias en aprendizaje automático mediante el análisis, modelado e interpretación de datos reales.

Público al que va dirigido

Este libro está dirigido a estudiantes de ingeniería, ciencias, ciencia de datos, estadística aplicada y áreas afines que desean iniciar en aprendizaje automático con un enfoque práctico.

Software utilizado

- RStudio
- Consola de R
- Google Colab
- GitHub
- GitHub Pages
- Quarto Pub

Cuaderno completo de Google Colab

El contenido del libro también está disponible como un cuaderno ejecutable de Google Colab. Las explicaciones se presentan en celdas de texto y cada código R está separado para ejecutarse paso a paso.

- [Abrir el cuaderno en Google Colab](#)
- [Descargar el archivo .ipynb](#)
- [Consultar instrucciones de uso](#)

Video de presentación del libro

Este video ofrece una introducción general a la presentación, la licencia de uso y el cuaderno completo de Google Colab.

[Ver el video en YouTube](#)

Curso completo en YouTube

Escanea el código QR o abre el enlace para acceder a la lista de reproducción oficial del curso:



<https://www.youtube.com/playlist?list=PLDJYd2v7Kt-Q>

Curso completo en YouTube

El libro cuenta con una lista oficial de reproducción en YouTube, donde se reúnen los videos de presentación y de cada capítulo.

► [Ver el curso completo en YouTube](#)

Enlace directo: <https://www.youtube.com/playlist?list=PLDJYd2v7Kt-Q>

Lista oficial del curso



<https://www.youtube.com/playlist?list=PLDJYd2v7Kt-Q>

Materiales complementarios descargables

Para facilitar la presentación, consulta y difusión de las primeras secciones del libro —**Presentación, Licencia, autoría y forma de citar**, y **Cuaderno completo para Google Colab**— se ofrecen los siguientes recursos complementarios:

Recurso	Descripción	Abrir o reproducir	Descargar
Presentación en PDF	Diapositivas de lectura sobre la Presentación, la licencia y el cuaderno de Colab.	Ver PDF	Descargar PDF
Presentación editable	Archivo PowerPoint que puede utilizarse para exponer o adaptar el contenido.	Abrir o descargar PPTX	Descargar PPTX
Infografía	Síntesis visual para consulta rápida, clase o difusión educativa.	Ver infografía	Descargar PNG

Cómo descargar los materiales

Seleccione el enlace de la columna **Descargar**. Dependiendo de la configuración del navegador, el archivo puede descargarse inmediatamente o abrirse primero en una pestaña nueva. En ese caso, utilice el botón de descarga del navegador.

Los archivos se encuentran alojados en el mismo repositorio de GitHub que el libro, por lo que no es necesario iniciar sesión en Google Drive ni solicitar permisos adicionales.

Elaboración de los recursos

Estos materiales complementarios fueron elaborados con apoyo de **NotebookLM de Google**, a partir del contenido del libro, y posteriormente revisados, seleccionados y adaptados por el autor. El contenido del libro y sus archivos fuente constituyen la referencia principal.

Estructura actual del libro

1. Introducción al aprendizaje automático.
2. Preparación de datos reales.
3. Análisis exploratorio de datos.
4. Regresión lineal simple y múltiple.
5. Regresión logística para clasificación.

6. k vecinos más cercanos (k-NN).
7. Árboles de decisión.
8. Random Forest.
9. Evaluación, validación y comparación de modelos.
10. Máquinas de Vectores de Soporte (SVM).
11. Clasificador Naive Bayes.
12. Redes neuronales artificiales.
13. Agrupamiento con k-means.

Ruta prevista para los siguientes capítulos

14. Análisis de Componentes Principales (PCA).
15. Reglas de asociación con Apriori.
16. Comparación integral y selección de algoritmos.
17. Proyecto aplicado final con datos abiertos.

Nota para el lector

Los capítulos están diseñados para leerse y ejecutarse paso a paso. Cada bloque de código busca responder una pregunta concreta y cada resultado se acompaña de una interpretación didáctica.

Fuente principal de los datos

Los ejemplos aplicados utilizan información de la Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas del INEGI (Instituto Nacional de Estadística y Geografía 2025). Cuando una tabla o gráfica se elabora con estos datos, se indica expresamente la fuente y se distingue el procesamiento realizado por el autor.

Licencia, autoría y forma de citar

Autor

MI Jesús Gilberto Rodríguez Escobedo

Licencia

Este libro se distribuye bajo la licencia **Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0)**.

Esto significa que el material puede compartirse y adaptarse con fines educativos y no comerciales, siempre que se otorgue el crédito correspondiente al autor y las obras derivadas se compartan bajo una licencia compatible.

Cita sugerida

Rodríguez Escobedo, J. G. (2026). *Aprendizaje y Clasificación Automática con R: Un enfoque práctico con datos abiertos reales*. Libro web publicado con Quarto y GitHub Pages.

Uso educativo

El propósito de este libro es apoyar la enseñanza y el aprendizaje de técnicas de aprendizaje automático con R, usando datos reales y un enfoque didáctico, reproducible y aplicado.

Cuaderno completo para Google Colab

Además de la versión web y del documento PDF, el libro cuenta con un **cuaderno completo de Google Colab**. En él, las explicaciones aparecen en celdas de texto y cada bloque de código R se encuentra en una celda independiente para que pueda ejecutarse paso a paso.

💡 Abrir o descargar el cuaderno

Abrir directamente en Google Colab:

[Ejecutar el libro completo en Google Colab](#)

Descargar el archivo .ipynb:

[Descargar el cuaderno completo](#)

¿Qué contiene el cuaderno?

El cuaderno reproduce el contenido del libro y contiene:

- un índice navegable con todos los capítulos y secciones;
- explicaciones conceptuales en celdas de texto;
- fórmulas matemáticas;
- códigos R separados en celdas independientes;
- instalación y carga de los paquetes necesarios;
- preparación y lectura de los datos ATUS;
- análisis exploratorio;
- regresión logística;
- k vecinos más cercanos;
- árboles de decisión;
- Random Forest;
- evaluación y comparación de modelos;
- ejercicios, interpretaciones y referencias.

Cómo utilizarlo

1. Abra el enlace **Ejecutar el libro completo en Google Colab**.
2. Inicie sesión con una cuenta de Google, si Colab lo solicita.
3. Seleccione **Archivo** → **Guardar una copia en Drive** para conservar una copia editable.
4. Compruebe que el entorno de ejecución utilice **R**.
5. Ejecute primero las celdas de preparación e instalación de paquetes.

6. Continúe en orden, ejecutando cada celda con el botón de reproducción o con `Shift + Enter`.

⚠ Ejecución en orden

Algunos capítulos utilizan objetos creados en capítulos anteriores. Durante la primera ejecución se recomienda avanzar en el orden establecido y no saltar las celdas de preparación de datos.

Diferencias entre el libro web y el cuaderno

La versión web es la opción más cómoda para leer y consultar el contenido. El cuaderno de Colab está pensado para experimentar: permite modificar valores, ejecutar nuevamente los modelos y observar cómo cambian los resultados sin instalar R ni RStudio en la computadora.

i Datos y acceso a internet

El cuaderno necesita conexión a internet para instalar paquetes y descargar los archivos de datos cuando no estén disponibles en la sesión. La descarga de ATUS puede tardar debido al tamaño de la base.

Ejecución autónoma en Google Colab

El cuaderno incluye una celda inicial de **preparación automática** que:

- instala los paquetes de R que hagan falta;
- define dentro del propio cuaderno todas las funciones gráficas;
- crea la carpeta `datos`;
- descarga las bases ATUS desde el repositorio estable cuando no estén presentes;
- comprueba que los recursos necesarios estén disponibles.

Por esta razón, el usuario no necesita descargar manualmente `util_graficas.R` ni subir archivos auxiliares a Colab.

Instalación automática de paquetes

El cuaderno utiliza la función `cargar_paquete()`. Antes de cargar un paquete, comprueba si está instalado y, cuando hace falta, lo instala automáticamente desde CRAN. Esto evita errores como `there is no package called 'ranger'`.

Referencias bibliográficas en Colab

En los capítulos Quarto se conservan claves como `@breiman1984classification`, porque Quarto las transforma al generar el libro. En Google Colab esas claves se sustituyen por citas legibles como **Breiman et al. (1984)**, y la bibliografía completa aparece al final del cuaderno.

Descarga automática de los datos

Para evitar archivos demasiado grandes dentro del cuaderno, las bases ATUS se publican en formato comprimido `.csv.gz`. La primera celda de Colab:

1. descarga los archivos comprimidos desde GitHub Pages;
2. los descomprime automáticamente;
3. guarda los CSV en la carpeta `datos`;
4. comprueba que quedaron disponibles antes de continuar.

Los archivos publicados son:

- `recursos_colab/atus_ml_preparado.csv.gz`;
- `recursos_colab/atus_2024.csv.gz`.

Si la celda informa un error 404, primero debe publicarse esta versión del sitio para que esos recursos estén disponibles.

Capítulo 1

¿Qué es el aprendizaje automático?

1.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de explicar qué es el aprendizaje automático, diferenciarlo de la programación tradicional e identificar problemas de clasificación, regresión y agrupamiento.

1.2 Introducción

El aprendizaje automático, también conocido como *Machine Learning*, permite construir modelos capaces de aprender patrones a partir de datos.

En la programación tradicional, una persona define reglas. En aprendizaje automático, proporcionamos ejemplos para que un algoritmo aprenda una relación entre variables de entrada y una salida esperada.

1.3 Programación tradicional contra aprendizaje automático

```
pm10 <- 85

if (pm10 > 75) {
  print("Contaminación alta")
} else {
  print("Contaminación baja")
}
```

```
#> [1] "Contaminación alta"
```

i Explicación del código

Se define un valor de PM10 y después se aplica una regla fija. Si el valor es mayor que 75, el programa clasifica el día como de contaminación alta.

💡 Interpretación del resultado

Este ejemplo representa programación tradicional: la decisión depende de una regla escrita manualmente. En aprendizaje automático, la regla se aprende a partir de ejemplos.

```
datos <- data.frame(
  dia = 1:12,
  pm10 = c(42, 88, 55, 120, 63, 95, 38, 110, 72, 130, 47, 99),
  temperatura = c(25, 28, 22, 30, 24, 29, 21, 31, 27, 32, 23, 30),
  humedad = c(40, 35, 60, 30, 55, 33, 65, 29, 42, 28, 62, 34),
  clase = c("Baja", "Alta", "Baja", "Alta", "Baja", "Alta", "Baja", "Alta", "Baja", "Alta", "Baja", "Alta",
    ↪ "Baja", "Alta")
)

datos
```

```
#>   dia pm10 temperatura humedad clase
#> 1    1   42          25      40  Baja
#> 2    2   88          28      35  Alta
#> 3    3   55          22      60  Baja
#> 4    4  120          30      30  Alta
#> 5    5   63          24      55  Baja
#> 6    6   95          29      33  Alta
#> 7    7   38          21      65  Baja
#> 8    8  110          31      29  Alta
#> 9    9   72          27      42  Baja
#> 10  10  130          32      28  Alta
#> 11  11   47          23      62  Baja
#> 12  12   99          30      34  Alta
```

i Explicación del código

Se construye una pequeña base de datos con 12 días. Cada fila contiene variables ambientales y una clase conocida.

1.4 Variables predictoras y variable respuesta

La variable respuesta es la variable que queremos predecir. Las variables predictoras son las variables que usamos como entrada para el modelo.

1.5 Tipos principales de aprendizaje automático

1. Aprendizaje supervisado.
2. Aprendizaje no supervisado.
3. Aprendizaje por refuerzo.

1.6 Notación básica

Supongamos que tenemos n observaciones y p variables predictoras. Podemos representar los datos mediante una matriz X y una variable respuesta Y .

$$\hat{y} = f(x_1, x_2, \dots, x_p)$$

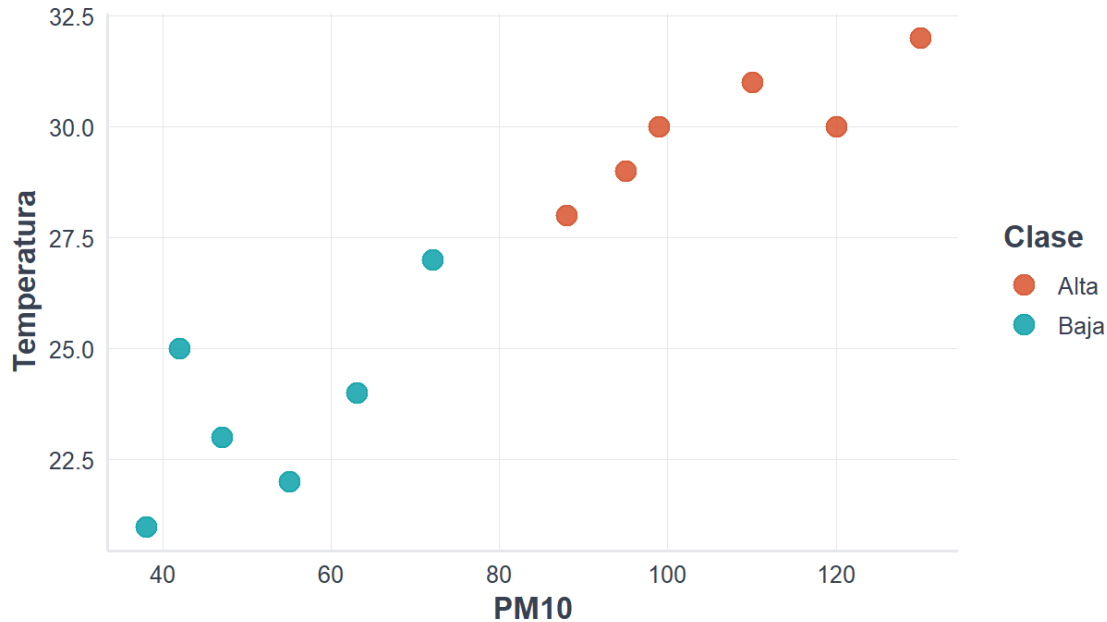
1.7 Primer ejemplo visual en R

```
library(ggplot2)
source("util_graficas.R")

ggplot(datos, aes(x = pm10, y = temperatura, color = clase)) +
  geom_point(size = 4, alpha = 0.9) +
  escala_clases_color() +
  labs(
    title = "Ejemplo inicial de clasificación",
    subtitle = "Clasificación de contaminación alta o baja",
    x = "PM10",
    y = "Temperatura",
    color = "Clase"
  ) +
  tema_libro()
```


Ejemplo inicial de clasificación

Clasificación de contaminación alta o baja



i Explicación del código

La gráfica coloca PM10 en el eje horizontal, temperatura en el eje vertical y usa color para distinguir la clase.

💡 Interpretación del resultado

Los puntos permiten visualizar si las clases se separan. Esta es la idea básica detrás de muchos métodos de clasificación.

1.8 División entre entrenamiento y prueba

```
set.seed(123)
n <- nrow(datos)
indices_entrenamiento <- sample(1:n, size = round(0.7 * n))

entrenamiento <- datos[indices_entrenamiento, ]
prueba <- datos[-indices_entrenamiento, ]

entrenamiento
```

```
#>   dia pm10 temperatura humedad clase
#> 3    3   55          22      60  Baja
#> 12   12   99          30      34  Alta
#> 10   10  130          32      28  Alta
#> 2    2   88          28      35  Alta
#> 6    6   95          29      33  Alta
#> 11   11   47          23      62  Baja
#> 5    5   63          24      55  Baja
#> 4    4  120          30      30  Alta
```

```
prueba
```

```
#>   dia pm10 temperatura humedad clase
#> 1    1   42          25      40  Baja
#> 7    7   38          21      65  Baja
#> 8    8  110          31      29  Alta
#> 9    9   72          27      42  Baja
```

Explicación del código

Se divide la base en dos partes: una para entrenar el modelo y otra para evaluar cómo funciona con datos no usados durante el ajuste.

1.9 Primer clasificador basado en una regla

```
datos$prediccion_regla <- ifelse(datos$pm10 > 75, "Alta", "Baja")

tabla_confusion <- table(
  Real = datos$clase,
  Predicho = datos$prediccion_regla
)

tabla_confusion
```

```
#>      Predicho
#> Real   Alta Baja
#>  Alta    6    0
#>  Baja    0    6
```

```
mean(datos$clase == datos$prediccion_regla)
```

```
#> [1] 1
```

💡 Interpretación del resultado

La matriz de confusión compara la clase real contra la clase predicha. La exactitud mide la proporción de aciertos.

1.10 Materiales complementarios del capítulo

Estos recursos permiten repasar los conceptos principales del capítulo mediante distintos formatos. La presentación puede consultarse en PDF o modificarse en PowerPoint; la infografía ofrece una síntesis visual y el video explica los contenidos de manera audiovisual.

Recurso	Utilidad	Abrir o reproducir	Descargar
Video explicativo	Introducción audiovisual al aprendizaje automático y a los contenidos del capítulo.	Ver en YouTube	—
Presentación en PDF	Diapositivas para lectura, estudio o exposición.	Ver PDF	Descargar PDF
Presentación editable	Archivo PowerPoint para utilizarlo en clase o adaptarlo.	Abrir PPTX	Descargar PPTX
Infografía	Resumen visual de las ideas fundamentales.	Ver infografía	Descargar PNG

Video del capítulo: <https://www.youtube.com/watch?v=xzTLERULM6s>

La presentación PDF, el archivo editable y la infografía pueden descargarse desde la versión web del libro.

 Elaboración de los materiales

Los materiales complementarios fueron elaborados con apoyo de **NotebookLM de Google**, a partir del contenido del capítulo, y posteriormente revisados y adaptados por el autor. El texto del libro y sus archivos fuente constituyen la referencia principal.

 Curso completo en YouTube

Este video forma parte de la lista oficial del curso **Aprendizaje y Clasificación Automática con R**.

[Consultar todos los videos del curso](#)

1.11 Conclusión

El aprendizaje automático permite construir modelos que aprenden patrones a partir de datos. En este capítulo vimos la diferencia entre reglas manuales y aprendizaje a partir de ejemplos.

Capítulo 2

Preparación de datos reales

2.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de:

- localizar y descargar la base ATUS desde el portal oficial del INEGI;
- organizar los archivos dentro del proyecto Quarto;
- leer y revisar una base real en R;
- seleccionar y transformar variables;
- construir una variable respuesta binaria;
- guardar una base preparada para los capítulos de modelado;
- citar correctamente la fuente de los datos.

2.2 Introducción

En la práctica profesional, los datos casi nunca llegan preparados para aplicar un algoritmo. Antes de modelar debemos conocer su procedencia, revisar su estructura, limpiar valores, transformar variables y documentar cada decisión.

En este libro utilizamos la **Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS)** del Instituto Nacional de Estadística y Geografía (Instituto Nacional de Estadística y Geografía 2025). Su objetivo es producir información anual sobre la siniestralidad del transporte terrestre en zonas de jurisdicción no federal, con desglose nacional, estatal y municipal.

! Uso responsable de la información

Los datos originales son del INEGI. La limpieza, transformación, selección de variables, gráficas, modelos e interpretaciones de este libro son elaboración del autor. No constituyen resultados oficiales ni implican el aval del Instituto.

2.3 Descargar la base ATUS desde el INEGI

La descarga debe realizarse desde el sitio oficial para conservar la procedencia y los metadatos del archivo.

1. Abra en su navegador el portal de **Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS)**:
<https://www.inegi.org.mx/programas/accidentes/>
2. Localice el apartado **Datos abiertos** o la opción de descarga correspondiente.
3. Seleccione el periodo anual que desea utilizar. Para reproducir los ejemplos de esta versión, elija **2024**.
4. Descargue el archivo en formato CSV. Es posible que el portal entregue un archivo comprimido en formato ZIP.
5. Descomprima el archivo descargado.
6. Identifique el CSV anual y cópielo en la carpeta `datos` del proyecto.
7. Para seguir exactamente los ejemplos, renómbrelo como:

```
atus_2024.csv
```

La estructura mínima del proyecto debe quedar así:

```
libro-machine-learning-r/  
  _quarto.yml  
  index.qmd  
  02-preparacion-datos.qmd  
  datos/  
    atus_2024.csv  
  referencias.bib
```

💡 Otro año de información

Puede utilizar un año diferente. En ese caso, cambie el nombre del archivo o modifique la ruta utilizada en el código. Conviene anotar siempre el año de referencia porque los resultados pueden cambiar entre periodos.

2.4 Cómo citar los datos

Los términos de libre uso del INEGI permiten utilizar, adaptar y publicar su información, pero requieren citar la fuente de origen (Instituto Nacional de Estadística y Geografía 2026).

En el texto puede utilizarse una cita como esta:

Los datos proceden de la Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas del INEGI (Instituto Nacional de Estadística y Geografía 2025).

Debajo de una tabla o gráfica sin transformaciones importantes:

Fuente: INEGI, Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS), 2024.

Cuando exista procesamiento, agrupación, modelado o visualización propia:

Fuente: Elaboración propia con datos del INEGI, Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS), 2024.

2.5 Cargar paquetes

```
library(readr)
library(dplyr)
library(ggplot2)
library(stringr)
source("util_graficas.R")
```

i Explicación del código

Se cargan paquetes para leer archivos CSV, transformar datos, trabajar con texto y crear gráficas. El archivo `util_graficas.R` contiene funciones visuales comunes para mantener un estilo uniforme en todo el libro.

2.6 Localizar el archivo

En lugar de depender únicamente de un nombre fijo, el siguiente código busca archivos cuyo nombre comience con `atus` y termine en `.csv`.

```
archivos_atus <- list.files(
  path = "datos",
  pattern = "^atus.*\\.csv$",
  full.names = TRUE,
  ignore.case = TRUE
)

archivos_atus
```

```
#> [1] "datos/atus_2024.csv"      "datos/atus_ml_preparado.csv"
```

💡 Interpretación

Si aparece al menos una ruta, R encontró un archivo ATUS. Si el resultado es `character(0)`, revise que el archivo esté descomprimido y guardado dentro de la carpeta `datos`.

2.7 Leer el archivo CSV

```
if (length(archivos_atus) == 0) {
  stop(
    paste(
      "No se encontró un archivo ATUS en la carpeta datos.",
      "Descárguelo desde el portal oficial del INEGI y descomprímalo."
    )
  )
}

ruta_atus <- archivos_atus[1]

atus <- read_csv(
  file = ruta_atus,
  show_col_types = FALSE,
  locale = locale(encoding = "UTF-8")
)

message("Archivo leído: ", ruta_atus)
```


i Explicación del código

Se selecciona el primer archivo localizado y se lee con `read_csv()`. El argumento `show_col_types = FALSE` evita imprimir mensajes técnicos que distraen de los resultados principales.

2.8 Primer vistazo a los datos

```
resumen_dimensiones <- data.frame(
  filas = nrow(atus),
  columnas = ncol(atus)
)

resumen_dimensiones
```

```
#>   filas columnas
#> 1 390627      45
```

```
head(atus)
```

```
#> # A tibble: 6 x 45
#>   COBERTURA ID_ENTIDAD ID_MUNICIPIO ANIO MES ID_HORA ID_MINUTO ID_DIA
#>   <chr>      <chr>      <chr>      <dbl> <chr> <chr>      <chr>      <chr>
#> 1 Municipal 01        001        2024 01    02        25        01
#> 2 Municipal 01        001        2024 01    03        40        01
#> 3 Municipal 01        001        2024 01    04        37        01
#> 4 Municipal 01        001        2024 01    05         0        01
#> 5 Municipal 01        001        2024 01    06        45        01
#> 6 Municipal 01        001        2024 01    07        30        01
#> # i 37 more variables: DIASEMANA <chr>, URBANA <chr>, SUBURBANA <chr>,
#> #   TIPACCID <chr>, AUTOMOVIL <dbl>, CAMPASAJ <dbl>, MICROBUS <dbl>,
#> #   PASCAMION <dbl>, OMNIBUS <dbl>, TRANVIA <dbl>, CAMIONETA <dbl>,
#> #   CAMION <dbl>, TRACTOR <dbl>, FERROCARRI <dbl>, MOTOCICLET <dbl>,
#> #   BICICLETA <dbl>, OTROVEHIC <dbl>, CAUSAACCI <chr>, CAPAROD <chr>,
#> #   SEXO <chr>, ALIENTO <chr>, CINTURON <chr>, ID_EDAD <dbl>, CONDMUERTO <dbl>,
#> #   CONDHERIDO <dbl>, PASAMUERTO <dbl>, PASAHERIDO <dbl>, PEATMUERTO <dbl>, ...
```

💡 Interpretación

Las filas representan registros de accidentes y las columnas describen sus características temporales, geográficas y operativas. Antes de modelar es indispensable confirmar que las variables esperadas estén disponibles.

2.9 Normalizar los nombres de variables

```
names(atus) <- toupper(names(atus))
names(atus)
```

```
#> [1] "COBERTURA"      "ID_ENTIDAD"      "ID_MUNICIPIO"    "ANIO"            "MES"
#> [6] "ID_HORA"         "ID_MINUTO"       "ID_DIA"          "DIASEMANA"       "URBANA"
#> [11] "SUBURBANA"       "TIPACCID"        "AUTOMOVIL"       "CAMPASAJ"        "MICROBUS"
#> [16] "PASCAMION"       "OMNIBUS"         "TRANVIA"         "CAMIONETA"       "CAMION"
#> [21] "TRACTOR"         "FERROCARRI"      "MOTOCICLET"      "BICICLETA"       "OTROVEHIC"
#> [26] "CAUSAACCI"       "CAPAROD"         "SEXO"            "ALIENTO"         "CINTURON"
#> [31] "ID_EDAD"         "CONDMUERTO"      "CONDHERIDO"      "PASAMUERTO"      "PASAHERIDO"
#> [36] "PEATMUERTO"      "PEATHERIDO"      "CICLMUERTO"      "CICLHERIDO"      "OTROMUERTO"
#> [41] "OTROHERIDO"      "NEMUERTO"        "NEHERIDO"        "CLASACC"         "ESTATUS"
```

Se convierten los nombres a mayúsculas para evitar errores por diferencias como Mes, MES o mes.

2.10 Seleccionar variables de interés

```
variables_interes <- c(
  "ANIO", "MES", "ID_ENTIDAD", "ID_MUNICIPIO",
  "ID_HORA", "ID_DIA", "DIASEMANA", "TIPACCID",
  "CAUSAACCI", "CLASACC"
)

variables_existentes <- intersect(
  variables_interes,
  names(atus)
)
```

```
variables_faltantes <- setdiff(
  variables_interes,
  names(atus)
)

atus_base <- atus |>
  select(all_of(variables_existentes))

variables_existentes
```

```
#> [1] "ANIO"          "MES"           "ID_ENTIDAD"    "ID_MUNICIPIO"  "ID_HORA"
#> [6] "ID_DIA"        "DIASEMANA"     "TIPACCID"      "CAUSAACCI"     "CLASACC"
```

```
variables_faltantes
```

```
#> character(0)
```

⚠ Compruebe el diccionario de datos

Los nombres y categorías pueden variar entre versiones. Si aparece una variable faltante, consulte los metadatos y el diccionario de datos descargados junto con la base antes de sustituirla.

2.11 Crear la variable respuesta

El primer problema de clasificación distinguirá entre:

- **Con víctimas:** accidentes fatales o no fatales con personas lesionadas o fallecidas.
- **Solo daños:** accidentes con daños materiales sin víctimas registradas.

```
atus_modelo <- atus_base |>
  mutate(
    CLASACC_TXT = str_to_lower(as.character(CLASACC)),
    accidente_con_victimas = case_when(
      str_detect(CLASACC_TXT, "sólo daños") ~ "Solo daños",
      str_detect(CLASACC_TXT, "solo daños") ~ "Solo daños",
      str_detect(CLASACC_TXT, "daños") ~ "Solo daños",
      str_detect(CLASACC_TXT, "no fatal") ~ "Con víctimas",
      str_detect(CLASACC_TXT, "fatal") ~ "Con víctimas",
    )
  )
```

```

    TRUE ~ NA_character_
  )
) |>
  filter(!is.na(accidente_con_victimas))

table(atus_modelo$accidente_con_victimas)

```

```

#>
#> Con víctimas      Solo daños
#>      68108      322519

```

i Explicación del código

La variable `CLASACC` se transforma en una respuesta binaria. `case_when()` asigna una nueva categoría según el texto encontrado y los registros que no pueden clasificarse se excluyen temporalmente.

2.12 Preparar las variables finales

```

atus_ml <- atus_modelo |>
  mutate(
    MES = as.factor(MES),
    ID_HORA = as.character(ID_HORA),
    DIASEMANA = as.factor(DIASEMANA),
    TIPACCID = as.factor(TIPACCID),
    CAUSAACCI = as.factor(CAUSAACCI),
    accidente_con_victimas = factor(
      accidente_con_victimas,
      levels = c("Con víctimas", "Solo daños")
    )
  ) |>
  select(
    accidente_con_victimas,
    MES,
    ID_HORA,
    DIASEMANA,
    TIPACCID,
    CAUSAACCI
  ) |>
  na.omit()

data.frame(

```

```

filas = nrow(atus_ml),
columnas = ncol(atus_ml)
)

```

```

#>      filas columnas
#> 1 390627         6

```

💡 Interpretación

La base final queda lista para el análisis exploratorio y el modelado. Cada fila representa un accidente y cada columna una variable predictora o la respuesta que se desea clasificar.

2.13 Guardar la base preparada

```

if (!dir.exists("datos")) {
  dir.create("datos")
}

write_csv(
  atus_ml,
  "datos/atus_ml_preparado.csv"
)

```

i Explicación del código

Guardar la base preparada evita repetir toda la limpieza en los capítulos siguientes y ayuda a mantener reproducible el flujo de trabajo.

2.14 Lista de comprobación

Antes de continuar, verifique que:

- el archivo proviene del portal oficial del INEGI;
- el año de los datos está documentado;
- el CSV se encuentra dentro de `datos`;
- las variables usadas existen en la versión descargada;
- la transformación de `CLASACC` corresponde con su diccionario;
- las tablas y gráficas incluyen la fuente;
- el archivo `atus_ml_preparado.csv` se creó correctamente.

2.15 Resumen del capítulo

En este capítulo descargamos y documentamos una base real del INEGI, verificamos su ubicación, seleccionamos variables, construimos una respuesta binaria y guardamos una base preparada para aprendizaje automático.

2.16 Materiales complementarios del capítulo

Estos recursos permiten repasar la preparación de datos reales mediante una presentación, una infografía y un video explicativo.

Recurso	Utilidad	Abrir o reproducir	Descargar
Video explicativo	Explicación audiovisual del proceso de descarga, revisión, limpieza y preparación de datos.	Ver en YouTube	—
Presentación en PDF	Diapositivas para lectura, estudio o exposición.	Ver PDF	Descargar PDF
Presentación editable	Archivo PowerPoint para utilizarlo en clase o adaptarlo.	Abrir PPTX	Descargar PPTX
Infografía	Resumen visual de las etapas de preparación de datos.	Ver infografía	Descargar PNG

Video del capítulo: <https://www.youtube.com/watch?v=bmv-tDPZGjQ>

La presentación PDF, el archivo editable y la infografía pueden descargarse desde la versión web del libro.

Elaboración de los materiales

Los materiales complementarios fueron elaborados con apoyo de **NotebookLM de Google**, a partir del contenido del capítulo, y posteriormente revisados y adaptados por el autor. El texto del libro y sus archivos fuente constituyen la referencia principal.

Curso completo en YouTube

Este video forma parte de la lista oficial del curso **Aprendizaje y Clasificación Automática con R**.

[Consultar todos los videos del curso](#)

Capítulo 3

Análisis exploratorio de datos

3.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de cargar una base preparada, revisar su estructura, calcular frecuencias, construir gráficas e interpretar patrones iniciales.

3.2 Cargar paquetes y base

```
library(readr)
library(dplyr)
library(ggplot2)
source("util_graficas.R")

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
} else {
  atus_ml <- NULL
  print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo
↪ 2.")
}
```

```
#> [1] "Base preparada cargada correctamente."
```

i Explicación del código

Se carga la base preparada en el capítulo anterior. Esto permite empezar directamente con el análisis exploratorio.

3.3 Preparar variables

```
if (!is.null(atus_ml)) {
  atus_ml <- atus_ml |>
  mutate(
    ID_HORA_NUM = as.numeric(ID_HORA),
    MES_NUM = as.numeric(MES),
    MES_FACTOR = factor(sprintf("%02d", MES_NUM), levels = sprintf("%02d", 1:12)),
    accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas",
  ↪  "Solo daños"))
  )
}
```

i Explicación del código

Se crean variables numéricas y factores ordenados para facilitar tablas y gráficas.

3.4 Distribución de la variable respuesta

```
if (!is.null(atus_ml)) {
  tabla_clase <- table(atus_ml$accidente_con_victimas)
  round(100 * prop.table(tabla_clase), 2)
}
```

```
#>
#> Con víctimas      Solo daños
#>          17.44          82.56
```

```
if (!is.null(atus_ml)) {
  ggplot(atus_ml, aes(x = accidente_con_victimas, fill = accidente_con_victimas)) +
    geom_bar(width = 0.7) +
    escala_clases_fill() +
    scale_y_continuous(labels = etiqueta_numero) +
    labs(
      title = "Distribución de accidentes según presencia de víctimas",
      subtitle = "Comparación entre clases",
      x = "Clase",
      y = "Número de accidentes",
    )
}
```

```
fill = "Clase"
) +
tema_libro() +
theme(legend.position = "none")
}
```

Distribución de accidentes según presencia de víctimas

Comparación entre clases



💡 Interpretación del resultado

La base está desbalanceada: hay más accidentes de solo daños que accidentes con víctimas. Esto será importante al evaluar modelos.

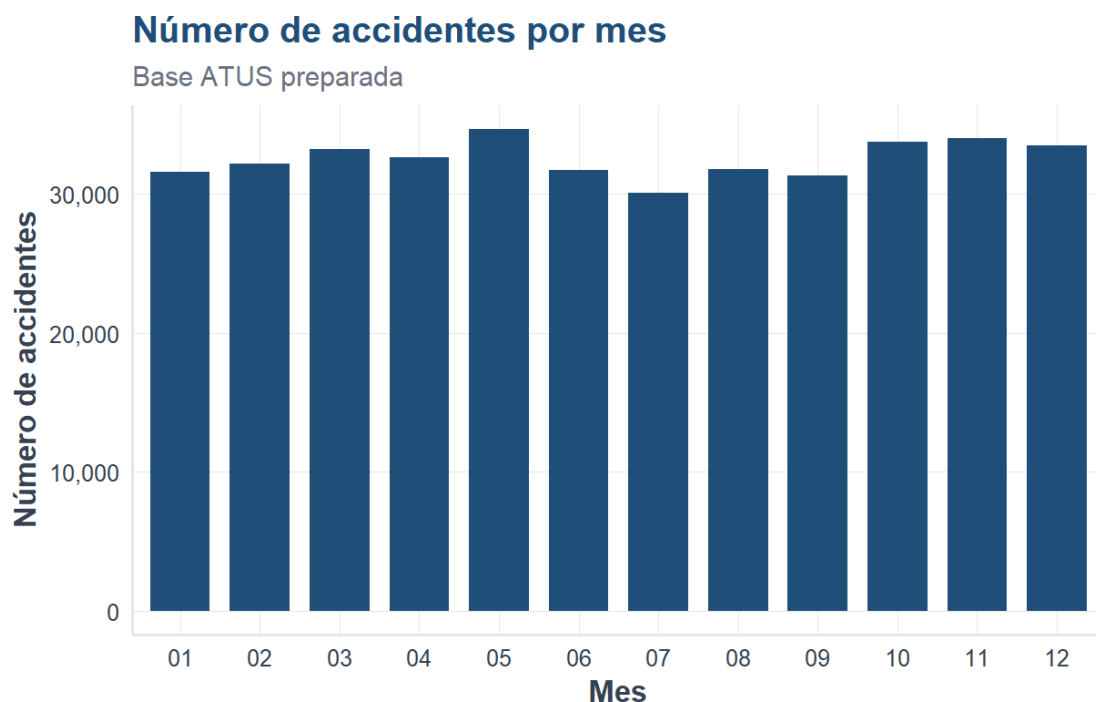
3.5 Accidentes por mes

```
if (!is.null(atus_ml)) {
  accidentes_mes <- atus_ml |> count(MES_FACTOR, name = "n") |> arrange(MES_FACTOR)
  accidentes_mes
}
```

```
#> # A tibble: 12 x 2
```

```
#>   MES_FACTOR    n
#>   <fct>      <int>
#> 1 01        31600
#> 2 02        32208
#> 3 03        33237
#> 4 04        32666
#> 5 05        34665
#> 6 06        31737
#> 7 07        30062
#> 8 08        31800
#> 9 09        31351
#> 10 10       33771
#> 11 11       34047
#> 12 12       33483
```

```
if (exists("accidentes_mes")) {
  ggplot(accidentes_mes, aes(x = MES_FACTOR, y = n)) +
    geom_col(fill = col_azul, width = 0.75) +
    scale_y_continuous(labels = etiqueta_numero) +
    labs(
      title = "Número de accidentes por mes",
      subtitle = "Base ATUS preparada",
      x = "Mes",
      y = "Número de accidentes"
    ) +
    tema_libro()
}
```



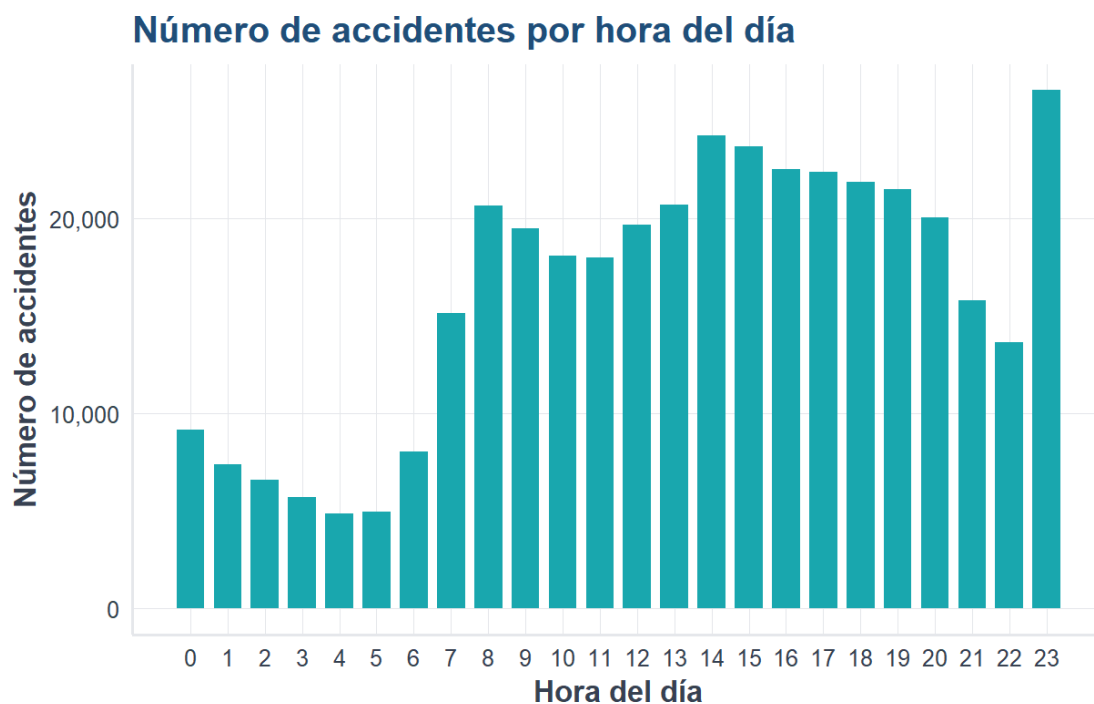
i Explicación del código

Se cuentan los accidentes por mes y se visualizan con barras. El eje vertical usa separadores de miles para facilitar la lectura.

3.6 Accidentes por hora del día

```
if (!is.null(atus_ml)) {
  accidentes_hora <- atus_ml |> count(ID_HORA_NUM, name = "n") |> arrange(ID_HORA_NUM)

  ggplot(accidentes_hora, aes(x = ID_HORA_NUM, y = n)) +
    geom_col(fill = col_turquesa, width = 0.75) +
    scale_x_continuous(breaks = 0:23) +
    scale_y_continuous(labels = etiqueta_numero) +
    labs(
      title = "Número de accidentes por hora del día",
      x = "Hora del día",
      y = "Número de accidentes"
    ) +
    tema_libro()
}
```



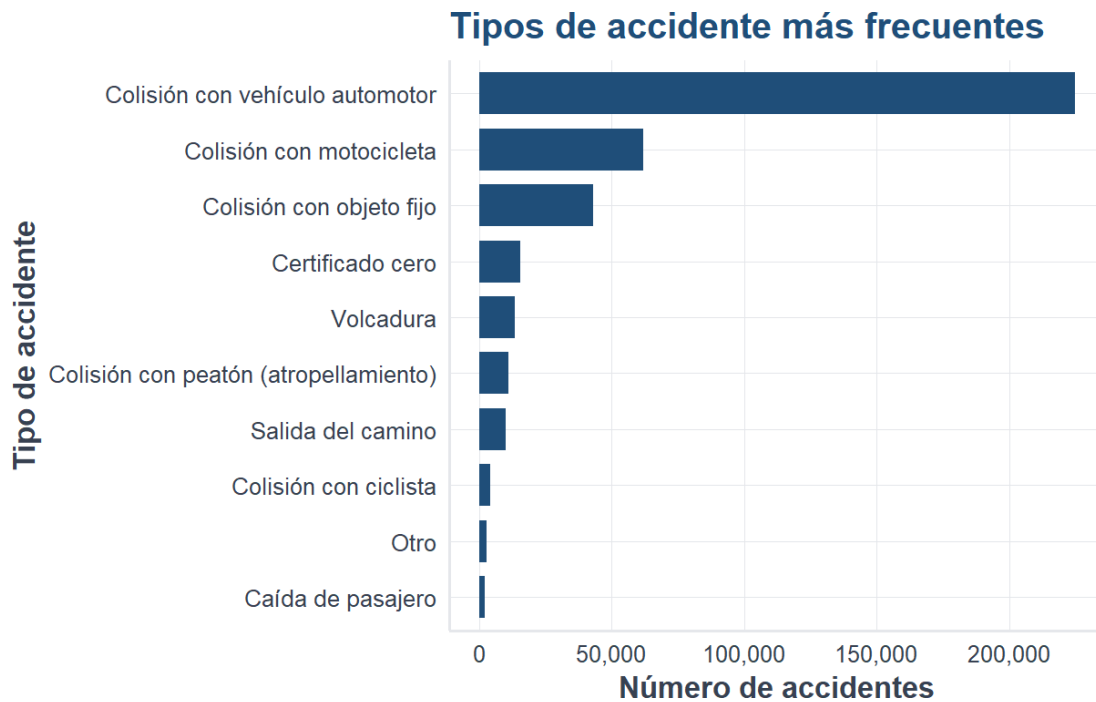
💡 Interpretación del resultado

La gráfica permite detectar horas con mayor concentración de accidentes.

3.7 Tipos de accidente más frecuentes

```
if (!is.null(atus_ml)) {
  accidentes_tipo_top <- atus_ml |> count(TIPACCID, name = "n") |> slice_max(n, n = 10)

  ggplot(accidentes_tipo_top, aes(x = reorder(TIPACCID, n), y = n)) +
    geom_col(fill = col_azul, width = 0.75) +
    coord_flip() +
    scale_y_continuous(labels = etiqueta_numero) +
    labs(
      title = "Tipos de accidente más frecuentes",
      x = "Tipo de accidente",
      y = "Número de accidentes"
    ) +
    tema_libro()
}
```



i Explicación del código

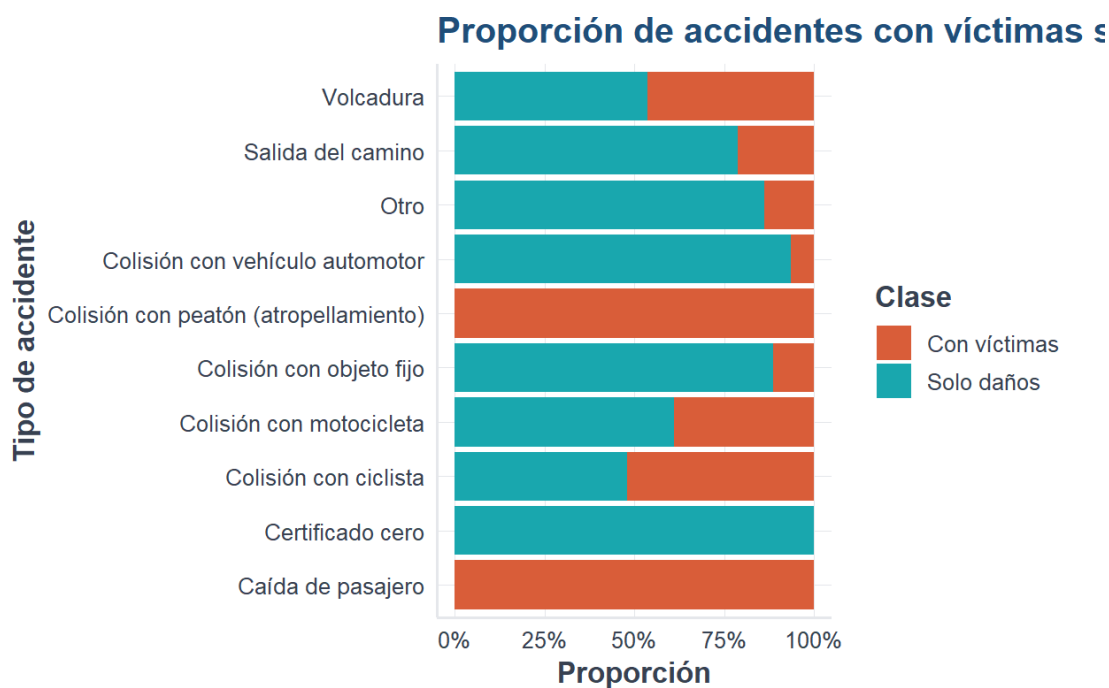
Se seleccionan los diez tipos de accidente con mayor frecuencia para evitar una gráfica saturada.

3.8 Proporción por tipo de accidente

```
if (!is.null(atus_ml)) {
  tipos_top <- atus_ml |> count(TIPACCID) |> slice_max(n, n = 10) |> pull(TIPACCID)
  atus_tipo_top <- atus_ml |> filter(TIPACCID %in% tipos_top)

  ggplot(atus_tipo_top, aes(x = TIPACCID, fill = accidente_con_victimas)) +
    geom_bar(position = "fill") +
    coord_flip() +
    escala_clases_fill() +
    scale_y_continuous(labels = etiqueta_porcentaje) +
    labs(
      title = "Proporción de accidentes con víctimas según tipo de accidente",
      x = "Tipo de accidente",
      y = "Proporción",
      fill = "Clase"
    ) +
}
```

```
tema_libro()
}
```



💡 Interpretación del resultado

Esta gráfica compara proporciones, no cantidades absolutas. Permite identificar tipos de accidente con mayor presencia relativa de víctimas.

3.9 Materiales complementarios del capítulo

Estos recursos permiten repasar los conceptos esenciales del análisis exploratorio de datos mediante distintos formatos.

Recurso	Utilidad	Abrir o reproducir	Descargar
Video explicativo	Explicación audiovisual del análisis exploratorio de datos y de sus principales herramientas.	Ver en YouTube	—

Recurso	Utilidad	Abrir o reproducir	Descargar
Presentación en PDF	Diapositivas para lectura, estudio o exposición.	Ver PDF	Descargar PDF
Presentación editable	Archivo PowerPoint para utilizarlo en clase o adaptarlo.	Abrir PPTX	Descargar PPTX
Infografía	Síntesis visual de conceptos, procedimientos y gráficos del capítulo.	Ver infografía	Descargar PNG

Video del capítulo: <https://www.youtube.com/watch?v=U2hd8iEovCY>

La presentación PDF, el archivo editable y la infografía pueden descargarse desde la versión web del libro.



Curso completo en YouTube

Este video forma parte de la lista oficial del curso **Aprendizaje y Clasificación Automática con R**.

[Consultar todos los videos del curso](#)



Elaboración de los materiales

Los materiales complementarios fueron elaborados con apoyo de **NotebookLM de Google**, a partir del contenido del capítulo, y posteriormente revisados y adaptados por el autor. El texto del libro y sus archivos fuente constituyen la referencia principal.

3.10 Conclusión

El análisis exploratorio permite detectar patrones iniciales, revisar el desbalance de clases e identificar variables útiles para modelar.

Capítulo 4

Regresión lineal simple y múltiple

4.1 Objetivos del capítulo

Al terminar este capítulo, el lector podrá:

- distinguir entre regresión y clasificación;
- interpretar una relación lineal entre una variable respuesta y una variable explicativa;
- ajustar una regresión lineal simple en R;
- construir e interpretar un modelo de regresión lineal múltiple;
- evaluar el ajuste mediante R^2 , R^2 ajustado, RMSE y análisis de residuos;
- reconocer cuándo la regresión lineal no es apropiada.

4.2 ¿Por qué estudiar regresión lineal en un libro de clasificación?

La regresión lineal no es un algoritmo de clasificación: su respuesta es numérica y continua. Sin embargo, es una base fundamental del aprendizaje supervisado porque introduce ideas que reaparecen en otros modelos: función de predicción, coeficientes, error, ajuste, generalización y evaluación fuera de muestra.

Además, permite comprender mejor la regresión logística del capítulo siguiente. Ambos modelos construyen una combinación de variables explicativas, aunque difieren en la naturaleza de la respuesta y en la función utilizada para obtener la predicción (James et al. 2021).

i Idea central

En regresión lineal se predice una cantidad, por ejemplo el número mensual de accidentes. En clasificación se predice una categoría, por ejemplo si un accidente tuvo víctimas o solo daños.

4.3 Regresión lineal simple

La regresión lineal simple relaciona una variable respuesta Y con una sola variable explicativa X :

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i.$$

Donde:

- β_0 es la ordenada al origen;
- β_1 representa el cambio promedio esperado en Y cuando X aumenta una unidad;
- ε_i representa la parte no explicada por el modelo.

Los coeficientes se estiman normalmente mediante mínimos cuadrados, buscando minimizar la suma de los errores al cuadrado (Montgomery et al. 2021).

4.4 Primer ejemplo didáctico

Supongamos que se desea relacionar el número de horas de estudio con la calificación obtenida.

```
horas <- c(2, 3, 4, 5, 6, 7, 8, 9)
calificacion <- c(55, 60, 64, 70, 75, 79, 86, 90)

datos_estudio <- data.frame(
  horas = horas,
  calificacion = calificacion
)

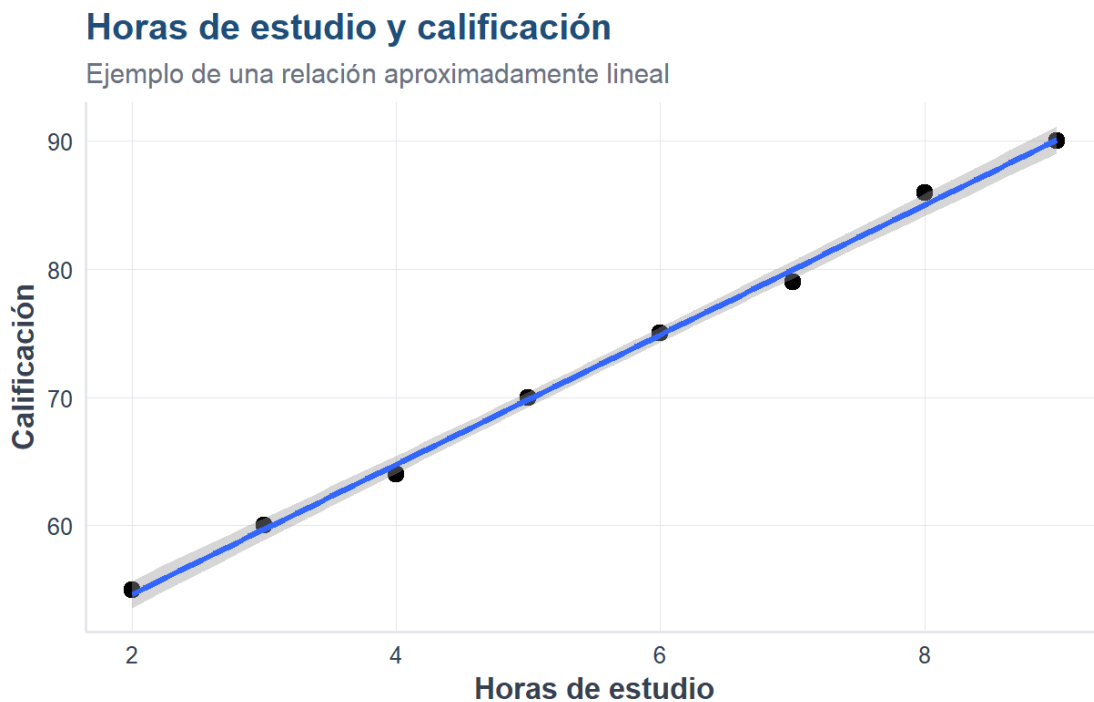
datos_estudio
```

```
#>   horas calificacion
#> 1     2           55
#> 2     3           60
#> 3     4           64
#> 4     5           70
#> 5     6           75
#> 6     7           79
#> 7     8           86
#> 8     9           90
```

4.5 Visualizar la relación

```
library(ggplot2)
source("util_graficas.R")

ggplot(datos_estudio, aes(x = horas, y = calificacion)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = TRUE) +
  labs(
    title = "Horas de estudio y calificación",
    subtitle = "Ejemplo de una relación aproximadamente lineal",
    x = "Horas de estudio",
    y = "Calificación"
  ) +
  tema_libro()
```



4.6 Ajustar el modelo simple

```
modelo_simple <- lm(
```

```
calificacion ~ horas,
data = datos_estudio
)

summary(modelo_simple)
```

```
#>
#> Call:
#> lm(formula = calificacion ~ horas, data = datos_estudio)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -0.9643 -0.2589  0.1250  0.2887  0.9762
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  44.5476     0.6197   71.88 4.88e-10 ***
#> horas         5.0595     0.1040   48.64 5.06e-09 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.6741 on 6 degrees of freedom
#> Multiple R-squared:  0.9975, Adjusted R-squared:  0.997
#> F-statistic: 2366 on 1 and 6 DF,  p-value: 5.061e-09
```

4.7 Interpretar los coeficientes

```
coeficientes_simple <- coef(modelo_simple)
coeficientes_simple
```

```
#> (Intercept)      horas
#>  44.547619    5.059524
```

El modelo estimado tiene la forma:

$$\widehat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X.$$

La pendiente indica cuántos puntos cambia, en promedio, la calificación por cada hora adicional de estudio. La ordenada al origen es necesaria matemáticamente, aunque no siempre tenga una interpretación práctica dentro del rango observado.

4.8 Realizar una predicción

```
nuevo_estudiante <- data.frame(horas = 6.5)

predict(
  modelo_simple,
  newdata = nuevo_estudiante,
  interval = "prediction",
  level = 0.95
)
```

```
#>      fit      lwr      upr
#> 1 77.43452 75.66668 79.20237
```

El intervalo de predicción es más amplio que un intervalo para la media porque considera tanto la incertidumbre del modelo como la variabilidad individual.

4.9 Residuos

El residuo de la observación i es:

$$e_i = y_i - \hat{y}_i.$$

```
diagnostico_simple <- data.frame(
  observado = datos_estudio$calificacion,
  predicho = fitted(modelo_simple),
  residuo = residuals(modelo_simple)
)

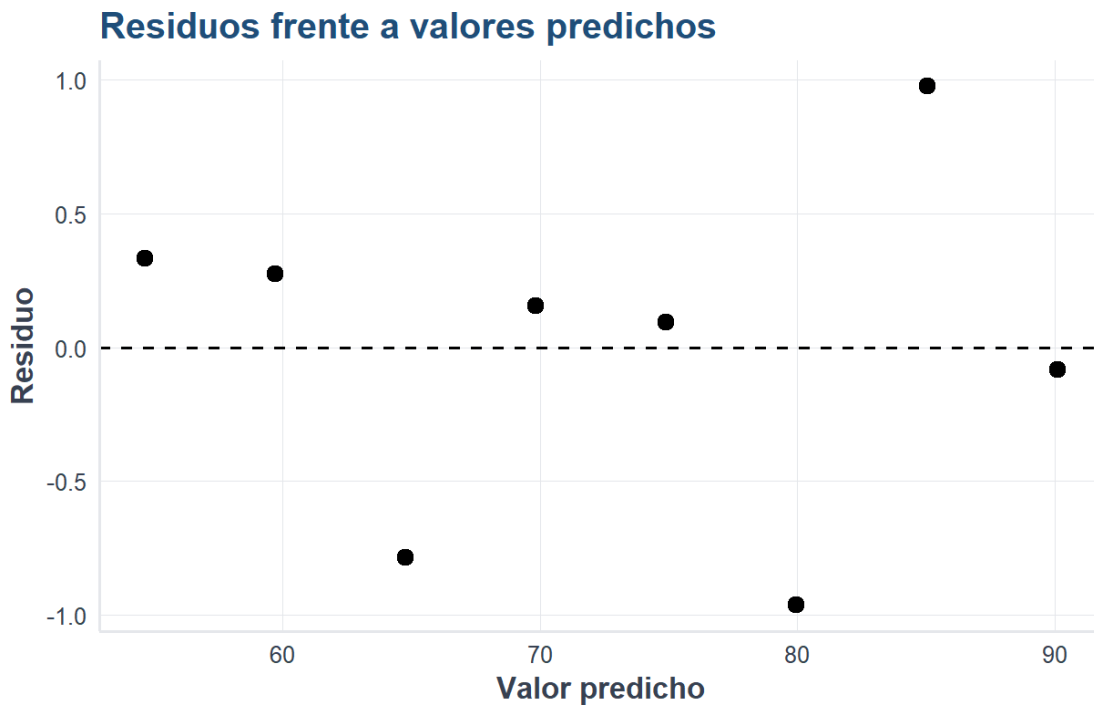
diagnostico_simple
```

```
#>  observado predicho      residuo
#> 1         55 54.66667 0.33333333
#> 2         60 59.72619 0.27380952
```

```
#> 3      64 64.78571 -0.78571429
#> 4      70 69.84524  0.15476190
#> 5      75 74.90476  0.09523810
#> 6      79 79.96429 -0.96428571
#> 7      86 85.02381  0.97619048
#> 8      90 90.08333 -0.08333333
```

4.10 Visualizar los residuos

```
ggplot(diagnostico_simple, aes(x = predicho, y = residuo)) +
  geom_hline(yintercept = 0, linetype = 2) +
  geom_point(size = 3) +
  labs(
    title = "Residuos frente a valores predichos",
    x = "Valor predicho",
    y = "Residuo"
  ) +
  tema_libro()
```



Una nube sin patrón claro alrededor de cero es compatible con una relación lineal razonable. Curvaturas, forma de embudo o puntos extremos sugieren revisar el modelo.

4.11 Regresión lineal múltiple

La regresión múltiple incorpora varias variables explicativas:

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \dots + \beta_p X_{ip} + \varepsilon_i.$$

Cada coeficiente representa el cambio esperado en la respuesta cuando la variable correspondiente aumenta una unidad, **manteniendo constantes las demás variables**.

4.12 Preparar un ejemplo múltiple

```
set.seed(2026)

n <- 80

datos_ambientales <- data.frame(
  temperatura = runif(n, 12, 35),
  velocidad_viento = runif(n, 0.5, 8),
  trafico = runif(n, 100, 900)
)

datos_ambientales$pm10 <-
  18 +
  0.9 * datos_ambientales$temperatura -
  2.1 * datos_ambientales$velocidad_viento +
  0.045 * datos_ambientales$trafico +
  rnorm(n, mean = 0, sd = 6)

head(datos_ambientales)
```

```
#>   temperatura velocidad_viento  trafico    pm10
#> 1    28.06949         1.426882 199.1416 60.66469
#> 2    24.80020         7.636602 504.3138 59.76823
#> 3    15.22322         6.362863 847.4719 57.86386
#> 4    18.57164         7.716020 190.2525 32.45387
#> 5    24.77349         1.356889 199.1549 35.97589
#> 6    12.57802         7.831641 402.8500 33.81511
```

4.13 Ajustar el modelo múltiple

```
modelo_multiple <- lm(
  pm10 ~ temperatura + velocidad_viento + trafico,
  data = datos_ambientales
)

summary(modelo_multiple)
```

```
#>
#> Call:
#> lm(formula = pm10 ~ temperatura + velocidad_viento + trafico,
#>     data = datos_ambientales)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -11.4969  -4.0965   0.1626   3.3999  12.7856
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept)    13.831686    2.970644   4.656 1.34e-05 ***
#> temperatura     1.016287    0.095374  10.656 < 2e-16 ***
#> velocidad_viento -1.681322    0.282650  -5.948 7.75e-08 ***
#> trafico          0.047190    0.002486  18.982 < 2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 5.446 on 76 degrees of freedom
#> Multiple R-squared:  0.866, Adjusted R-squared:  0.8607
#> F-statistic: 163.7 on 3 and 76 DF, p-value: < 2.2e-16
```

4.14 Interpretar los coeficientes múltiples

```
tabla_coeficientes <- data.frame(
  termino = names(coef(modelo_multiple)),
  estimacion = as.numeric(coef(modelo_multiple))
)
```



```
tabla_coeficientes
```

```
#>           termino  estimacion
#> 1 (Intercept) 13.83168556
#> 2 temperatura  1.01628652
#> 3 velocidad_viento -1.68132208
#> 4 trafico      0.04718991
```

Por ejemplo, el coeficiente de `velocidad_viento` estima el cambio promedio de PM10 asociado con un aumento de una unidad en la velocidad del viento, manteniendo constantes la temperatura y el tráfico.

⚠ Asociación no significa causalidad

Un coeficiente estadísticamente significativo no demuestra por sí solo una relación causal. La causalidad requiere diseño, conocimiento sustantivo y control adecuado de variables de confusión.

4.15 Separar entrenamiento y prueba

Evaluar el modelo con los mismos datos usados para ajustarlo produce una visión demasiado optimista. Por ello se separan datos de entrenamiento y prueba.

```
set.seed(2026)

indices_entrenamiento <- sample(
  seq_len(nrow(datos_ambientales)),
  size = floor(0.8 * nrow(datos_ambientales))
)

entrenamiento_reg <- datos_ambientales[indices_entrenamiento, ]
prueba_reg <- datos_ambientales[-indices_entrenamiento, ]

modelo_multiple_prueba <- lm(
  pm10 ~ temperatura + velocidad_viento + trafico,
  data = entrenamiento_reg
)

prediccion_reg <- predict(
  modelo_multiple_prueba,
  newdata = prueba_reg
)
```

4.16 Métricas de evaluación

El error cuadrático medio y su raíz se calculan como:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

```
rmse <- sqrt(mean((prueba_reg$pm10 - prediccion_reg)^2))
mae <- mean(abs(prueba_reg$pm10 - prediccion_reg))

metricas_regresion <- data.frame(
  metrica = c("RMSE", "MAE"),
  valor = c(rmse, mae)
)

metricas_regresion
```

```
#>   metrica   valor
#> 1   RMSE 4.290665
#> 2    MAE 3.749943
```

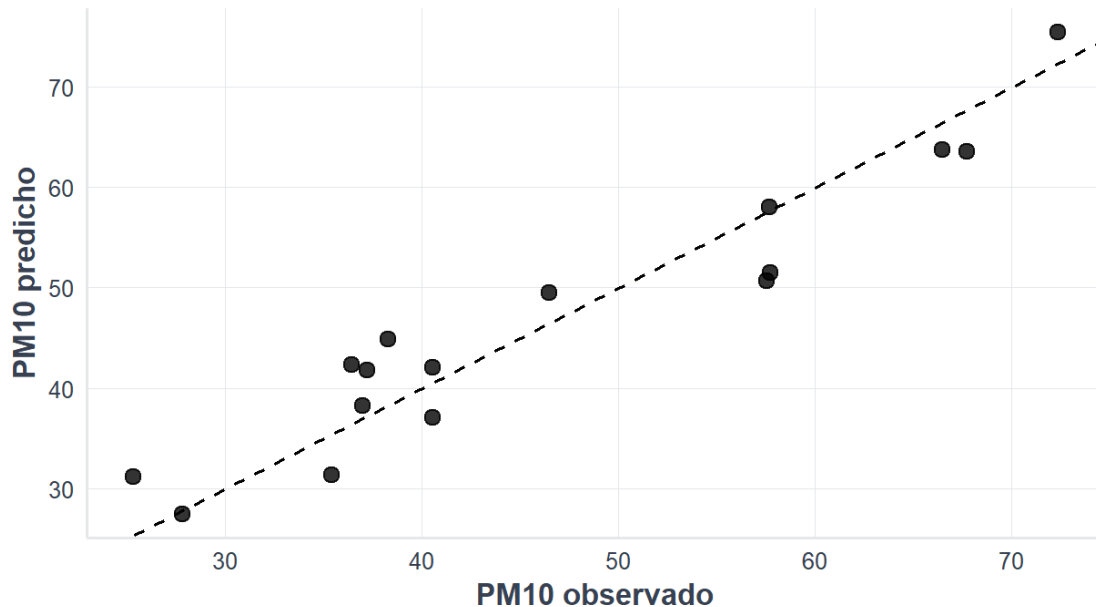
4.17 Observado frente a predicho

```
comparacion_regresion <- data.frame(
  observado = prueba_reg$pm10,
  predicho = prediccion_reg
)

ggplot(comparacion_regresion, aes(x = observado, y = predicho)) +
  geom_point(size = 3, alpha = 0.8) +
  geom_abline(slope = 1, intercept = 0, linetype = 2) +
  labs(
    title = "Valores observados y predichos",
    subtitle = "La línea diagonal representa predicción perfecta",
    x = "PM10 observado",
    y = "PM10 predicho"
  ) +
  tema_libro()
```

Valores observados y predichos

La línea diagonal representa predicción perfecta



4.18 Aplicación con datos ATUS agregados

En ATUS cada fila corresponde a un accidente. La base preparada del libro conserva las variables necesarias para los algoritmos de clasificación, pero no incluye entidad ni municipio. Por ello, en este ejemplo los registros se agregan por mes y se modela el número mensual de accidentes registrados.

```
library(readr)
library(dplyr)

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (!file.exists(ruta_atus_ml)) {
  stop(
    paste(
      "No se encontró datos/atus_ml_preparado.csv.",
      "Ejecute primero la celda de preparación automática."
    )
  )
}

atus_reg <- read_csv(
  ruta_atus_ml,
  show_col_types = FALSE
)
```

```
)
names(atus_reg)
```

```
#> [1] "accidente_con_victimas" "MES" "ID_HORA"
#> [4] "DIASEMANA" "TIPACCID" "CAUSAACCI"
```

4.19 Construir una base mensual

```
variables_necesarias <- c(
  "MES", "ID_HORA", "DIASEMANA", "TIPACCID",
  "accidente_con_victimas"
)

faltantes <- setdiff(variables_necesarias, names(atus_reg))

if (length(faltantes) > 0) {
  stop(
    paste(
      "Faltan variables para el ejemplo:",
      paste(faltantes, collapse = ", ")
    )
  )
}

atus_mensual <- atus_reg |>
  mutate(
    ID_HORA = as.numeric(ID_HORA),
    MES = as.numeric(MES),
    fin_semana = DIASEMANA %in% c(
      "sábado", "sabado", "domingo",
      "Sábado", "Sabado", "Domingo"
    ),
    con_victimas = accidente_con_victimas == "Con víctimas"
  ) |>
  group_by(MES) |>
  summarise(
    accidentes = n(),
    tipos_accidente = n_distinct(TIPACCID, na.rm = TRUE),
    hora_promedio = mean(ID_HORA, na.rm = TRUE),
    proporcion_fin_semana = mean(fin_semana, na.rm = TRUE),
    proporcion_con_victimas = mean(con_victimas, na.rm = TRUE),
    .groups = "drop"
  ) |>
  filter(
```

```

is.finite(accidentes),
is.finite(tipos_accidente),
is.finite(hora_promedio),
is.finite(proporcion_fin_semana),
is.finite(proporcion_con_victimas)
)

atus_mensual

```

```

#> # A tibble: 12 x 6
#>       MES accidentes tipos_accidente hora_promedio proporcion_fin_semana
#>   <dbl>      <int>      <int>      <dbl>      <dbl>
#> 1     1         31600          13         13.6         0.252
#> 2     2         32208          13         13.7         0.271
#> 3     3         33237          13         13.7         0.320
#> 4     4         32666          13         13.7         0.272
#> 5     5         34665          13         13.8         0.260
#> 6     6         31737          13         13.7         0.338
#> 7     7         30062          13         13.7         0.270
#> 8     8         31800          13         13.8         0.293
#> 9     9         31351          13         13.6         0.292
#> 10    10         33771          13         13.6         0.251
#> 11    11         34047          13         13.6         0.293
#> 12    12         33483          13         13.6         0.291
#> # i 1 more variable: proporcion_con_victimas <dbl>

```

4.20 Ajustar la regresión múltiple con ATUS

```

modelo_atus_reg <- lm(
  accidentes ~ MES + tipos_accidente +
    hora_promedio + proporcion_fin_semana,
  data = atus_mensual
)

summary(modelo_atus_reg)

```

```

#>
#> Call:
#> lm(formula = accidentes ~ MES + tipos_accidente + hora_promedio +

```

```
#>      proporcion_fin_semana, data = atus_mensual)
#>
#> Residuals:
#>      Min        1Q    Median        3Q        Max
#> -2402.87  -610.94    75.52   707.04  2319.88
#>
#> Coefficients: (1 not defined because of singularities)
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept)      71520.81   93274.17   0.767   0.465
#> MES              65.83     140.08   0.470   0.651
#> tipos_accidente      NA         NA      NA      NA
#> hora_promedio    -2749.42    6808.57  -0.404   0.697
#> proporcion_fin_semana -6359.05  17099.04  -0.372   0.720
#>
#> Residual standard error: 1487 on 8 degrees of freedom
#> Multiple R-squared:  0.08946,    Adjusted R-squared:  -0.252
#> F-statistic: 0.262 on 3 and 8 DF,  p-value: 0.8509
```



Limitación del ejemplo

El número de accidentes es una variable de conteo. La regresión lineal se utiliza aquí con fines didácticos; en un estudio formal podrían ser más apropiados modelos de Poisson o binomial negativa. Esta comparación ayuda a comprender que la elección del algoritmo depende de la naturaleza de la respuesta.

4.21 Supuestos principales

La interpretación inferencial tradicional de la regresión lineal se apoya en varios supuestos:

1. relación aproximadamente lineal;
2. errores independientes;
3. varianza aproximadamente constante;
4. ausencia de colinealidad extrema;
5. normalidad aproximada de los errores cuando se construyen pruebas e intervalos.

Los supuestos deben analizarse mediante gráficas, conocimiento del problema y medidas diagnósticas; no deben tratarse como una lista mecánica.

4.22 Diferencias entre regresión lineal y logística

Característica	Regresión lineal	Regresión logística
Respuesta	Númerica continua	Categorica binaria
Salida directa	Cualquier número real	Probabilidad entre 0 y 1
Función usual	Identidad	Logística
Evaluación	RMSE, MAE, R^2	Matriz de confusión, sensibilidad, especificidad, AUC
Ejemplo	Predecir PM10	Clasificar accidente con víctimas

4.23 Laboratorio interactivo: regresión lineal

4.23.1 Laboratorio disponible en la versión web

El laboratorio permite modificar intercepto, pendiente, ruido, tamaño de muestra y semilla, y comparar la recta verdadera con la estimada.

4.24 Actividades para el lector

1. Modifique el ejemplo simple y prediga la calificación para 7.5 horas de estudio.
2. Retire una variable del modelo ambiental y compare el R^2 ajustado y el RMSE.
3. Añada una interacción entre temperatura y velocidad del viento.
4. Analice los residuos del modelo múltiple.
5. Explique por qué no sería correcto utilizar regresión lineal ordinaria para predecir directamente una categoría como `Con víctimas` o `Solo daños`.
6. Compare conceptualmente el modelo ATUS de este capítulo con la regresión logística del capítulo siguiente.

4.25 Conclusiones

La regresión lineal simple explica una respuesta cuantitativa mediante una variable y la regresión múltiple permite considerar varios factores simultáneamente. Su valor en este libro no se limita a la predicción numérica: proporciona el lenguaje básico para comprender coeficientes, errores, validación y generalización, conceptos que continuarán apareciendo en los algoritmos de clasificación.

4.26 Materiales complementarios del capítulo

Recurso	Utilidad	Abrir o reproducir	Descargar
Video explicativo	Explicación audiovisual de la regresión lineal simple y múltiple.	Ver en YouTube	—
Presentación en PDF	Diapositivas para lectura, estudio o exposición.	Ver PDF	Descargar PDF
Presentación editable	Archivo PowerPoint para utilizarlo en clase o adaptarlo.	Abrir PPTX	Descargar PPTX
Infografía	Síntesis visual de conceptos, fórmulas e interpretación.	Ver infografía	Descargar PNG

Video del capítulo: <https://www.youtube.com/watch?v=SE6DCeU9h1w>

Curso completo en YouTube: <https://www.youtube.com/playlist?list=PLDJYd2v7Kt-Q>

 Curso completo en YouTube

[Consultar todos los videos del curso](#)

Capítulo 5

Regresión logística para clasificación

5.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de ajustar una regresión logística, predecir probabilidades y evaluar el desempeño.

5.2 Fundamento matemático

La regresión logística utiliza la función:

$$p = \frac{1}{1 + e^{-z}}$$

donde $z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$.

5.3 Visualización de la función logística

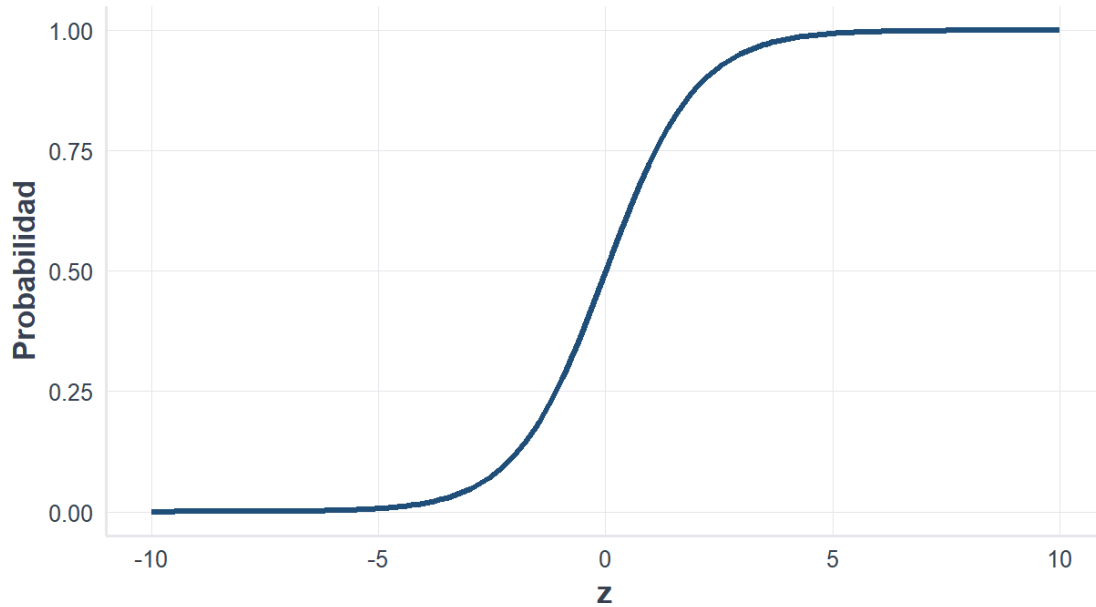
```
library(ggplot2)
source("util_graficas.R")

z <- seq(-10, 10, length.out = 200)
datos_logistica <- data.frame(z = z, p = 1 / (1 + exp(-z)))

ggplot(datos_logistica, aes(x = z, y = p)) +
  geom_line(linewidth = 1.2, color = col_azul) +
  labs(
    title = "Función logística",
    subtitle = "Transforma valores reales en probabilidades",
    x = "z",
    y = "Probabilidad"
  ) +
  tema_libro()
```

Función logística

Transforma valores reales en probabilidades



Explicación del código

Se genera una secuencia de valores para z y se calcula su probabilidad usando la función logística.

Interpretación del resultado

La curva tiene forma de S. Valores negativos producen probabilidades cercanas a cero y valores positivos probabilidades cercanas a uno.

5.4 Cargar paquetes y datos

```
library(readr)
library(dplyr)
source("util_graficas.R")

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
}
```

```

} else {
  atus_ml <- NULL
  print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo
  ↪ 2.")
}

```

```
#> [1] "Base preparada cargada correctamente."
```

5.5 Preparar y dividir datos

```

if (!is.null(atus_ml)) {
  atus_modelo <- atus_ml |>
  mutate(
    victimas_binaria = ifelse(accidente_con_victimas == "Con víctimas", 1, 0),
    MES = as.factor(MES),
    ID_HORA = as.numeric(ID_HORA),
    DIASEMANA = as.factor(DIASEMANA),
    TIPACCID = as.factor(TIPACCID),
    CAUSAACCI = as.factor(CAUSAACCI),
    accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas",
  ↪  "Solo daños"))
  )

  set.seed(123)
  idx <- sample(1:nrow(atus_modelo), size = round(0.7 * nrow(atus_modelo)))
  entrenamiento <- atus_modelo[idx, ]
  prueba <- atus_modelo[-idx, ]
}

```

i Explicación del código

La variable respuesta se transforma a binaria y la base se divide en entrenamiento y prueba.

5.6 Ajustar modelo

```

if (exists("entrenamiento")) {
  modelo_logistico <- glm(

```

```
victimas_binaria ~ MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
data = entrenamiento,
family = binomial
)

head(summary(modelo_logistico)$coefficients, 12)
}
```

```
#>               Estimate Std. Error   z value    Pr(>|z|)
#> (Intercept) 18.107257168 101.84719297  0.17778848 8.588891e-01
#> MES02      -0.045765700  0.02975556 -1.53805548 1.240350e-01
#> MES03      -0.032191364  0.02927313 -1.09968980 2.714673e-01
#> MES04      -0.002838049  0.02945462 -0.09635327 9.232400e-01
#> MES05      -0.015408552  0.02906672 -0.53010979 5.960358e-01
#> MES06      -0.054753129  0.02983667 -1.83509526 6.649158e-02
#> MES07      -0.076296781  0.03035194 -2.51373620 1.194598e-02
#> MES08      -0.094049541  0.02997105 -3.13801278 1.700975e-03
#> MES09      -0.077109438  0.03015044 -2.55748998 1.054306e-02
#> MES10      -0.154152374  0.02979697 -5.17342485 2.298416e-07
#> MES11      -0.106385033  0.02950811 -3.60528073 3.118157e-04
#> MES12      -0.111888509  0.02959338 -3.78086333 1.562855e-04
```

Interpretación del resultado

Los coeficientes estiman el efecto de cada variable sobre el logit de la probabilidad de accidente con víctimas.

5.7 Predicción y matriz de confusión

```
if (exists("modelo_logistico")) {
  prueba$probabilidad_victimas <- predict(modelo_logistico, newdata = prueba, type =
  ↪ "response")
  prueba$prediccion_03 <- factor(
    ifelse(prueba$probabilidad_victimas >= 0.3, "Con víctimas", "Solo daños"),
    levels = c("Con víctimas", "Solo daños")
  )

  matriz_confusion_03 <- table(
    Real = prueba$accidente_con_victimas,
    Predicho = prueba$prediccion_03
  )
}
```

```
matriz_confusion_03
}
```

```
#>               Predicho
#> Real           Con víctimas Solo daños
#>   Con víctimas      13765      6739
#>   Solo daños       14302      82382
```

Explicación del código

Las probabilidades se convierten en clases usando un punto de corte de 0.3.

5.8 Métricas

```
if (exists("matriz_confusion_03")) {
  VP <- matriz_confusion_03["Con víctimas", "Con víctimas"]
  FN <- matriz_confusion_03["Con víctimas", "Solo daños"]
  FP <- matriz_confusion_03["Solo daños", "Con víctimas"]
  VN <- matriz_confusion_03["Solo daños", "Solo daños"]

  data.frame(
    exactitud = (VP + VN) / (VP + FN + FP + VN),
    sensibilidad = VP / (VP + FN),
    especificidad = VN / (VN + FP)
  )
}
```

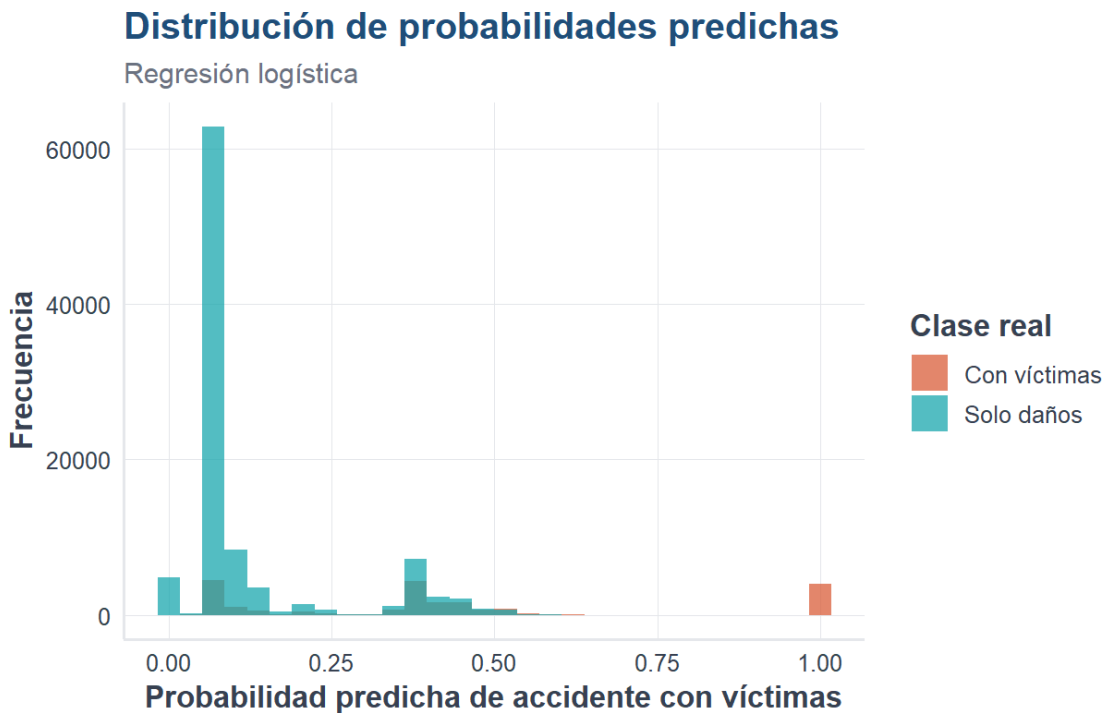
```
#>   exactitud sensibilidad especificidad
#> 1 0.8204509    0.6713324    0.8520748
```

Interpretación del resultado

La sensibilidad mide la capacidad para detectar accidentes con víctimas; la especificidad mide la capacidad para detectar accidentes de solo daños.

5.9 Distribución de probabilidades

```
if (exists("prueba")) {
  ggplot(prueba, aes(x = probabilidad_victimas, fill = accidente_con_victimas)) +
    geom_histogram(bins = 30, alpha = 0.75, position = "identity") +
    escala_clases_fill(name = "Clase real") +
    labs(
      title = "Distribución de probabilidades predichas",
      subtitle = "Regresión logística",
      x = "Probabilidad predicha de accidente con víctimas",
      y = "Frecuencia"
    ) +
    tema_libro()
}
```



5.10 Materiales complementarios del capítulo

Estos recursos permiten estudiar la regresión logística desde una perspectiva audiovisual, gráfica y práctica.

Recurso	Utilidad	Abrir o reproducir	Descargar
Video explicativo	Explicación audiovisual de la regresión logística aplicada a problemas de clasificación.	Ver en YouTube	—
Presentación en PDF	Diapositivas para lectura, estudio o exposición.	Ver PDF	Descargar PDF
Presentación editable	Archivo PowerPoint para utilizarlo en clase o adaptarlo.	Abrir PPTX	Descargar PPTX
Infografía	Síntesis visual de los conceptos, la función logística y el proceso de clasificación.	Ver infografía	Descargar PNG

Video del capítulo: <https://www.youtube.com/watch?v=7CwP40QDeys>

Curso completo en YouTube: <https://www.youtube.com/playlist?list=PLDJYd2v7Kt-Q>

La presentación PDF, el archivo PowerPoint y la infografía pueden descargarse desde la versión web del libro.



Curso completo en YouTube

Este video forma parte de la lista oficial del curso.

[Consultar todos los videos del curso](#)



Elaboración de los materiales

Los materiales complementarios fueron elaborados con apoyo de **NotebookLM de Google**, a partir del contenido del capítulo, y posteriormente revisados y adaptados por el autor.

5.11 Laboratorio interactivo: regresión logística

5.11.1 Laboratorio disponible en la versión web

El laboratorio permite cambiar los coeficientes, el umbral y el valor de una observación nueva para estudiar la curva logística y la clase predicha.

5.12 Conclusión

La regresión logística es un modelo interpretable para clasificación binaria. El punto de corte permite ajustar el balance entre sensibilidad y especificidad.

5.13 Referencias fundamentales de la regresión logística

La formulación moderna de la regresión para respuestas binarias se relaciona con el trabajo de Cox (Cox 1958). Para profundizar en ajuste, interpretación, diagnóstico y aplicaciones del modelo logístico puede consultarse a Hosmer et al. (2013). Una presentación orientada al aprendizaje estadístico y a su implementación en R aparece en James et al. (2021).

Capítulo 6

k vecinos más cercanos, k-NN

6.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de explicar k-NN, preparar datos para este método, estandarizar variables y evaluar el desempeño.

6.2 Idea intuitiva

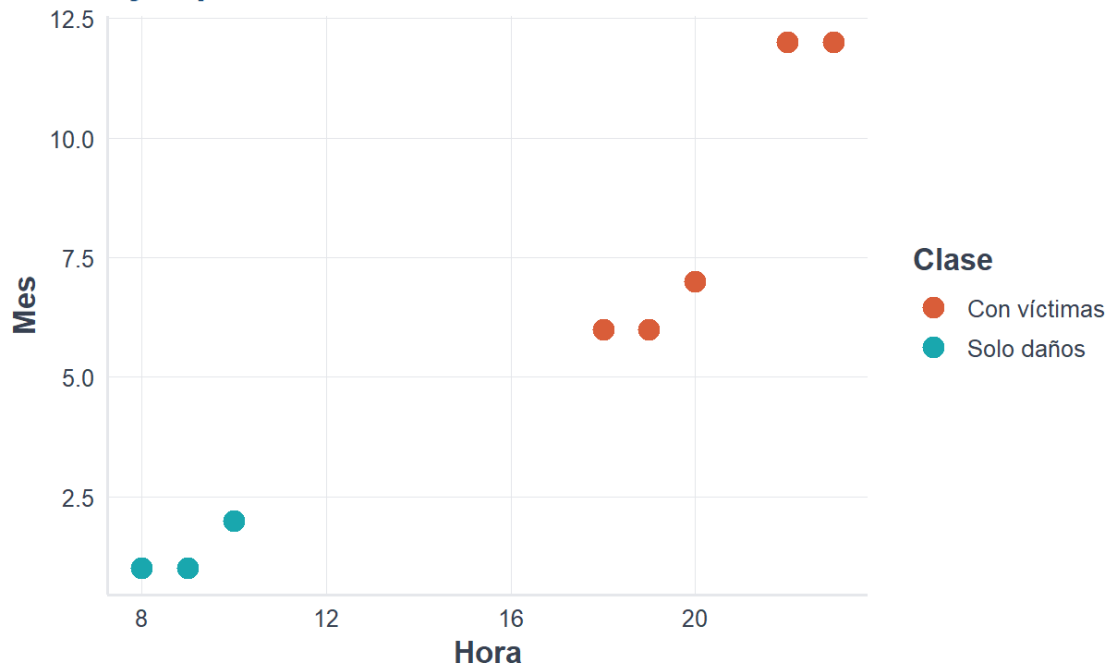
k-NN clasifica una nueva observación según la clase más común entre sus vecinos más cercanos.

```
library(ggplot2)
library(class)
library(dplyr)
library(tidyr)
library(readr)
source("util_graficas.R")

datos_ejemplo <- data.frame(
  hora = c(8, 9, 10, 18, 19, 20, 22, 23),
  mes = c(1, 1, 2, 6, 6, 7, 12, 12),
  clase = c("Solo daños", "Solo daños", "Solo daños", "Con víctimas", "Con víctimas", "Con
    <- víctimas", "Con víctimas", "Con víctimas")
)

ggplot(datos_ejemplo, aes(x = hora, y = mes, color = clase)) +
  geom_point(size = 4) +
  escala_clases_color() +
  labs(title = "Ejemplo intuitivo de k-NN", x = "Hora", y = "Mes", color = "Clase") +
  tema_libro()
```

Ejemplo intuitivo de k-NN



i Explicación del código

Se construyen puntos artificiales para visualizar la idea de vecinos cercanos.

6.3 Distancia euclidiana

$$d(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

```
sqrt((8 - 10)^2 + (1 - 2)^2)
```

```
#> [1] 2.236068
```

💡 Interpretación del resultado

Una distancia menor indica mayor similitud entre dos observaciones.

6.4 Cargar y preparar datos

```
ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  set.seed(123)
  atus_knn_base <- atus_ml |>
    sample_n(min(12000, nrow(atus_ml))) |>
    mutate(
      accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas",
        ↪ "Solo daños")),
      MES = as.factor(MES),
      ID_HORA = as.numeric(ID_HORA),
      DIASEMANA = as.factor(DIASEMANA),
      TIPACCID = as.factor(TIPACCID),
      CAUSAACCI = as.factor(CAUSAACCI)
    ) |>
    na.omit()
}
```

i Explicación del código

Se toma una muestra para que k-NN sea rápido y reproducible. Este método puede ser costoso en bases grandes.

6.5 Modelo k-NN

```
if (exists("atus_knn_base")) {
  matriz_predictoras <- model.matrix(~ ID_HORA + MES + DIASEMANA + TIPACCID + CAUSAACCI -
    ↪ 1, data = atus_knn_base)
  y <- atus_knn_base$accidente_con_victimas

  set.seed(123)
  idx <- sample(1:nrow(matriz_predictoras), size = round(0.7 * nrow(matriz_predictoras)))
  x_entrenamiento <- matriz_predictoras[idx, ]
  x_prueba <- matriz_predictoras[-idx, ]
  y_entrenamiento <- y[idx]
  y_prueba <- y[-idx]

  medias <- apply(x_entrenamiento, 2, mean)
  desviaciones <- apply(x_entrenamiento, 2, sd)
```

```

desviaciones[desviaciones == 0] <- 1

x_entrenamiento_esc <- scale(x_entrenamiento, center = medias, scale = desviaciones)
x_prueba_esc <- scale(x_prueba, center = medias, scale = desviaciones)

pred_knn_5 <- knn(x_entrenamiento_esc, x_prueba_esc, y_entrenamiento, k = 5)
table(Real = y_prueba, Predicho = pred_knn_5)
}

```

```

#>               Predicho
#> Real              Con víctimas Solo daños
#>   Con víctimas      211          399
#>   Solo daños       144         2846

```

Explicación del código

Se crean variables dummy, se estandarizan las columnas y se clasifica con k igual a 5.

6.6 Comparación de valores de k

```

calcular_metricas <- function(real, predicho) {
  real <- factor(real, levels = c("Con víctimas", "Solo daños"))
  predicho <- factor(predicho, levels = c("Con víctimas", "Solo daños"))
  m <- table(Real = real, Predicho = predicho)
  VP <- m["Con víctimas", "Con víctimas"]
  FN <- m["Con víctimas", "Solo daños"]
  FP <- m["Solo daños", "Con víctimas"]
  VN <- m["Solo daños", "Solo daños"]
  data.frame(exactitud=(VP+VN) / (VP+FN+FP+VN), sensibilidad=VP / (VP+FN),
    ↪ especificidad=VN / (VN+FP))
}

if (exists("x_entrenamiento_esc")) {
  valores_k <- c(3, 5, 11)
  resultados_knn <- data.frame()

  for (k_actual in valores_k) {
    pred_actual <- knn(x_entrenamiento_esc, x_prueba_esc, y_entrenamiento, k = k_actual)
    tmp <- calcular_metricas(y_prueba, pred_actual)
    tmp$k <- k_actual
    resultados_knn <- rbind(resultados_knn, tmp)
  }
}

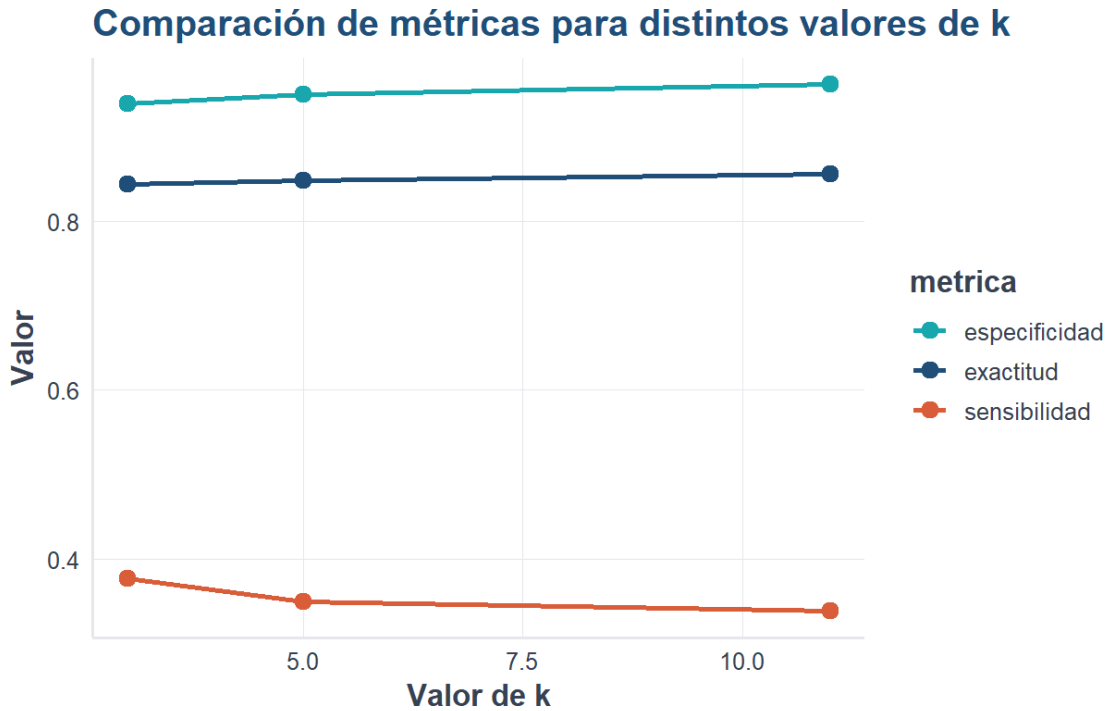
```

```
resultados_knn <- resultados_knn |> select(k, exactitud, sensibilidad, especificidad)
resultados_knn
}
```

```
#>   k exactitud sensibilidad especificidad
#> 1  3 0.8433333    0.3770492    0.9384615
#> 2  5 0.8480556    0.3491803    0.9498328
#> 3 11 0.8558333    0.3377049    0.9615385
```

```
if (exists("resultados_knn")) {
  resultados_largos <- resultados_knn |>
    pivot_longer(cols = c(exactitud, sensibilidad, especificidad), names_to = "metrica",
      ↪ values_to = "valor")

  ggplot(resultados_largos, aes(x = k, y = valor, color = metrica, group = metrica)) +
    geom_line(linewidth = 1.1) +
    geom_point(size = 3) +
    scale_color_manual(values = c(exactitud = col_azul, sensibilidad = col_coral,
      ↪ especificidad = col_turquesa)) +
    labs(title = "Comparación de métricas para distintos valores de k", x = "Valor de k", y
      ↪ = "Valor") +
    tema_libro()
}
```



💡 Interpretación del resultado

Cambiar k modifica el balance entre exactitud, sensibilidad y especificidad. No existe un k universalmente mejor.

6.7 Laboratorio interactivo: k-NN

6.7.1 Laboratorio disponible en la versión web

El lector puede cambiar el valor de k , mover una observación nueva y visualizar sus vecinos, distancias y clase predicha.

6.8 Materiales complementarios del capítulo

6.8.1 Video del capítulo

Disponible en YouTube:

<https://youtu.be/NLJj0QAWomU>

Recurso	Descripción	Abrir o descargar
Presentación en PDF	Síntesis del capítulo para lectura o exposición.	Abrir PDF
Presentación editable	Diapositivas en PowerPoint.	Descargar PPTX
Infografía	Resumen visual del algoritmo k-NN.	Abrir infografía

Los materiales fueron creados con apoyo de NotebookLM de Google a partir del contenido del libro y revisados y adaptados por el autor.

6.9 Conclusión

k-NN es un método intuitivo basado en similitud. Su desempeño depende del valor de k , del escalamiento y de la calidad de las variables predictoras.

6.10 Referencias fundamentales de k-NN

El método de los vecinos más cercanos tiene como antecedente clásico el informe de Fix y Hodges (1951). Sus propiedades de clasificación y cotas de error fueron estudiadas por Cover y Hart (1967). Una exposición moderna, acompañada de aplicaciones en R, puede consultarse en James et al. (2021).

Capítulo 7

Árboles de decisión para clasificación

7.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de explicar la idea básica de un árbol de decisión, entrenarlo en R y evaluar su desempeño.

7.2 Introducción

Los árboles de decisión clasifican observaciones mediante una secuencia de preguntas. Son interpretables y pueden expresarse como reglas tipo si-entonces.

7.3 Fundamento matemático

Una medida común de impureza es el índice de Gini:

$$Gini = 1 - \sum_{k=1}^K p_k^2$$

7.4 Cargar paquetes y datos

```
library(readr)
library(dplyr)
library(rpart)
source("util_graficas.R")

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
} else {
  atus_ml <- NULL
}
```



```
print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo
↪ 2.")
}
```

```
#> [1] "Base preparada cargada correctamente."
```

7.5 Usar la base completa

```
if (!is.null(atus_ml)) {
  atus_arbol_base <- atus_ml |>
  mutate(
    accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas",
↪ "Solo daños")),
    MES = as.factor(MES),
    ID_HORA = as.numeric(ID_HORA),
    DIASEMANA = as.factor(DIASEMANA),
    TIPACCID = as.factor(TIPACCID),
    CAUSAACCI = as.factor(CAUSAACCI)
  ) |>
  na.omit()

  data.frame(filas = nrow(atus_arbol_base), columnas = ncol(atus_arbol_base))
}
```

```
#>      filas columnas
#> 1 390627         6
```

i Explicación del código

Se usa la base completa, pero el crecimiento del árbol se controla mediante parámetros del modelo.

7.6 División y entrenamiento

```
if (exists("atus_arbol_base")) {
  set.seed(123)
  idx <- sample(1:nrow(atus_arbol_base), size = round(0.7 * nrow(atus_arbol_base)))
}
```

```

entrenamiento <- atus_arbol_base[idx, ]
prueba <- atus_arbol_base[-idx, ]

modelo_arbol <- rpart(
  accidente_con_victimas ~ MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
  data = entrenamiento,
  method = "class",
  control = rpart.control(cp = 0.005, minsplit = 1000, minbucket = 300, maxdepth = 5)
)

```

i Explicación del código

Se ajusta un árbol de decisión controlado para evitar sobreajuste y salidas demasiado grandes.

7.7 Complejidad e importancia

```

if (exists("modelo_arbol")) printcp(modelo_arbol)

```

```

#>
#> Classification tree:
#> rpart(formula = accidente_con_victimas ~ MES + ID_HORA + DIASEMANA +
#>       TIPACCID + CAUSAACCI, data = entrenamiento, method = "class",
#>       control = rpart.control(cp = 0.005, minsplit = 1000, minbucket = 300,
#>       maxdepth = 5))
#>
#> Variables actually used in tree construction:
#> [1] TIPACCID
#>
#> Root node error: 47604/273439 = 0.17409
#>
#> n= 273439
#>
#>      CP nsplit rel error xerror      xstd
#> 1 0.097051      0   1.0000 1.0000 0.0041653
#> 2 0.005000      2   0.8059 0.8059 0.0038150

```

```

if (exists("modelo_arbol")) {
  importancia <- modelo_arbol$variable.importance
  if (!is.null(importancia)) {
    importancia_df <- data.frame(variable = names(importancia), importancia =
↪ as.numeric(importancia), row.names = NULL)
    importancia_df <- importancia_df[order(-importancia_df$importancia), ]
    importancia_df
  }
}

```

```

#>   variable importancia
#> 1 TIPACCID    22919.330
#> 2 CAUSAACCI     1093.984

```

💡 Interpretación del resultado

La importancia de variables indica qué variables ayudan más a separar accidentes con víctimas de accidentes de solo daños.

7.8 Gráfica opcional del árbol

La gráfica completa del árbol puede ser pesada en PDF. El lector puede ejecutarla manualmente en RStudio.

```

plot(modelo_arbol, uniform = TRUE, margin = 0.1)
text(modelo_arbol, use.n = TRUE, cex = 0.7)

```

7.9 Predicción y métricas

```

if (exists("modelo_arbol")) {
  pred_arbol <- predict(modelo_arbol, newdata = prueba, type = "class")
  real_arbol <- factor(prueba$accidente_con_victimas, levels = c("Con víctimas", "Solo
↪ daños"))
  pred_arbol <- factor(pred_arbol, levels = c("Con víctimas", "Solo daños"))

  matriz_confusion_arbol <- table(Real = real_arbol, Predicho = pred_arbol)
  matriz_confusion_arbol
}

```

```
#>               Predicho
#> Real           Con víctimas Solo daños
#>   Con víctimas      3965      16539
#>   Solo daños        0       96684
```

```
if (exists("matriz_confusion_arbol")) {
  VP <- matriz_confusion_arbol["Con víctimas", "Con víctimas"]
  FN <- matriz_confusion_arbol["Con víctimas", "Solo daños"]
  FP <- matriz_confusion_arbol["Solo daños", "Con víctimas"]
  VN <- matriz_confusion_arbol["Solo daños", "Solo daños"]

  data.frame(exactitud=(VP+VN) / (VP+FN+FP+VN), sensibilidad=VP / (VP+FN),
    ↪ especificidad=VN / (VN+FP))
}
```

```
#>   exactitud sensibilidad especificidad
#> 1 0.8588678    0.1933769            1
```

Interpretación del resultado

La matriz de confusión y las métricas permiten evaluar el desempeño del árbol sobre datos de prueba.

7.10 Laboratorio interactivo: construye una división del árbol

7.10.1 Laboratorio disponible en la versión web

La versión web permite cambiar la variable y el punto de corte, observar la división de las especies de `iris` y comparar la impureza de Gini de los nodos.

7.11 Materiales complementarios del capítulo

7.11.1 Video del capítulo

Disponible en YouTube:

<https://youtu.be/UperQjxBYEY>

Recurso	Descripción	Abrir o descargar
Presentación en PDF	Síntesis del capítulo para lectura o exposición.	Abrir PDF
Presentación editable	Diapositivas en PowerPoint.	Descargar PPTX
Infografía	Resumen visual de árboles de decisión.	Abrir infografía

Los materiales fueron creados con apoyo de NotebookLM de Google a partir del contenido del libro y revisados y adaptados por el autor.

7.12 Conclusión

Los árboles de decisión son modelos intuitivos, visuales e interpretables. Este capítulo prepara el camino para modelos basados en muchos árboles, como Random Forest.

7.13 Referencias fundamentales de los árboles de decisión

La metodología CART fue desarrollada sistemáticamente por Breiman et al. (1984). El algoritmo ID3 y la inducción de árboles mediante ganancia de información fueron presentados por Quinlan (1986). Para una introducción contemporánea con ejemplos en R puede consultarse James et al. (2021).

Capítulo 8

Random Forest para clasificación

8.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de explicar Random Forest, entrenar un modelo en R, interpretar importancia de variables y evaluar desempeño.

8.2 Introducción

Random Forest combina muchos árboles de decisión. Cada árbol aprende sobre una muestra aleatoria de los datos y la predicción final se obtiene por votación mayoritaria.

8.3 Fundamento matemático

Si se construyen B árboles, la predicción final es:

$$\hat{y}(x) = \text{moda}\{\hat{y}^{(1)}(x), \hat{y}^{(2)}(x), \dots, \hat{y}^{(B)}(x)\}$$

8.4 Cargar paquetes y datos

```
library(readr)
library(dplyr)
library(ggplot2)
library(ranger)
source("util_graficas.R")

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
} else {
  atus_ml <- NULL
  print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo
↪ 2.")
}
```

```
}
```

```
#> [1] "Base preparada cargada correctamente."
```

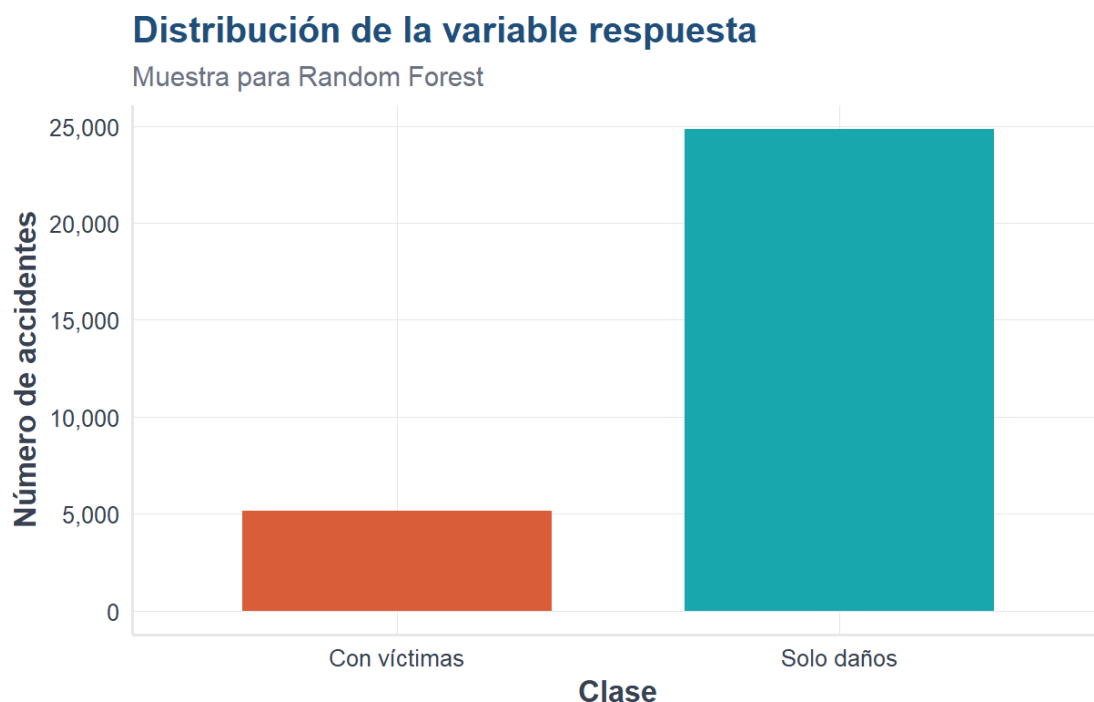
8.5 Muestra de trabajo

```
if (!is.null(atus_ml)) {
  set.seed(123)
  atus_rf_base <- atus_ml |>
    sample_n(min(30000, nrow(atus_ml))) |>
    mutate(
      accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas",
        ↪ "Solo daños")),
      MES = as.factor(MES),
      ID_HORA = as.numeric(ID_HORA),
      DIASEMANA = as.factor(DIASEMANA),
      TIPACCID = as.factor(TIPACCID),
      CAUSAACCI = as.factor(CAUSAACCI)
    ) |>
    na.omit()

  data.frame(filas = nrow(atus_rf_base), columnas = ncol(atus_rf_base))
}
```

```
#>   filas columnas
#> 1 30000         6
```

```
if (exists("atus_rf_base")) {
  ggplot(atus_rf_base, aes(x = accidente_con_victimas, fill = accidente_con_victimas)) +
    geom_bar(width = 0.7) +
    escala_clases_fill() +
    scale_y_continuous(labels = etiqueta_numero) +
    labs(
      title = "Distribución de la variable respuesta",
      subtitle = "Muestra para Random Forest",
      x = "Clase",
      y = "Número de accidentes"
    ) +
    tema_libro() +
    theme(legend.position = "none")
}
```



i Explicación del código

Se toma una muestra de trabajo para mantener el tiempo de ejecución razonable. El modelo Random Forest suele ser más pesado que un solo árbol.

8.6 División en entrenamiento y prueba

```
if (exists("atus_rf_base")) {
  set.seed(123)
  idx <- sample(1:nrow(atus_rf_base), size = round(0.7 * nrow(atus_rf_base)))
  entrenamiento <- atus_rf_base[idx, ]
  prueba <- atus_rf_base[-idx, ]

  data.frame(conjunto = c("Entrenamiento", "Prueba"), registros = c(nrow(entrenamiento),
    ↪ nrow(prueba)))
}
```

```
#>      conjunto registros
#> 1 Entrenamiento  21000
#> 2      Prueba    9000
```


8.7 Ajustar Random Forest

```
if (exists("entrenamiento")) {
  set.seed(123)

  modelo_rf <- ranger(
    accidente_con_victimas ~ MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
    data = entrenamiento,
    num.trees = 200,
    mtry = 3,
    min.node.size = 20,
    importance = "impurity",
    probability = TRUE,
    seed = 123
  )

  modelo_rf
}
```

```
#> Ranger result
#>
#> Call:
#> ranger(accidente_con_victimas ~ MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI, data = entrenamiento)
#>
#> Type:                                Probability estimation
#> Number of trees:                      200
#> Sample size:                          21000
#> Number of independent variables:      5
#> Mtry:                                  3
#> Target node size:                     20
#> Variable importance mode:             impurity
#> Splitrule:                            gini
#> OOB prediction error (Brier s.):      0.1066668
```

Interpretación del resultado

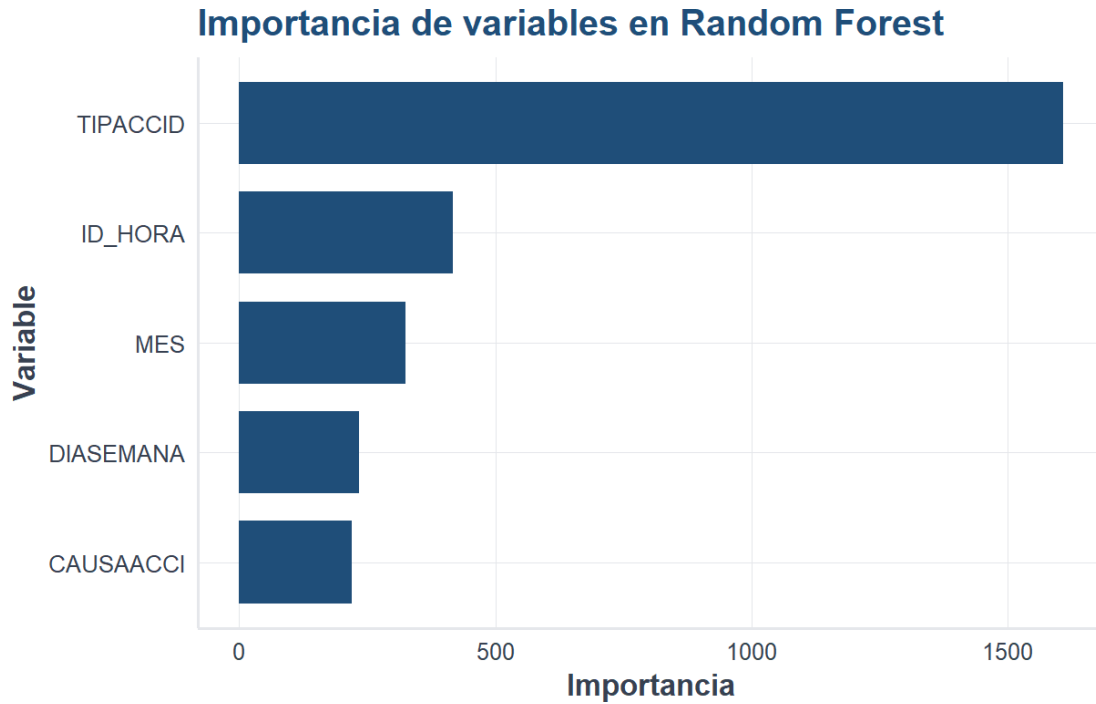
El resumen del modelo muestra cuántos árboles se construyeron, cuántas variables se probaron en cada división y el error fuera de bolsa.

8.8 Importancia de variables

```
if (exists("modelo_rf")) {  
  importancia_rf <- data.frame(  
    variable = names(modelo_rf$variable.importance),  
    importancia = as.numeric(modelo_rf$variable.importance)  
  ) |>  
    arrange(desc(importancia))  
  
  importancia_rf  
}
```

```
#>   variable importancia  
#> 1 TIPACCID    1608.8347  
#> 2  ID_HORA     416.7870  
#> 3      MES     324.9780  
#> 4 DIASEMANA    234.4944  
#> 5 CAUSAACCI    220.5923
```

```
if (exists("importancia_rf")) {  
  ggplot(importancia_rf, aes(x = reorder(variable, importancia), y = importancia)) +  
    geom_col(fill = col_azul, width = 0.75) +  
    coord_flip() +  
    labs(title = "Importancia de variables en Random Forest", x = "Variable", y =  
      ↪ "Importancia") +  
    tema_libro()  
}
```



i Explicación del código

La importancia de variables muestra qué predictores aportan más a la reducción de impureza en los árboles del bosque.

8.9 Predicción y métricas

```
if (exists("modelo_rf")) {
  predicciones_rf <- predict(modelo_rf, data = prueba)
  probabilidades_rf <- predicciones_rf$predictions[, "Con víctimas"]
  clases_rf <- colnames(predicciones_rf$predictions)[max.col(predicciones_rf$predictions,
↪ ties.method = "first")]
  clases_rf <- factor(clases_rf, levels = c("Con víctimas", "Solo daños"))
  real_rf <- factor(prueba$accidente_con_victimas, levels = c("Con víctimas", "Solo
↪ daños"))

  matriz_confusion_rf <- table(Real = real_rf, Predicho = clases_rf)
  matriz_confusion_rf
}
```

```
#> Predicho
```

```
#> Real          Con víctimas Solo daños
#>   Con víctimas      475      991
#>   Solo daños      265     7269
```

```
if (exists("matriz_confusion_rf")) {
  VP <- matriz_confusion_rf["Con víctimas", "Con víctimas"]
  FN <- matriz_confusion_rf["Con víctimas", "Solo daños"]
  FP <- matriz_confusion_rf["Solo daños", "Con víctimas"]
  VN <- matriz_confusion_rf["Solo daños", "Solo daños"]

  data.frame(exactitud=(VP+VN) / (VP+FN+FP+VN), sensibilidad=VP / (VP+FN),
    ↪ especificidad=VN / (VN+FP))
}
```

```
#> exactitud sensibilidad especificidad
#> 1 0.8604444 0.3240109 0.9648261
```

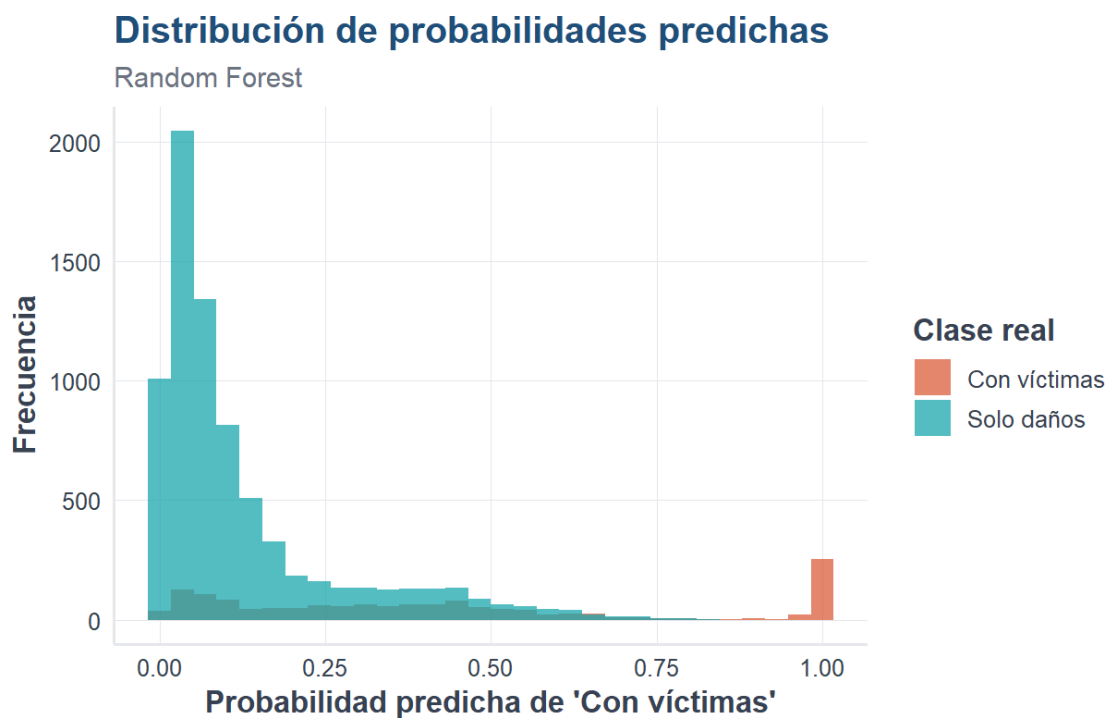
💡 Interpretación del resultado

La sensibilidad indica qué tan bien detecta accidentes con víctimas. La especificidad indica qué tan bien reconoce accidentes de solo daños.

8.10 Distribución de probabilidades predichas

```
if (exists("probabilidades_rf")) {
  datos_prob_rf <- data.frame(probabilidad = probabilidades_rf, clase_real = real_rf)

  ggplot(datos_prob_rf, aes(x = probabilidad, fill = clase_real)) +
    geom_histogram(bins = 30, alpha = 0.75, position = "identity") +
    escala_clases_fill(name = "Clase real") +
    labs(
      title = "Distribución de probabilidades predichas",
      subtitle = "Random Forest",
      x = "Probabilidad predicha de 'Con víctimas'",
      y = "Frecuencia"
    ) +
    tema_libro()
}
```



8.11 Conclusión

Random Forest mejora la estabilidad y la capacidad predictiva de un solo árbol al combinar muchos árboles. Es uno de los métodos más útiles para clasificación aplicada.

8.12 Referencias fundamentales de Random Forest

El algoritmo Random Forest fue formalizado por Breiman (2001) como un ensamble de árboles construido mediante remuestreo y selección aleatoria de variables. Una explicación didáctica y comparativa puede consultarse también en James et al. (2021).

Capítulo 9

Evaluación, validación y comparación de modelos

9.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de:

- explicar por qué la exactitud no siempre es suficiente;
- construir e interpretar una matriz de confusión;
- calcular exactitud, sensibilidad, especificidad, precisión y valor F1;
- distinguir entrenamiento, validación y prueba;
- aplicar validación cruzada estratificada;
- comparar modelos con criterios predictivos y prácticos;
- seleccionar un modelo sin depender de una sola métrica.

9.2 Introducción

Entrenar un modelo es apenas la mitad del trabajo. Después debemos responder una pregunta más delicada: **¿qué tan bien funcionará con accidentes que nunca vio durante el aprendizaje?**

Un modelo puede memorizar los datos de entrenamiento y parecer excelente, pero fallar con observaciones nuevas. Por eso la evaluación debe realizarse con datos separados y métricas apropiadas para el objetivo del problema.

En este capítulo se utiliza nuevamente la base preparada a partir de ATUS (Instituto Nacional de Estadística y Geografía 2025).

9.3 Cargar los datos

```
library(readr)
library(dplyr)
library(ggplot2)
```

```

source("util_graficas.R")

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (!file.exists(ruta_atus_ml)) {
  stop(
    paste(
      "No se encontró datos/atus_ml_preparado.csv.",
      "Renderice primero el capítulo de preparación de datos."
    )
  )
}

atus_ml <- read_csv(
  ruta_atus_ml,
  show_col_types = FALSE
)

```

9.4 Preparar una muestra reproducible

```

set.seed(123)

atus_eval <- atus_ml |>
  sample_n(min(30000, nrow(atus_ml))) |>
  mutate(
    accidente_con_victimas = factor(
      accidente_con_victimas,
      levels = c("Solo daños", "Con víctimas")
    ),
    MES = factor(MES),
    ID_HORA = as.numeric(ID_HORA),
    DIASEMANA = factor(DIASEMANA),
    TIPACCID = factor(TIPACCID),
    CAUSAACCI = factor(CAUSAACCI)
  ) |>
  na.omit()

prop.table(table(atus_eval$accidente_con_victimas))

```

```

#>
#> Solo daños Con víctimas
#> 0.8277333 0.1722667

```

💡 Interpretación

La tabla de proporciones permite identificar si una clase es mucho más frecuente que la otra. Cuando existe desbalance, una exactitud alta puede resultar engañosa.

9.5 Entrenamiento, validación y prueba

Los datos pueden dividirse en tres partes:

- **Entrenamiento:** se utiliza para estimar los parámetros del modelo.
- **Validación:** ayuda a elegir hiperparámetros y comparar alternativas.
- **Prueba:** se reserva para estimar el desempeño final.

En ejercicios introductorios es frecuente utilizar entrenamiento y prueba, mientras que la validación se realiza mediante validación cruzada dentro del conjunto de entrenamiento.

9.6 División estratificada

Una división estratificada conserva aproximadamente la proporción de cada clase.

```
set.seed(123)

indices_entrenamiento <- unlist(
  lapply(
    split(seq_len(nrow(atus_eval)), atus_eval$accidente_con_victimas),
    function(indices) {
      sample(indices, size = floor(0.70 * length(indices)))
    }
  )
)

entrenamiento <- atus_eval[indices_entrenamiento, ]
prueba <- atus_eval[-indices_entrenamiento, ]

prop.table(table(entrenamiento$accidente_con_victimas))
```

```
#>
#> Solo daños Con víctimas
#> 0.8277537 0.1722463
```



```
prop.table(table(prueba$accidente_con_victimas))
```

```
#>
#> Solo daños Con víctimas
#> 0.8276858 0.1723142
```

i Explicación del código

Los índices se separan por clase y después se toma 70 % de cada grupo. Así se reduce el riesgo de que una clase quede sobrerrepresentada en entrenamiento o prueba.

9.7 Matriz de confusión

Para una clase positiva denominada **Con víctimas**, la matriz contiene:

- **Verdadero positivo (VP)**: el accidente tenía víctimas y el modelo lo detectó.
- **Falso negativo (FN)**: tenía víctimas, pero el modelo lo clasificó como solo daños.
- **Falso positivo (FP)**: era de solo daños, pero el modelo predijo víctimas.
- **Verdadero negativo (VN)**: era de solo daños y se clasificó correctamente.

Clase real	Predicción positiva	Predicción negativa
Positiva	VP	FN
Negativa	FP	VN

Un falso negativo puede ser especialmente importante cuando el propósito consiste en identificar accidentes con víctimas.

9.8 Métricas principales

9.8.1 Exactitud

$$\text{Exactitud} = \frac{VP + VN}{VP + FN + FP + VN}$$

Mide la proporción total de predicciones correctas.

9.8.2 Sensibilidad

$$\text{Sensibilidad} = \frac{VP}{VP + FN}$$

Mide qué proporción de los accidentes con víctimas fue detectada.

9.8.3 Especificidad

$$\text{Especificidad} = \frac{VN}{VN + FP}$$

Mide qué proporción de los accidentes de solo daños fue reconocida correctamente.

9.8.4 Precisión

$$\text{Precisión} = \frac{VP}{VP + FP}$$

Indica qué proporción de las predicciones positivas realmente tenía víctimas.

9.8.5 Valor F1

$$F_1 = 2 \frac{\text{Precisión} \times \text{Sensibilidad}}{\text{Precisión} + \text{Sensibilidad}}$$

El valor F1 equilibra precisión y sensibilidad.

9.9 Función para calcular métricas

```
calcular_metricas <- function(real, predicho, positiva = "Con víctimas") {
  real <- factor(real)
  predicho <- factor(predicho, levels = levels(real))

  negativas <- setdiff(levels(real), positiva)

  if (length(negativas) != 1) {
    stop("La función requiere exactamente dos clases.")
  }
}
```

```

}

negativa <- negativas[1]
matriz <- table(Real = real, Predicho = predicho)

VP <- matriz[positiva, positiva]
FN <- matriz[positiva, negativa]
FP <- matriz[negativa, positiva]
VN <- matriz[negativa, negativa]

division_segura <- function(numerador, denominador) {
  if (
    is.na(numerador) ||
    is.na(denominador) ||
    denominador == 0
  ) {
    return(NA_real_)
  }

  as.numeric(numerador / denominador)
}

exactitud <- division_segura(VP + VN, VP + FN + FP + VN)
sensibilidad <- division_segura(VP, VP + FN)
especificidad <- division_segura(VN, VN + FP)
precision <- division_segura(VP, VP + FP)
f1 <- division_segura(
  2 * precision * sensibilidad,
  precision + sensibilidad
)

list(
  matriz = matriz,
  metricas = data.frame(
    exactitud = exactitud,
    sensibilidad = sensibilidad,
    especificidad = especificidad,
    precision = precision,
    f1 = f1
  )
)
}

```

9.10 Modelo de referencia: regla mayoritaria

Antes de celebrar un modelo complejo, conviene compararlo con una regla muy simple: predecir siempre la clase más frecuente.

```
clase_mayoritaria <- names(
  which.max(table(entrenamiento$accidente_con_victimas))
)

prediccion_referencia <- factor(
  rep(clase_mayoritaria, nrow(prueba)),
  levels = levels(entrenamiento$accidente_con_victimas)
)

resultado_referencia <- calcular_metricas(
  real = prueba$accidente_con_victimas,
  predicho = prediccion_referencia
)

resultado_referencia$matriz
```

```
#>               Predicho
#> Real              Solo daños Con víctimas
#> Solo daños           7450           0
#> Con víctimas        1551           0
```

```
resultado_referencia$metricas
```

```
#> exactitud sensibilidad especificidad precision fl
#> 1 0.8276858           0           1      NA NA
```

i ¿Por qué puede aparecer NA?

El modelo de referencia predice únicamente la clase mayoritaria. Si nunca predice la clase positiva **Con víctimas**, la precisión no puede calcularse porque su denominador es cero. En ese caso R muestra NA, que significa **métrica no definida**, pero el libro continúa procesándose normalmente.

⚠ Una exactitud alta puede engañar

Si 90 % de los accidentes perteneciera a una sola clase, predecir siempre esa clase produciría 90 % de exactitud sin aprender ninguna relación útil. Por eso deben revisarse sensibilidad, especificidad, precisión y F1.

9.11 Evaluar una regresión logística

```

modelo_logistico <- glm(
  accidente_con_victimas ~
    MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
  data = entrenamiento,
  family = binomial
)

probabilidad_logistica <- predict(
  modelo_logistico,
  newdata = prueba,
  type = "response"
)

# glm() modela la probabilidad del segundo nivel del factor.
clase_positiva_modelada <- levels(
  entrenamiento$accidente_con_victimas
)[2]

prediccion_logistica <- ifelse(
  probabilidad_logistica >= 0.50,
  clase_positiva_modelada,
  levels(entrenamiento$accidente_con_victimas)[1]
)

prediccion_logistica <- factor(
  prediccion_logistica,
  levels = levels(entrenamiento$accidente_con_victimas)
)

resultado_logistico <- calcular_metricas(
  real = prueba$accidente_con_victimas,
  predicho = prediccion_logistica
)

resultado_logistico$matriz

```

#> Predicho

```
#> Real          Solo daños Con víctimas
#> Solo daños      7403         47
#> Con víctimas    1198        353
```

```
resultado_logistico$metricas
```

```
#> exactitud sensibilidad especificidad precision      f1
#> 1  0.861682    0.2275951    0.9936913    0.8825 0.3618657
```

! Revise qué clase modela R

En una regresión logística binaria, `glm()` calcula la probabilidad del segundo nivel del factor respuesta. El orden de los niveles debe verificarse antes de transformar probabilidades en clases.

9.12 Comparación inicial

```
comparacion_inicial <- bind_rows(
  cbind(
    modelo = "Regla mayoritaria",
    resultado_referencia$metricas
  ),
  cbind(
    modelo = "Regresión logística",
    resultado_logistico$metricas
  )
)

comparacion_inicial
```

```
#>          modelo exactitud sensibilidad especificidad precision      f1
#> 1  Regla mayoritaria 0.8276858    0.0000000    1.0000000      NA      NA
#> 2 Regresión logística 0.8616820    0.2275951    0.9936913    0.8825 0.3618657
```

```
comparacion_larga <- comparacion_inicial |>
  tidyr::pivot_longer(
    cols = -modelo,
```

```

names_to = "metrica",
values_to = "valor"
)

ggplot(
  comparacion_larga,
  aes(x = metrica, y = valor, fill = modelo)
) +
  geom_col(position = "dodge") +
  scale_y_continuous(
    limits = c(0, 1),
    labels = scales::label_percent(accuracy = 1)
  ) +
  labs(
    title = "Un modelo debe superar una referencia sencilla",
    x = "Métrica",
    y = "Valor",
    fill = "Modelo"
  ) +
  tema_libro()

```

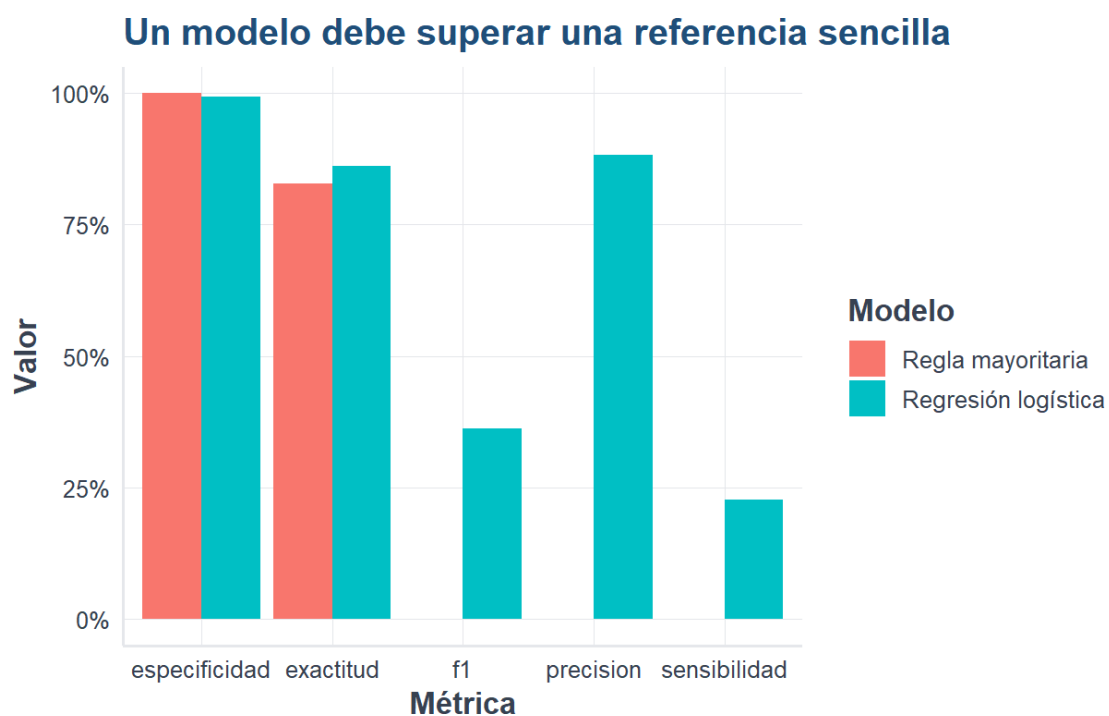


Figura 9.1: Comparación de métricas entre una regla de referencia y la regresión logística.

Fuente: Elaboración propia con datos del INEGI, ATUS 2024.

9.13 Umbral de clasificación

El umbral de 0.50 no es una ley universal. Reducirlo puede aumentar la sensibilidad, aunque también puede generar más falsos positivos.

```
umbrales <- seq(0.10, 0.90, by = 0.05)

metricas_umbral <- lapply(umbrales, function(umbral) {
  prediccion <- ifelse(
    probabilidad_logistica >= umbral,
    clase_positiva_modelada,
    levels(entrenamiento$accidente_con_victimas)[1]
  )

  prediccion <- factor(
    prediccion,
    levels = levels(entrenamiento$accidente_con_victimas)
  )

  metricas <- calcular_metricas(
    prueba$accidente_con_victimas,
    prediccion
  )$metricas

  cbind(umbral = umbral, metricas)
}) |>
  bind_rows()

metricas_umbral
```

```
#>   umbral exactitud sensibilidad especificidad precision      f1
#> 1   0.10 0.7351405    0.7878788    0.7241611 0.3729020 0.5062138
#> 2   0.15 0.7996889    0.7150226    0.8173154 0.4489879 0.5516041
#> 3   0.20 0.8055772    0.7047066    0.8265772 0.4582809 0.5553862
#> 4   0.25 0.8182424    0.6834300    0.8463087 0.4807256 0.5644302
#> 5   0.30 0.8196867    0.6756931    0.8496644 0.4833948 0.5635924
#> 6   0.35 0.8237974    0.6434558    0.8613423 0.4913836 0.5572306
#> 7   0.40 0.8447950    0.4455190    0.9279195 0.5627036 0.4973012
#> 8   0.45 0.8586824    0.2778852    0.9795973 0.7392796 0.4039363
#> 9   0.50 0.8616820    0.2275951    0.9936913 0.8825000 0.3618657
#> 10  0.55 0.8602378    0.2127660    0.9950336 0.8991826 0.3441084
#> 11  0.60 0.8593490    0.1889104    0.9989262 0.9734219 0.3164147
#> 12  0.65 0.8596823    0.1856867    1.0000000 1.0000000 0.3132137
#> 13  0.70 0.8596823    0.1856867    1.0000000 1.0000000 0.3132137
#> 14  0.75 0.8596823    0.1856867    1.0000000 1.0000000 0.3132137
```



```
#> 15    0.80 0.8596823    0.1856867    1.0000000 1.0000000 0.3132137
#> 16    0.85 0.8596823    0.1856867    1.0000000 1.0000000 0.3132137
#> 17    0.90 0.8596823    0.1856867    1.0000000 1.0000000 0.3132137
```

i Decisión práctica

El mejor umbral depende del costo de cada error. Cuando omitir un accidente con víctimas es más grave que generar una alerta adicional, puede priorizarse la sensibilidad.

9.14 Validación cruzada

La validación cruzada divide el conjunto de entrenamiento en varios pliegues. Cada pliegue funciona una vez como validación y los restantes como entrenamiento.

En validación cruzada de K pliegues:

1. se divide la muestra en K partes;
2. se entrena con $K - 1$ partes;
3. se evalúa en la parte restante;
4. se repite hasta utilizar todos los pliegues;
5. se promedian las métricas.

9.14.1 Crear pliegues estratificados

```
crear_pliegues <- function(respuesta, k = 5, semilla = 123) {
  set.seed(semilla)

  pliegues <- integer(length(respuesta))

  for (clase in levels(factor(respuesta))) {
    indices <- which(respuesta == clase)
    etiquetas <- rep(seq_len(k), length.out = length(indices))
    pliegues[indices] <- sample(etiquetas)
  }

  pliegues
}

pliegue <- crear_pliegues(
  entrenamiento$accidente_con_victimas,
  k = 5
)
```

```
table(pliegue, entrenamiento$accidente_con_victimas)
```

```
#>
#> pliegue Solo daños Con víctimas
#>      1      3477      724
#>      2      3477      724
#>      3      3476      723
#>      4      3476      723
#>      5      3476      723
```

9.14.2 Validación cruzada de la regresión logística

```
resultados_cv <- lapply(1:5, function(k) {
  datos_entrenamiento <- entrenamiento[pliegue != k, ]
  datos_validacion <- entrenamiento[pliegue == k, ]

  modelo <- glm(
    accidente_con_victimas ~
      MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
    data = datos_entrenamiento,
    family = binomial
  )

  probabilidad <- predict(
    modelo,
    newdata = datos_validacion,
    type = "response"
  )

  clase_modelada <- levels(
    datos_entrenamiento$accidente_con_victimas
  )[2]

  prediccion <- ifelse(
    probabilidad >= 0.50,
    clase_modelada,
    levels(datos_entrenamiento$accidente_con_victimas)[1]
  )

  prediccion <- factor(
    prediccion,
    levels = levels(datos_entrenamiento$accidente_con_victimas)
  )
})
```

```
metricas <- calcular_metricas(
  datos_validacion$accidente_con_victimas,
  prediccion
)$metricas

cbind(pliegue = k, metricas)
}) |>
  bind_rows()

resultados_cv
```

```
#>   pliegue exactitud sensibilidad especificidad precision      f1
#> 1      1 0.8631278    0.2389503    0.9930975 0.8781726 0.3756786
#> 2      2 0.8590812    0.2140884    0.9933851 0.8707865 0.3436807
#> 3      3 0.8630626    0.2268326    0.9953970 0.9111111 0.3632337
#> 4      4 0.8637771    0.2406639    0.9933832 0.8832487 0.3782609
#> 5      5 0.8525839    0.1798064    0.9925201 0.8333333 0.2957907
```

```
resumen_cv <- resultados_cv |>
  summarise(
    across(
      where(is.numeric) & !matches("pliegue"),
      list(media = mean, desviacion = sd),
      na.rm = TRUE
    )
  )

resumen_cv
```

```
#>   exactitud_media exactitud_desviacion sensibilidad_media
#> 1      0.8603265      0.004710052      0.2200683
#>   sensibilidad_desviacion especificidad_media especificidad_desviacion
#> 1      0.02491609      0.9935566      0.001087613
#>   precision_media precision_desviacion  f1_media f1_desviacion
#> 1      0.8753305      0.02799749 0.3513289    0.03392253
```

Interpretación

La media resume el desempeño esperado y la desviación estándar muestra su estabilidad. Dos modelos con medias parecidas pueden diferir mucho si uno presenta resultados más variables entre pliegues.

9.15 Cómo comparar varios modelos

Para comparar regresión logística, k-NN, árbol de decisión y Random Forest conviene utilizar:

1. la misma muestra;
2. la misma variable respuesta;
3. la misma partición de entrenamiento y prueba;
4. el mismo criterio para definir la clase positiva;
5. las mismas métricas;
6. una validación cruzada equivalente;
7. tiempos de entrenamiento y predicción;
8. facilidad de interpretación.

Una tabla de decisión puede tener esta estructura:

Modelo	Exactitud	Sensibilidad	Especificidad	F1	Interpre- tación	Costo compu- tacional
Regresión logística					Alta	Bajo
k-NN					Media- baja	Medio
Árbol de decisión					Alta	Bajo
Random Forest					Media	Alto

9.16 Selección del modelo final

No existe un modelo universalmente mejor. La selección depende de la finalidad:

- Si se requiere explicar el efecto de las variables, puede preferirse regresión logística.
- Si se necesita una regla visual y fácil de comunicar, un árbol puede ser apropiado.
- Si se prioriza capacidad predictiva y estabilidad, Random Forest puede resultar competitivo.
- Si el costo de los falsos negativos es alto, debe priorizarse sensibilidad.
- Si las alertas falsas son costosas, precisión y especificidad cobran mayor importancia.

! Principio fundamental

El mejor modelo no es necesariamente el que tiene la mayor exactitud, sino el que responde mejor al objetivo, controla los errores relevantes y mantiene un desempeño estable con datos nuevos.

9.17 Actividad guiada

1. Ejecute la regresión logística con umbrales de 0.30, 0.50 y 0.70.
2. Registre la matriz de confusión de cada caso.
3. Compare sensibilidad, especificidad, precisión y F1.
4. Explique qué umbral elegiría para detectar accidentes con víctimas.
5. Justifique su decisión considerando el costo de falsos negativos y falsos positivos.

9.18 Actividades para el lector

1. Construya una función que calcule la exactitud balanceada:

$$\text{Exactitud balanceada} = \frac{\text{Sensibilidad} + \text{Especificidad}}{2}$$

2. Compare la variabilidad de las métricas en validación cruzada.
3. Repita el procedimiento con un árbol de decisión.
4. Elabore una tabla comparativa con los cuatro modelos estudiados.
5. Escriba una recomendación técnica de no más de 200 palabras.

9.19 Resumen del capítulo

En este capítulo aprendimos que evaluar un modelo requiere datos no utilizados durante el entrenamiento, una clase positiva bien definida y varias métricas. También construimos una regla de referencia, estudiamos el efecto del umbral y aplicamos validación cruzada estratificada. Estas herramientas permiten comparar modelos de forma más justa y elegirlos según el objetivo real del análisis.

9.20 Referencias fundamentales sobre evaluación

La validación cruzada y su uso para estimar el desempeño de modelos fueron examinados comparativamente por Kohavi (1995). Para el análisis mediante curvas ROC y AUC, una referencia ampliamente utilizada es Fawcett (2006). Una visión integrada de partición de datos, remuestreo y comparación de modelos aparece en James et al. (2021).

Capítulo 10

Máquinas de Vectores de Soporte (SVM)

10.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de:

- explicar la idea del hiperplano de separación y del margen máximo;
- identificar los vectores de soporte;
- distinguir entre una SVM lineal y una SVM con kernel;
- comprender el papel de los parámetros `cost`, `gamma` y del tipo de kernel;
- entrenar modelos SVM en R;
- evaluar una SVM mediante una matriz de confusión y métricas de clasificación;
- utilizar ponderación de clases cuando la variable respuesta está desbalanceada;
- comparar una SVM lineal con una SVM de kernel radial.

10.2 Introducción

Las **máquinas de vectores de soporte**, conocidas como **SVM** por *Support Vector Machines*, son métodos supervisados utilizados principalmente para clasificación. Su propósito es construir una frontera que separe las clases con el margen más amplio posible.

La idea es sencilla de visualizar: si dos grupos pueden separarse mediante una línea, existen muchas líneas posibles, pero la SVM busca aquella que deja la mayor distancia entre la frontera y las observaciones más cercanas de cada clase.

Esas observaciones cercanas reciben el nombre de **vectores de soporte**. Aunque el conjunto de datos contenga miles de registros, los vectores de soporte son los que determinan directamente la posición de la frontera.

En este capítulo se emplean primero datos simulados para entender la geometría del método y después la base preparada de ATUS (Instituto Nacional de Estadística y Geografía 2025).

10.3 Intuición geométrica

Supongamos que cada observación tiene dos variables, x_1 y x_2 . Una frontera lineal puede escribirse como:

$$w_1 x_1 + w_2 x_2 + b = 0$$

donde:

- w_1 y w_2 determinan la orientación de la frontera;
- b desplaza la frontera;
- el signo de la expresión determina de qué lado queda cada observación.

La SVM busca una frontera con **margen máximo**. En una separación ideal se desea que:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

para todas las observaciones, donde y_i representa la clase codificada como -1 o 1 .

El ancho del margen es proporcional a:

$$\frac{2}{\|\mathbf{w}\|}$$

Por ello, maximizar el margen equivale a minimizar la magnitud de $\mathbf{w}$.

Idea esencial

Una SVM no busca solamente clasificar bien los datos de entrenamiento. Busca una frontera que conserve la mayor separación posible entre las clases, con la intención de generalizar mejor ante observaciones nuevas.

10.4 Crear un ejemplo didáctico

```
set.seed(123)

n <- 120

datos_svm <- data.frame(
  x1 = c(rnorm(n / 2, mean = -1.5, sd = 0.8),
        rnorm(n / 2, mean = 1.5, sd = 0.8)),
```

```
x2 = c(rnorm(n / 2, mean = -1.0, sd = 0.9),
       rnorm(n / 2, mean = 1.0, sd = 0.9)),
       clase = factor(rep(c("Clase A", "Clase B"), each = n / 2))
)

head(datos_svm)
```

```
#>           x1           x2  clase
#> 1 -1.9483805 -0.8941181 Clase A
#> 2 -1.6841420 -1.8527272 Clase A
#> 3 -0.2530333 -1.4415017 Clase A
#> 4 -1.4435933 -1.2304830 Clase A
#> 5 -1.3965698  0.6594758 Clase A
#> 6 -0.1279480 -1.5867549 Clase A
```

i Explicación del código

Se generan dos grupos artificiales. El primero se concentra alrededor de valores negativos y el segundo alrededor de valores positivos. El ejemplo permite observar con claridad la frontera de clasificación.

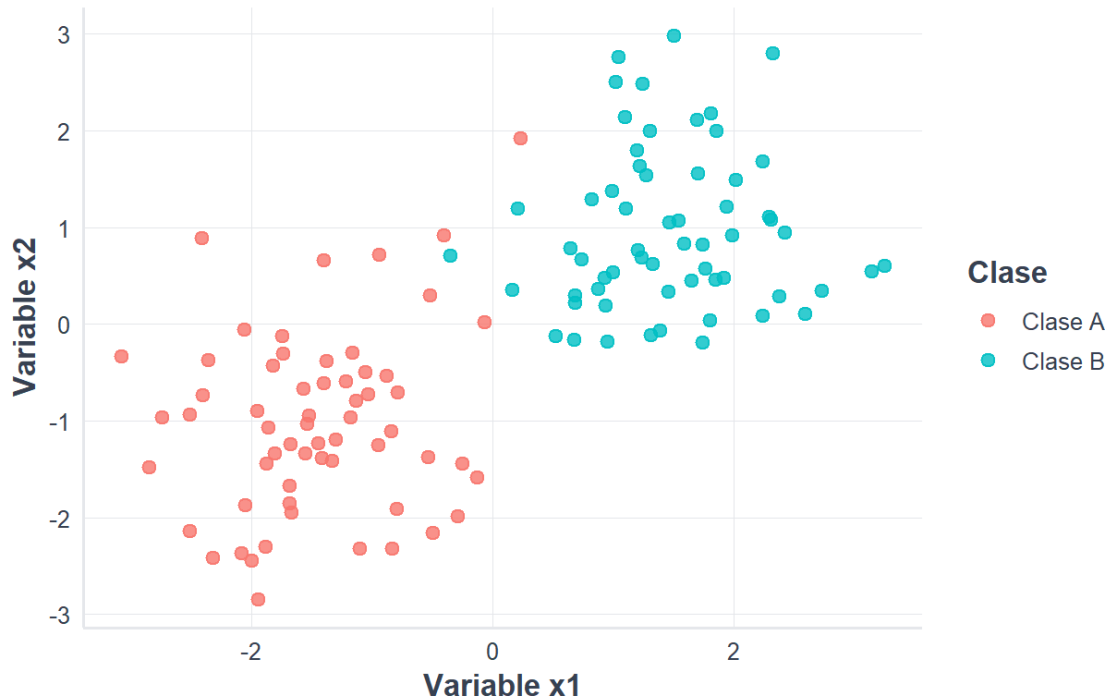
10.5 Visualizar las clases

```
library(ggplot2)
source("util_graficas.R")

ggplot(datos_svm, aes(x = x1, y = x2, color = clase)) +
  geom_point(size = 2.5, alpha = 0.8) +
  labs(
    title = "Datos simulados para clasificación",
    subtitle = "Cada punto representa una observación",
    x = "Variable x1",
    y = "Variable x2",
    color = "Clase"
  ) +
  tema_libro()
```


Datos simulados para clasificación

Cada punto representa una observación



10.6 Instalar y cargar el paquete e1071

```
if (!requireNamespace("e1071", quietly = TRUE)) {
  install.packages("e1071", repos = "https://cloud.r-project.org")
}

library(e1071)
```

El paquete e1071 proporciona la función `svm()`, que permite entrenar modelos lineales y no lineales.

10.7 Entrenar una SVM lineal

```
modelo_svm_lineal <- svm(
  clase ~ x1 + x2,
```

```
data = datos_svm,
kernel = "linear",
cost = 1,
scale = TRUE
)

modelo_svm_lineal
```

```
#>
#> Call:
#> svm(formula = clase ~ x1 + x2, data = datos_svm, kernel = "linear",
#>      cost = 1, scale = TRUE)
#>
#>
#> Parameters:
#>   SVM-Type:  C-classification
#> SVM-Kernel:  linear
#>      cost:   1
#>
#> Number of Support Vectors:  14
```

Los argumentos principales son:

- `kernel = "linear"`: utiliza una frontera lineal;
- `cost = 1`: establece la penalización por errores;
- `scale = TRUE`: estandariza las variables numéricas antes del ajuste.

10.8 Identificar los vectores de soporte

```
indices_soporte <- modelo_svm_lineal$index
vectores_soporte <- datos_svm[indices_soporte, ]

nrow(vectores_soporte)
```

```
#> [1] 14
```

```
head(vectores_soporte)
```

```
#>           x1           x2  clase
#> 3  -0.25303335 -1.44150170 Clase A
#> 6  -0.12794801 -1.58675491 Clase A
#> 11 -0.52073456  0.30009577 Clase A
#> 16 -0.07046949  0.01820349 Clase A
#> 19 -0.93891528  0.71819321 Clase A
#> 44  0.23516477  1.91693594 Clase A
```

💡 Interpretación

Los vectores de soporte son las observaciones más influyentes para establecer la frontera. Los puntos alejados del margen suelen tener poca o ninguna influencia directa sobre su posición.

10.9 Dibujar la frontera de decisión

```
rejilla <- expand.grid(
  x1 = seq(min(datos_svm$x1) - 0.5,
            max(datos_svm$x1) + 0.5,
            length.out = 180),
  x2 = seq(min(datos_svm$x2) - 0.5,
            max(datos_svm$x2) + 0.5,
            length.out = 180)
)

rejilla$prediccion <- predict(modelo_svm_lineal, newdata = rejilla)

ggplot() +
  geom_tile(
    data = rejilla,
    aes(x = x1, y = x2, fill = prediccion),
    alpha = 0.18
  ) +
  geom_point(
    data = datos_svm,
    aes(x = x1, y = x2, color = clase),
    size = 2.2
  ) +
  geom_point(
    data = vectores_soporte,
    aes(x = x1, y = x2),
    shape = 21,
    size = 4,
    stroke = 1.1,
```

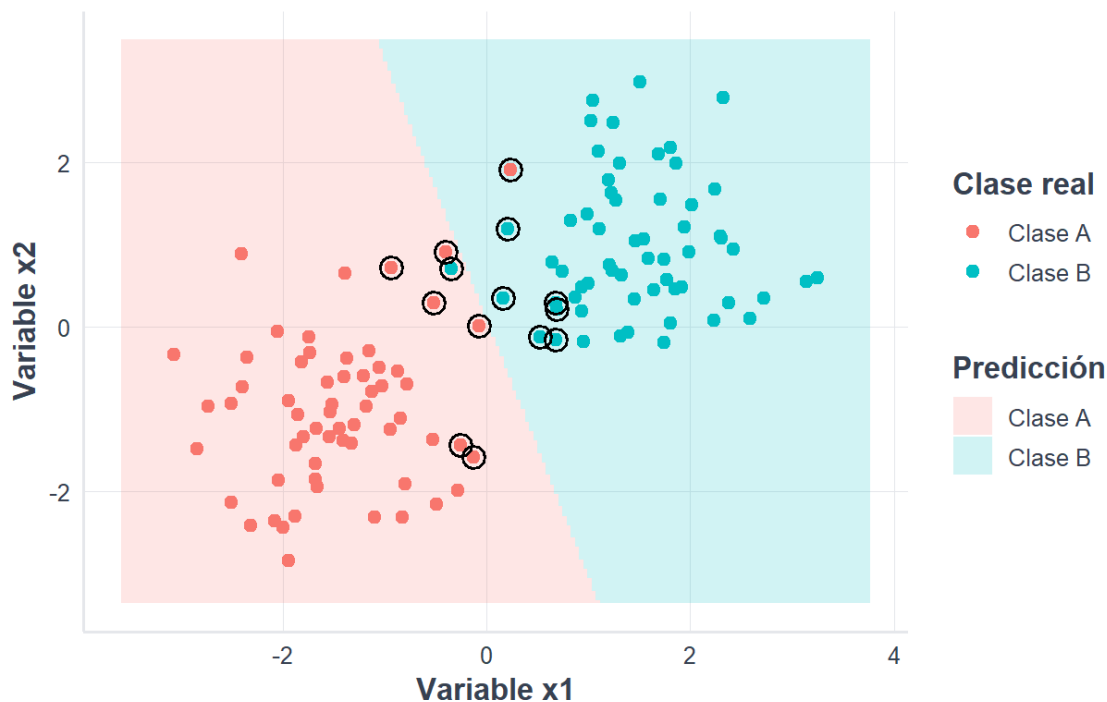
```

fill = NA
) +
labs(
  title = "Frontera de decisión de una SVM lineal",
  subtitle = "Los círculos grandes identifican los vectores de soporte",
  x = "Variable x1",
  y = "Variable x2",
  fill = "Predicción",
  color = "Clase real"
) +
tema_libro()

```

Frontera de decisión de una SVM lineal

Los círculos grandes identifican los vectores de soporte



10.10 El parámetro de costo

El parámetro `cost` controla la penalización asignada a las observaciones clasificadas incorrectamente.

- Un costo pequeño permite más errores y favorece un margen amplio.
- Un costo grande penaliza fuertemente los errores y puede producir una frontera más ajustada a los datos de entrenamiento.

No existe un valor universalmente mejor. Debe seleccionarse con validación cruzada.

```
modelos_cost <- lapply(
  c(0.1, 1, 10),
  function(valor_cost) {
    svm(
      clase ~ x1 + x2,
      data = datos_svm,
      kernel = "linear",
      cost = valor_cost,
      scale = TRUE
    )
  }
)

resumen_cost <- data.frame(
  cost = c(0.1, 1, 10),
  vectores_soporte = vapply(
    modelos_cost,
    function(modelo) modelo$tot.nSV,
    numeric(1)
  )
)

resumen_cost
```

```
#>   cost vectores_soporte
#> 1  0.1                30
#> 2  1.0                14
#> 3 10.0                 8
```

10.11 Cuando una línea no es suficiente

Algunos conjuntos de datos no pueden separarse adecuadamente mediante una línea o un hiperplano. En esos casos, una SVM puede utilizar una función denominada **kernel**.

El kernel calcula similitudes entre observaciones y permite representar fronteras no lineales sin construir explícitamente todas las nuevas variables.

Los kernels más utilizados son:

Kernel	Uso general
Lineal	Cuando la separación es aproximadamente lineal o existen muchas variables
Polinomial	Cuando se esperan relaciones de tipo polinómico

Kernel	Uso general
Radial	Cuando la frontera puede tener formas curvas y complejas
Sigmoide	Menos frecuente; guarda relación con funciones de activación

10.12 Kernel radial

El kernel radial utiliza una función semejante a:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

El parámetro γ controla el alcance de la influencia de cada observación:

- valores pequeños producen fronteras más suaves;
- valores grandes permiten fronteras más locales y complejas.

Riesgo de sobreajuste

Una combinación de `cost` y `gamma` demasiado grande puede ajustar muy bien el entrenamiento, pero funcionar peor con datos nuevos. La validación cruzada ayuda a evitar esa elección.

10.13 Aplicación con los datos ATUS

10.14 Cargar la base preparada

```
library(readr)
library(dplyr)

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (!file.exists(ruta_atus_ml)) {
  stop(
    paste(
      "No se encontró datos/atus_ml_preparado.csv.",
      "Renderice primero el capítulo de preparación de datos."
    )
  )
}
```

```
atus_ml <- read_csv(
  ruta_atus_ml,
  show_col_types = FALSE
)
```

10.15 Preparar una muestra reproducible

Las SVM pueden requerir bastante memoria y tiempo cuando el conjunto es muy grande. Para fines didácticos se utiliza una muestra de hasta 6 000 accidentes.

```
set.seed(123)

atus_svm <- atus_ml |>
  sample_n(min(6000, nrow(atus_ml))) |>
  transmute(
    accidente_con_victimas = factor(
      accidente_con_victimas,
      levels = c("Solo daños", "Con víctimas")
    ),
    MES = factor(MES),
    ID_HORA = as.numeric(ID_HORA),
    DIASEMANA = factor(DIASEMANA),
    TIPACCID = factor(TIPACCID),
    CAUSAACCI = factor(CAUSAACCI)
  ) |>
  na.omit()

prop.table(table(atus_svm$accidente_con_victimas))
```

```
#>
#> Solo daños Con víctimas
#>      0.82      0.18
```

i Nota sobre la muestra

La muestra reduce el tiempo de ejecución y hace posible repetir el ejercicio en computadoras personales y en Google Colab. Para un estudio definitivo se recomienda evaluar tamaños mayores y documentar los recursos computacionales utilizados.

10.16 Dividir los datos de forma estratificada

```
set.seed(123)

indices_entrenamiento <- unlist(
  lapply(
    split(seq_len(nrow(atus_svm)), atus_svm$accidente_con_victimas),
    function(indices) {
      sample(indices, size = floor(0.70 * length(indices)))
    }
  )
)

entrenamiento_svm <- atus_svm[indices_entrenamiento, ]
prueba_svm <- atus_svm[-indices_entrenamiento, ]

prop.table(table(entrenamiento_svm$accidente_con_victimas))
```

```
#>
#> Solo daños Con víctimas
#>      0.82      0.18
```

```
prop.table(table(prueba_svm$accidente_con_victimas))
```

```
#>
#> Solo daños Con víctimas
#>      0.82      0.18
```

10.17 Calcular pesos para las clases

Si una clase aparece con menor frecuencia, la SVM puede recibir pesos inversamente proporcionales a sus frecuencias.

```
frecuencias <- table(entrenamiento_svm$accidente_con_victimas)

pesos_clase <- sum(frecuencias) /
  (length(frecuencias) * frecuencias)

pesos_clase <- as.numeric(pesos_clase)
```



```
names(pesos_clase) <- names(frecuencias)

pesos_clase
```

```
#> Solo daños Con víctimas
#> 0.6097561 2.7777778
```

La clase menos frecuente recibe un peso mayor, de modo que sus errores tengan más influencia durante el entrenamiento.

10.18 Ajustar una SVM lineal

```
set.seed(123)

modelo_svm_atus_lineal <- svm(
  accidente_con_victimas ~
    MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
  data = entrenamiento_svm,
  kernel = "linear",
  cost = 1,
  class.weights = pesos_clase,
  scale = TRUE
)

modelo_svm_atus_lineal
```

```
#>
#> Call:
#> svm(formula = accidente_con_victimas ~ MES + ID_HORA + DIASEMANA +
#> TIPACCID + CAUSAACCI, data = entrenamiento_svm, kernel = "linear",
#> cost = 1, class.weights = pesos_clase, scale = TRUE)
#>
#>
#> Parameters:
#> SVM-Type: C-classification
#> SVM-Kernel: linear
#> cost: 1
#>
#> Number of Support Vectors: 2215
```

10.19 Realizar predicciones

```
prediccion_svm_lineal <- predict(
  modelo_svm_atus_lineal,
  newdata = prueba_svm
)

matriz_svm_lineal <- table(
  Real = prueba_svm$accidente_con_victimas,
  Predicho = prediccion_svm_lineal
)

matriz_svm_lineal
```

```
#>               Predicho
#> Real               Solo daños Con víctimas
#> Solo daños         1172         304
#> Con víctimas       89         235
```

10.20 Función segura para calcular métricas

```
metricas_clasificacion <- function(
  real,
  predicho,
  positiva = "Con víctimas"
) {
  real <- factor(real)
  predicho <- factor(predicho, levels = levels(real))

  negativa <- setdiff(levels(real), positiva)

  if (length(negativa) != 1) {
    stop("La función requiere exactamente dos clases.")
  }

  negativa <- negativa[1]
  matriz <- table(Real = real, Predicho = predicho)

  VP <- matriz[positiva, positiva]
  FN <- matriz[positiva, negativa]
  FP <- matriz[negativa, positiva]
  VN <- matriz[negativa, negativa]
```

```

dividir <- function(a, b) {
  if (is.na(a) || is.na(b) || b == 0) {
    return(NA_real_)
  }
  as.numeric(a / b)
}

exactitud <- dividir(VP + VN, VP + FN + FP + VN)
sensibilidad <- dividir(VP, VP + FN)
especificidad <- dividir(VN, VN + FP)
precision <- dividir(VP, VP + FP)
f1 <- dividir(
  2 * precision * sensibilidad,
  precision + sensibilidad
)

data.frame(
  exactitud = exactitud,
  sensibilidad = sensibilidad,
  especificidad = especificidad,
  precision = precision,
  f1 = f1
)
}

```

10.21 Evaluar la SVM lineal

```

metricas_svm_lineal <- metricas_clasificacion(
  real = prueba_svm$accidente_con_victimas,
  predicho = prediccion_svm_lineal
)

metricas_svm_lineal

```

```

#>   exactitud sensibilidad especificidad precision      f1
#> 1 0.7816667    0.7253086    0.7940379 0.4359926 0.5446118

```

Interpretación

La sensibilidad indica la proporción de accidentes con víctimas detectada por el modelo. La especificidad indica la proporción de accidentes de solo daños reconocida correctamente.

te. Cuando las clases están desbalanceadas, estas métricas suelen ser más informativas que la exactitud aislada.

10.22 Ajustar una SVM con kernel radial

```
set.seed(123)

modelo_svm_atus_radial <- svm(
  accidente_con_victimas ~
    MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
  data = entrenamiento_svm,
  kernel = "radial",
  cost = 1,
  gamma = 0.03,
  class.weights = pesos_clase,
  scale = TRUE
)

modelo_svm_atus_radial
```

```
#>
#> Call:
#> svm(formula = accidente_con_victimas ~ MES + ID_HORA + DIASEMANA +
#>     TIPACCID + CAUSAACCI, data = entrenamiento_svm, kernel = "radial",
#>     cost = 1, gamma = 0.03, class.weights = pesos_clase, scale = TRUE)
#>
#>
#> Parameters:
#>   SVM-Type:  C-classification
#> SVM-Kernel:  radial
#>       cost:  1
#>
#> Number of Support Vectors:  2318
```

10.23 Evaluar la SVM radial

```
prediccion_svm_radial <- predict(
  modelo_svm_atus_radial,
```

```

newdata = prueba_svm
)

matriz_svm_radial <- table(
  Real = prueba_svm$accidente_con_victimas,
  Predicho = prediccion_svm_radial
)

matriz_svm_radial

```

```

#>               Predicho
#> Real              Solo daños Con víctimas
#> Solo daños          1180          296
#> Con víctimas         93          231

```

```

metricas_svm_radial <- metricas_clasificacion(
  real = prueba_svm$accidente_con_victimas,
  predicho = prediccion_svm_radial
)

metricas_svm_radial

```

```

#> exactitud sensibilidad especificidad precision      f1
#> 1 0.7838889      0.712963      0.799458 0.4383302 0.5428907

```

10.24 Comparar los dos modelos

```

comparacion_svm <- bind_rows(
  transform(metricas_svm_lineal, modelo = "SVM lineal"),
  transform(metricas_svm_radial, modelo = "SVM radial")
) |>
  select(modelo, everything())

comparacion_svm

```

```

#>      modelo exactitud sensibilidad especificidad precision      f1
#> 1 SVM lineal 0.7816667      0.7253086      0.7940379 0.4359926 0.5446118
#> 2 SVM radial 0.7838889      0.7129630      0.7994580 0.4383302 0.5428907

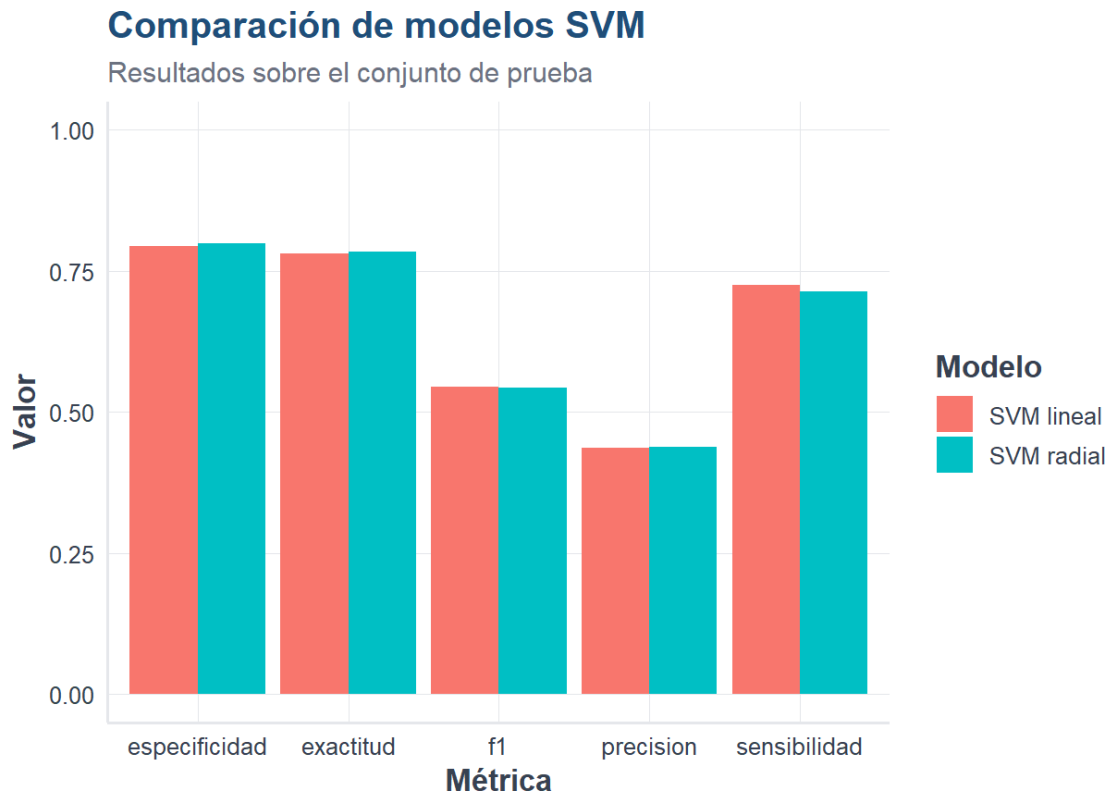
```

```

comparacion_larga <- comparacion_svm |>
  tidyr::pivot_longer(
    cols = -modelo,
    names_to = "metrica",
    values_to = "valor"
  )

ggplot(
  comparacion_larga,
  aes(x = metrica, y = valor, fill = modelo)
) +
  geom_col(position = "dodge") +
  coord_cartesian(ylim = c(0, 1)) +
  labs(
    title = "Comparación de modelos SVM",
    subtitle = "Resultados sobre el conjunto de prueba",
    x = "Métrica",
    y = "Valor",
    fill = "Modelo"
  ) +
  tema_libro()

```



10.25 Selección de hiperparámetros con validación cruzada

La función `tune.svm()` permite probar diferentes combinaciones de hiperparámetros. Para mantener un tiempo razonable se usa una cuadrícula pequeña.

```
set.seed(123)

muestra_ajuste <- entrenamiento_svm |>
  sample_n(min(3000, nrow(entrenamiento_svm)))

ajuste_svm <- tune.svm(
  accidente_con_victimas ~
    MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
  data = muestra_ajuste,
  kernel = "radial",
  cost = c(0.5, 1, 2),
  gamma = c(0.01, 0.03, 0.05),
  tunecontrol = tune.control(cross = 5),
  scale = TRUE
)

ajuste_svm$best.parameters
```

```
#>   gamma cost
#> 2  0.03  0.5
```

```
mejor_modelo_svm <- ajuste_svm$best.model

prediccion_mejor_svm <- predict(
  mejor_modelo_svm,
  newdata = prueba_svm
)

metricas_mejor_svm <- metricas_clasificacion(
  real = prueba_svm$accidente_con_victimas,
  predicho = prediccion_mejor_svm
)

metricas_mejor_svm
```

```
#>   exactitud sensibilidad especificidad precision    f1
#> 1 0.8533333    0.1851852             1         1 0.3125
```

⚠ Validación sin fuga de información

Los hiperparámetros deben seleccionarse usando exclusivamente los datos de entrenamiento. El conjunto de prueba se reserva para la evaluación final.

10.26 Ventajas y limitaciones

10.26.1 Ventajas

- puede construir fronteras lineales y no lineales;
- funciona bien en espacios con muchas variables;
- el margen máximo favorece la generalización;
- utiliza principalmente los vectores de soporte para definir la frontera;
- permite ponderar clases desbalanceadas.

10.26.2 Limitaciones

- puede ser lenta con conjuntos de datos muy grandes;
- requiere elegir kernel e hiperparámetros;
- sus resultados son menos interpretables que los de una regresión logística o un árbol pequeño;
- la estandarización de variables numéricas es importante;
- una búsqueda extensa de hiperparámetros puede consumir muchos recursos.

10.27 Recomendaciones prácticas

1. Comience con una SVM lineal como referencia.
2. Estandarice las variables numéricas.
3. Use validación cruzada para seleccionar `cost` y `gamma`.
4. Revise sensibilidad, especificidad, precisión y F1, no solo exactitud.
5. Utilice pesos de clase cuando exista desbalance.
6. Trabaje inicialmente con una muestra si el conjunto es muy grande.
7. Evalúe el modelo final una sola vez sobre datos de prueba no utilizados durante el ajuste.

10.28 Actividad guiada

Modifique el código para comparar:

- `cost = 0.1, 1` y `10` en la SVM lineal;

- `gamma = 0.01, 0.05 y 0.10` en la SVM radial;
- modelos con y sin `class.weights`;
- muestras de 3 000 y 6 000 accidentes.

Registre para cada modelo:

- número de vectores de soporte;
- tiempo de entrenamiento;
- exactitud;
- sensibilidad;
- especificidad;
- precisión;
- valor F1.

10.29 Ejercicios

1. Explique con sus propias palabras qué es un vector de soporte.
2. ¿Qué diferencia existe entre una frontera lineal y una frontera construida con kernel radial?
3. ¿Qué ocurre generalmente cuando `cost` es demasiado grande?
4. ¿Por qué es importante estandarizar variables numéricas?
5. Entrene una SVM lineal sin pesos de clase y compare su sensibilidad con el modelo ponderado.
6. Amplíe la cuadrícula de validación cruzada y explique si el mejor modelo cambia.
7. Compare la mejor SVM con la regresión logística y Random Forest estudiados anteriormente.
8. Analice si una mejora pequeña en exactitud justifica una pérdida importante de interpretabilidad.

10.30 Conclusiones

Las máquinas de vectores de soporte buscan una frontera con margen amplio y se apoyan en un subconjunto de observaciones denominado vectores de soporte. Mediante kernels pueden representar relaciones no lineales y producir modelos predictivos potentes.

Sin embargo, una SVM requiere seleccionar cuidadosamente sus hiperparámetros y evaluar su desempeño con datos no utilizados durante el entrenamiento. En problemas con clases desbalanceadas, la ponderación de clases y el análisis conjunto de sensibilidad, especificidad, precisión y F1 son fundamentales.

Fuente de los datos de aplicación: elaboración propia con datos del INEGI, Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS), 2024.

10.31 Referencias fundamentales de SVM

Las máquinas de vectores de soporte fueron presentadas formalmente por Cortes y Vapnik (1995). El tutorial de Burges (1998) desarrolla la intuición geométrica, los márgenes y el uso de kernels. Una introducción aplicada en R puede consultarse en James et al. (2021).

Capítulo 11

Clasificador Naive Bayes

11.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de:

- explicar el teorema de Bayes aplicado a clasificación;
- interpretar probabilidades previas, verosimilitudes y probabilidades posteriores;
- comprender el supuesto de independencia condicional;
- distinguir entre Naive Bayes categórico y gaussiano;
- construir un clasificador sencillo sin paquetes especializados;
- entrenar y evaluar un modelo Naive Bayes en R;
- aplicar el algoritmo a una muestra de la base ATUS;
- reconocer sus ventajas, limitaciones y condiciones de uso.

11.2 Introducción

Naive Bayes es una familia de clasificadores probabilísticos basada en el teorema de Bayes. Su nombre incluye la palabra *naive* —ingenuo— porque supone que las variables predictoras son condicionalmente independientes una vez conocida la clase.

Este supuesto rara vez se cumple de manera exacta en datos reales. Sin embargo, el clasificador puede funcionar sorprendentemente bien incluso cuando existe cierta dependencia entre las variables (Domingos y Pazzani 1997; Hand y Yu 2001).

La principal fortaleza del método es que transforma la clasificación en una comparación de probabilidades. Para una observación nueva, se calcula qué clase resulta más probable dados los valores de sus predictores.

11.3 Recordatorio del teorema de Bayes

Para una clase C y un conjunto de características X , el teorema de Bayes establece:

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}$$

donde:

- $P(C)$ es la **probabilidad previa** de la clase;
- $P(X | C)$ es la **verosimilitud** de observar las características cuando la clase es C ;
- $P(C | X)$ es la **probabilidad posterior** de la clase después de observar X ;
- $P(X)$ es una constante común al comparar las clases.

Para clasificar no es necesario calcular explícitamente $P(X)$. Basta comparar:

$$P(C | X) \propto P(X | C)P(C)$$

11.4 El supuesto ingenuo de independencia

Si una observación tiene predictores $X_1, X_2, \dots, X_p$, Naive Bayes supone que, conocida la clase, sus contribuciones pueden multiplicarse:

$$P(X_1, \dots, X_p | C) = \prod_{j=1}^p P(X_j | C)$$

Por tanto:

$$P(C | X_1, \dots, X_p) \propto P(C) \prod_{j=1}^p P(X_j | C)$$

Idea esencial

Naive Bayes calcula un puntaje probabilístico para cada clase. La observación se asigna a la clase cuyo producto entre probabilidad previa y verosimilitudes es mayor.

11.5 Ejemplo manual con variables categóricas

Consideremos un pequeño conjunto de accidentes descritos por dos variables: condición climática y horario.

```
datos_nb <- data.frame(
  clima = c("Seco", "Seco", "Lluvia", "Lluvia", "Seco",
            "Lluvia", "Seco", "Lluvia", "Seco", "Lluvia"),
  horario = c("Día", "Noche", "Día", "Noche", "Día",
              "Noche", "Noche", "Día", "Día", "Noche"),
  victimas = factor(
    c("No", "No", "Sí", "Sí", "No", "Sí", "Sí", "No", "No", "Sí")
  )
)
```

```
)
datos_nb
```

```
#>      clima horario victimas
#> 1     Seco      Día       No
#> 2     Seco     Noche       No
#> 3  Lluvia      Día       Sí
#> 4  Lluvia     Noche       Sí
#> 5     Seco      Día       No
#> 6  Lluvia     Noche       Sí
#> 7     Seco     Noche       Sí
#> 8  Lluvia      Día       No
#> 9     Seco      Día       No
#> 10 Lluvia     Noche       Sí
```

11.6 Calcular las probabilidades previas

```
previas <- prop.table(table(datos_nb$victimas))
previas
```

```
#>
#> No  Sí
#> 0.5 0.5
```

Las probabilidades previas representan la proporción inicial de cada clase antes de observar las variables predictoras.

11.7 Calcular probabilidades condicionales

```
prob_clima <- prop.table(
  table(datos_nb$clima, datos_nb$victimas),
  margin = 2
)
```

```

prob_horario <- prop.table(
  table(datos_nb$horario, datos_nb$victimas),
  margin = 2
)

prob_clima

```

```

#>
#>           No  Sí
#>  Lluvia 0.2 0.8
#>   Seco  0.8 0.2

```

```

prob_horario

```

```

#>
#>           No  Sí
#>   Día  0.8 0.2
#>  Noche 0.2 0.8

```

11.8 Clasificar una observación manualmente

Supongamos un accidente ocurrido con lluvia durante la noche. Comparamos los puntajes para las dos clases.

```

puntaje_si <-
  previas["Sí"] *
  prob_clima["Lluvia", "Sí"] *
  prob_horario["Noche", "Sí"]

puntaje_no <-
  previas["No"] *
  prob_clima["Lluvia", "No"] *
  prob_horario["Noche", "No"]

c(Sí = puntaje_si, No = puntaje_no)

```

```

#> Sí.Sí No.No
#>  0.32  0.02

```

```
posteriores <- c(Sí = puntaje_si, No = puntaje_no)
posteriores <- posteriores / sum(posteriores)
posteriores
```

```
#>      Sí.Sí      No.No
#> 0.94117647 0.05882353
```

i Interpretación

Los productos iniciales son proporcionales a las probabilidades posteriores. Al dividir cada puntaje entre la suma de ambos se obtienen valores que suman uno y pueden interpretarse como probabilidades normalizadas bajo el modelo.

11.9 El problema de las probabilidades iguales a cero

Cuando una combinación nunca aparece en el entrenamiento, una de las probabilidades condicionales puede ser cero. Al multiplicar, todo el puntaje de esa clase se vuelve cero.

Una solución habitual es el **suavizado de Laplace**, que agrega una pequeña cantidad a los conteos antes de calcular las probabilidades.

$$\widehat{P}(X_j = x \mid C = c) = \frac{n_{x,c} + \alpha}{n_c + \alpha k}$$

Aquí, k es el número de categorías y normalmente se utiliza $\alpha = 1$.

11.10 Naive Bayes gaussiano

Cuando un predictor es numérico continuo, una opción común es suponer que, dentro de cada clase, sigue una distribución normal:

$$X_j \mid C = c \sim N(\mu_{jc}, \sigma_{jc}^2)$$

La media y la desviación estándar se estiman por separado para cada clase. Esta variante se denomina **Naive Bayes gaussiano**.

11.11 Instalar y cargar paquetes

```
paquetes_nb <- c("readr", "dplyr", "ggplot2", "e1071")

faltantes_nb <- paquetes_nb[
  !vapply(paquetes_nb, requireNamespace, logical(1), quietly = TRUE)
]

if (length(faltantes_nb) > 0) {
  install.packages(faltantes_nb, repos = "https://cloud.r-project.org")
}

library(readr)
library(dplyr)
library(ggplot2)
library(e1071)
source("util_graficas.R")
```

11.12 Cargar la base preparada de ATUS

```
ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (!file.exists(ruta_atus_ml)) {
  stop(
    paste(
      "No se encontró datos/atus_ml_preparado.csv.",
      "Ejecute primero el capítulo de preparación de datos."
    )
  )
}

atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
```

11.13 Preparar una muestra para el modelo

```
set.seed(123)

atus_nb <- atus_ml |>
```



```

sample_n(min(20000, nrow(atus_ml))) |>
transmute(
  accidente_con_victimas = factor(
    accidente_con_victimas,
    levels = c("Solo daños", "Con víctimas")
  ),
  MES = factor(MES),
  ID_HORA = as.numeric(ID_HORA),
  DIASEMANA = factor(DIASEMANA),
  TIPACCID = factor(TIPACCID),
  CAUSAACCI = factor(CAUSAACCI)
) |>
na.omit()

prop.table(table(atus_nb$accidente_con_victimas))

```

```

#>
#> Solo daños Con víctimas
#>      0.82805      0.17195

```

11.14 División estratificada en entrenamiento y prueba

```

set.seed(123)

indices_nb <- unlist(
  lapply(
    split(seq_len(nrow(atus_nb)), atus_nb$accidente_con_victimas),
    function(indices) {
      sample(indices, size = floor(0.70 * length(indices)))
    }
  )
)

entrenamiento_nb <- atus_nb[indices_nb, ]
prueba_nb <- atus_nb[-indices_nb, ]

prop.table(table(entrenamiento_nb$accidente_con_victimas))

```

```

#>
#> Solo daños Con víctimas
#>      0.8280591      0.1719409

```

```
prop.table(table(prueba_nb$accidente_con_victimas))
```

```
#>
#> Solo daños Con víctimas
#> 0.8280287 0.1719713
```

11.15 Entrenar el modelo Naive Bayes

```
modelo_nb <- naiveBayes(
  accidente_con_victimas ~
    MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
  data = entrenamiento_nb,
  laplace = 1
)
modelo_nb
```

```
#>
#> Naive Bayes Classifier for Discrete Predictors
#>
#> Call:
#> naiveBayes.default(x = X, y = Y, laplace = laplace)
#>
#> A-priori probabilities:
#> Y
#> Solo daños Con víctimas
#> 0.8280591 0.1719409
#>
#> Conditional probabilities:
#>
#> MES
#> Y
#> Solo daños 0.08178214 0.08307480 0.08436746 0.08445364 0.08807308
#> Con víctimas 0.10045473 0.08226540 0.08763952 0.08474576 0.09466722
#>
#> MES
#> Y
#> Solo daños 0.08117890 0.07350913 0.07945536 0.08048949 0.08781455
#> Con víctimas 0.06986358 0.06779661 0.08557255 0.07358413 0.09053328
#>
#> MES
```

```

#> Y              11              12
#> Solo daños      0.08919338 0.08660807
#> Con víctimas    0.07937164 0.08350558
#>
#> ID_HORA
#> Y              [,1]      [,2]
#> Solo daños      13.61051 6.079141
#> Con víctimas    13.49813 6.015211
#>
#> DIASEMANA
#> Y      Domingo      Jueves      lunes      Martes Miercoles      Sabado
#> Solo daños      0.1257867 0.1340633 0.1976895 0.1265626 0.1334598 0.1409604
#> Con víctimas    0.1615576 0.1342171 0.1296603 0.1429163 0.1296603 0.1528583
#>
#> DIASEMANA
#> Y      Viernes
#> Solo daños      0.1414777
#> Con víctimas    0.1491301
#>
#> TIPACCID
#> Y      Caída de pasajero Certificado cero Colisión con animal
#> Solo daños      8.616975e-05      5.247738e-02      2.498923e-03
#> Con víctimas    2.561983e-02      4.132231e-04      5.785124e-03
#>
#> TIPACCID
#> Y      Colisión con ciclista Colisión con ferrocarril
#> Solo daños      4.394657e-03      5.170185e-04
#> Con víctimas    2.892562e-02      1.652893e-03
#>
#> TIPACCID
#> Y      Colisión con motocicleta Colisión con objeto fijo
#> Solo daños      1.192589e-01      1.193451e-01
#> Con víctimas    3.661157e-01      6.983471e-02
#>
#> TIPACCID
#> Y      Colisión con peatón (atropellamiento)
#> Solo daños      8.616975e-05
#> Con víctimas    1.590909e-01
#>
#> TIPACCID
#> Y      Colisión con vehículo automotor      Incendio      Otro
#> Solo daños      6.473934e-01 1.034037e-03 6.893580e-03
#> Con víctimas    2.136364e-01 4.132231e-04 7.024793e-03
#>
#> TIPACCID
#> Y      Salida del camino      Volcadura
#> Solo daños      2.498923e-02 2.102542e-02
#> Con víctimas    3.429752e-02 8.719008e-02
#>

```

```
#>          CAUSAACCI
#> Y          Certificado cero      Conductor Falla del vehículo
#> Solo daños      0.0525090533 0.9173133299      0.0093981721
#> Con víctimas    0.0004144219 0.9270617489      0.0120182346
#>          CAUSAACCI
#> Y          Mala condición del camino      Otra Peatón o pasajero
#> Solo daños      0.0131057079 0.0066390757      0.0010346611
#> Con víctimas    0.0207210941 0.0107749689      0.0290095317
```

El argumento `laplace = 1` aplica suavizado de Laplace a los predictores categóricos.

11.16 Obtener predicciones de clase

```
prediccion_nb <- predict(
  modelo_nb,
  newdata = prueba_nb,
  type = "class"
)

head(prediccion_nb)
```

```
#> [1] Solo daños Solo daños Solo daños Solo daños Solo daños Solo daños
#> Levels: Solo daños Con víctimas
```

11.17 Obtener probabilidades posteriores

```
probabilidades_nb <- predict(
  modelo_nb,
  newdata = prueba_nb,
  type = "raw"
)

head(probabilidades_nb)
```

```
#>      Solo daños Con víctimas
#> [1,]  0.6232265  0.37677355
#> [2,]  0.9286211  0.07137891
```

```
#> [3,] 0.9364654 0.06353463
#> [4,] 0.7437280 0.25627196
#> [5,] 0.9315564 0.06844364
#> [6,] 0.9314021 0.06859794
```

11.18 Construir la matriz de confusión

```
matriz_nb <- table(
  Real = prueba_nb$accidente_con_victimas,
  Predicho = prediccion_nb
)

matriz_nb
```

```
#>               Predicho
#> Real              Solo daños Con víctimas
#> Solo daños          4900          69
#> Con víctimas        775          257
```

11.19 Calcular las métricas

```
division_segura_nb <- function(numerador, denominador) {
  if (is.na(denominador) || denominador == 0) {
    return(NA_real_)
  }

  numerador / denominador
}
```

```
clase_positiva <- "Con víctimas"
clase_negativa <- "Solo daños"

vp <- matriz_nb[clase_positiva, clase_positiva]
fn <- matriz_nb[clase_positiva, clase_negativa]
fp <- matriz_nb[clase_negativa, clase_positiva]
vn <- matriz_nb[clase_negativa, clase_negativa]

exactitud <- division_segura_nb(vp + vn, vp + vn + fp + fn)
```

```

sensibilidad <- division_segura_nb(vp, vp + fn)
especificidad <- division_segura_nb(vn, vn + fp)
precision <- division_segura_nb(vp, vp + fp)
f1 <- if (is.na(precision) || is.na(sensibilidad) ||
        precision + sensibilidad == 0) {
  NA_real_
} else {
  2 * precision * sensibilidad / (precision + sensibilidad)
}

metricas_nb <- data.frame(
  Metrica = c(
    "Exactitud", "Sensibilidad", "Especificidad", "Precisión", "F1"
  ),
  Valor = c(exactitud, sensibilidad, especificidad, precision, f1)
)

metricas_nb

```

```

#>      Metrica      Valor
#> 1   Exactitud 0.8593568
#> 2 Sensibilidad 0.2490310
#> 3 Especificidad 0.9861139
#> 4   Precisión 0.7883436
#> 5           F1 0.3784978

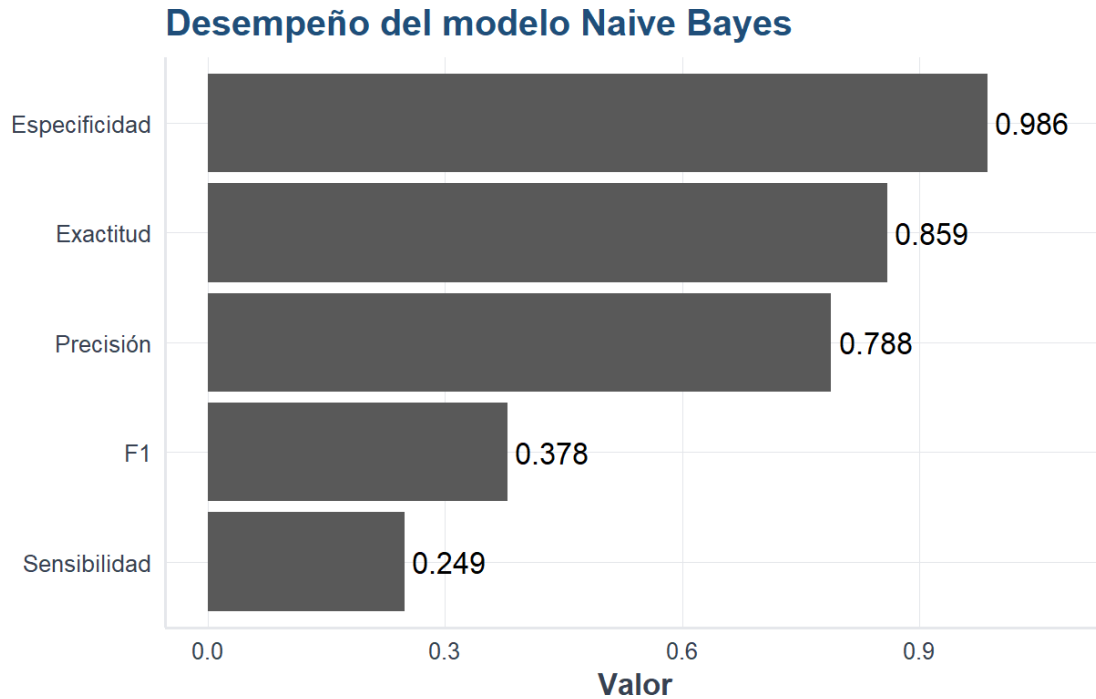
```

11.20 Visualizar las métricas

```

ggplot(metricas_nb, aes(x = reorder(Metrica, Valor), y = Valor)) +
  geom_col() +
  geom_text(
    aes(label = sprintf("%.3f", Valor)),
    hjust = -0.1,
    na.rm = TRUE
  ) +
  coord_flip() +
  scale_y_continuous(limits = c(0, 1.08)) +
  labs(
    title = "Desempeño del modelo Naive Bayes",
    x = NULL,
    y = "Valor"
  ) +
  tema_libro()

```



Fuente: elaboración propia mediante R con datos del INEGI, Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS), 2024.

11.21 Inspeccionar las probabilidades aprendidas

```
modelo_nb$sapriori
```

```
#> Y
#> Solo daños Con víctimas
#>      11592      2407
```

```
modelo_nb$tables$MES
```

```
#>      MES
#> Y      01      02      03      04      05
#> Solo daños  0.08178214 0.08307480 0.08436746 0.08445364 0.08807308
#> Con víctimas 0.10045473 0.08226540 0.08763952 0.08474576 0.09466722
```

```
#>
#> Y MES
#> Solo daños 0.08117890 0.07350913 0.07945536 0.08048949 0.08781455
#> Con víctimas 0.06986358 0.06779661 0.08557255 0.07358413 0.09053328
#>
#> Y MES
#> Solo daños 0.08919338 0.08660807
#> Con víctimas 0.07937164 0.08350558
```

La primera salida muestra las probabilidades previas de las clases. La segunda resume las probabilidades condicionales estimadas para los meses.

11.22 Ventajas de Naive Bayes

- Es sencillo y rápido de entrenar.
- Produce probabilidades de pertenencia a cada clase.
- Funciona con conjuntos de datos de alta dimensión.
- Puede combinar predictores categóricos y numéricos.
- Requiere menos datos que modelos más complejos.
- Constituye una excelente línea base probabilística.

11.23 Limitaciones

- El supuesto de independencia condicional puede ser poco realista.
- Variables muy correlacionadas pueden contar información repetida.
- Las probabilidades estimadas no siempre están bien calibradas.
- Las categorías no observadas requieren suavizado.
- La forma gaussiana puede ser inadecuada para variables numéricas asimétricas.

Precaución metodológica

Un buen valor de exactitud no demuestra que el supuesto de independencia sea verdadero. El modelo debe evaluarse con datos no utilizados en el entrenamiento y con métricas apropiadas para la clase de interés.

11.24 Comparación conceptual con otros algoritmos

Método	Idea principal	Escalamiento	Interpretabilidad	Relaciones no lineales
Regresión logística	Modela probabilidades mediante una función logística	Recomendable	Alta	Limitada sin transformaciones
k-NN	Clasifica por cercanía	Necesario	Media	Sí
Árbol	Divide el espacio mediante reglas	No necesario	Alta	Sí
Random Forest	Combina muchos árboles	No necesario	Media	Sí
SVM	Maximiza el margen	Muy recomendable	Media-baja	Sí, mediante kernels
Naive Bayes	Multiplica probabilidades condicionales	Depende de la variante	Alta	Mediante distribuciones por clase

11.25 Actividades para el lector

1. Cambie el suavizado de Laplace a 0, 0.5 y 2. Compare las métricas.
2. Elimine una variable predictora y examine si mejora la sensibilidad.
3. Modifique el tamaño de la muestra y mida el tiempo de entrenamiento.
4. Compare Naive Bayes con la regresión logística utilizando exactamente la misma partición.
5. Examine qué categorías presentan probabilidades condicionales muy distintas entre las clases.
6. Explique por qué dos variables correlacionadas pueden influir doblemente en el producto de verosimilitudes.

11.26 Conclusiones

Naive Bayes muestra que un modelo relativamente simple puede construir clasificaciones útiles mediante probabilidades. Su supuesto de independencia facilita el cálculo y reduce el costo computacional, aunque exige interpretar los resultados con prudencia.

En ATUS, el método proporciona una línea base probabilística rápida para predecir si un accidente pertenece a la clase con víctimas o a la clase de solo daños. Su desempeño debe juzgarse junto con la sensibilidad, la especificidad y F1, no únicamente por la exactitud.

11.27 Referencias fundamentales de Naive Bayes

El análisis de las condiciones bajo las cuales el clasificador bayesiano simple puede ser óptimo fue desarrollado por Domingos y Pazzani (1997). Una discusión crítica sobre por qué el método puede funcionar bien pese a su supuesto ingenuo aparece en Hand y Yu (2001). Para una introducción moderna dentro del aprendizaje estadístico puede consultarse James et al. (2021).

Capítulo 12

Redes neuronales artificiales

12.1 Objetivos de aprendizaje

Al finalizar este capítulo podrás:

- explicar la estructura básica de una neurona artificial;
- interpretar entradas, pesos, sesgo y funciones de activación;
- describir el funcionamiento de una red neuronal multicapa;
- comprender la propagación hacia adelante y la retropropagación;
- distinguir entre clasificación binaria y multiclase;
- preparar correctamente los datos para una red neuronal;
- implementar una red sencilla desde cero en R;
- entrenar modelos con `neuralnet`;
- construir una red moderna con `keras`;
- evaluar el desempeño mediante matriz de confusión y métricas;
- reconocer sobreajuste, regularización y errores frecuentes.

12.2 Introducción

Las **redes neuronales artificiales** son modelos computacionales inspirados, de forma muy simplificada, en la manera en que las neuronas biológicas reciben, transforman y transmiten señales.

Una red neuronal puede aprender relaciones complejas entre variables de entrada y una variable de respuesta. Esto la hace útil para:

- clasificación;
- regresión;
- reconocimiento de imágenes;
- procesamiento de señales;
- detección de patrones;
- predicción de series;
- aplicaciones biomédicas y ambientales.

La idea central es construir muchas unidades simples, llamadas **neuronas artificiales**, conectadas entre sí mediante pesos ajustables.

12.3 La neurona artificial

Una neurona recibe varias entradas:

$$x_1, x_2, \dots, x_p$$

Cada entrada tiene un peso:

$$w_1, w_2, \dots, w_p$$

La neurona calcula una combinación lineal:

$$z = w_1x_1 + w_2x_2 + \dots + w_px_p + b$$

donde b es el **sesgo**.

Después aplica una función de activación:

$$a = f(z)$$

El resultado a se transmite a otras neuronas o se utiliza como salida.

12.3.1 Interpretación

- Las entradas son las características del problema.
- Los pesos indican la importancia de cada entrada.
- El sesgo permite desplazar la frontera de decisión.
- La función de activación introduce no linealidad.
- La salida representa una predicción o una señal intermedia.

12.4 Ejemplo manual de una neurona

Supongamos que las entradas, los pesos y el sesgo son:

$$\begin{aligned}x_1 &= 0.8, & x_2 &= 0.4, \\w_1 &= 1.2, & w_2 &= -0.7, & b &= 0.1.\end{aligned}$$

La combinación lineal se obtiene sustituyendo los valores:

$$\begin{aligned} z &= w_1x_1 + w_2x_2 + b \\ &= (1.2)(0.8) + (-0.7)(0.4) + 0.1 \\ &= 0.96 - 0.28 + 0.1 \\ &= 0.78. \end{aligned}$$

Si usamos la función sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

obtenemos:

$$\sigma(0.78) \approx 0.686$$

La neurona produce una salida cercana a 0.686.

12.5 Funciones de activación

Las funciones de activación permiten que la red represente relaciones no lineales.

12.5.1 Función escalón

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

Fue utilizada en los primeros perceptrones.

12.5.2 Función sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Características:

- produce valores entre 0 y 1;
- es útil en clasificación binaria;
- puede interpretarse como probabilidad;
- puede saturarse para valores grandes.

12.5.3 Tangente hiperbólica

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Produce valores entre -1 y 1.

12.5.4 ReLU

$$\text{ReLU}(z) = \max(0, z)$$

Es una de las activaciones más utilizadas en capas ocultas.

12.5.5 Softmax

Para clasificación multiclase:

$$P(y = k) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Convierte las salidas en probabilidades que suman 1.

12.6 El perceptrón

El perceptrón es uno de los modelos neuronales más sencillos.

Calcula:

$$\hat{y} = f(\mathbf{w}^T \mathbf{x} + b)$$

y actualiza los pesos cuando comete errores.

La regla básica de actualización es:

$$w_j^{nuevo} = w_j^{anterior} + \eta(y - \hat{y})x_j$$

donde:

- η es la tasa de aprendizaje;
- y es la clase real;
- $\hat{y}$ es la predicción.

12.6.1 Limitación

Un perceptrón simple solo resuelve problemas linealmente separables. El problema XOR es el ejemplo clásico que requiere una capa oculta.

12.7 Estructura de una red multicapa

Una red neuronal multicapa suele tener:

1. capa de entrada;
2. una o más capas ocultas;
3. capa de salida.

Cada neurona de una capa puede conectarse con las neuronas de la siguiente.

12.7.1 Capa de entrada

Recibe las variables predictoras.

12.7.2 Capas ocultas

Aprenden combinaciones intermedias de las características.

12.7.3 Capa de salida

Produce la predicción final.

Ejemplos:

- una neurona sigmoide para clasificación binaria;
- varias neuronas softmax para clasificación multiclase;
- una neurona lineal para regresión.

12.8 Propagación hacia adelante

La **propagación hacia adelante** calcula la salida de la red a partir de las entradas.

Para una capa oculta:

$$\mathbf{h} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

Para la salida:

$$\hat{\mathbf{y}} = g(\mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)})$$

La red transforma sucesivamente la información hasta producir una predicción.

12.9 Función de pérdida

La función de pérdida mide qué tan lejos está la predicción del valor real.

12.9.1 Error cuadrático medio

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Se usa principalmente en regresión.

12.9.2 Entropía cruzada binaria

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

Se utiliza en clasificación binaria.

12.9.3 Entropía cruzada categórica

$$L = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\hat{p}_{ik})$$

Se utiliza en clasificación multiclase.

12.10 Descenso de gradiente

El entrenamiento busca valores de los pesos que minimicen la pérdida.

La actualización general es:

$$w^{\text{nuevo}} = w^{\text{anterior}} - \eta \frac{\partial L}{\partial w}$$

donde η es la tasa de aprendizaje.

12.10.1 Tasa demasiado pequeña

- aprendizaje lento;
- muchas iteraciones.

12.10.2 Tasa demasiado grande

- oscilaciones;
- divergencia;
- pérdida inestable.

12.11 Retropropagación

La **retropropagación** calcula cómo contribuye cada peso al error.

El proceso es:

1. realizar propagación hacia adelante;
2. calcular la pérdida;
3. obtener derivadas mediante la regla de la cadena;
4. propagar el error hacia atrás;
5. actualizar pesos y sesgos;
6. repetir.

La retropropagación no es un modelo distinto, sino el mecanismo principal para entrenar redes multicapa.

12.12 Implementación manual de una neurona en R

```

sigmoide <- function(z) {
  1 / (1 + exp(-z))
}

x <- c(0.8, 0.4)
w <- c(1.2, -0.7)
b <- 0.1

z <- sum(x * w) + b
salida <- sigmoide(z)

z

```

```
#> [1] 0.78
```

```
salida
```

```
#> [1] 0.6856801
```

12.13 Laboratorio interactivo: construye una neurona artificial

12.13.1 Laboratorio interactivo disponible en la versión web

La versión web del capítulo incluye un laboratorio donde pueden modificarse las entradas, los pesos, el sesgo, la función de activación y el umbral de clasificación de una neurona artificial.

12.14 Perceptrón desde cero en R

```

entrenar_perceptron <- function(X, y,
                                eta = 0.1,
                                epocas = 100) {

  X <- as.matrix(X)
  y <- as.numeric(y)

  pesos <- rep(0, ncol(X))
  sesgo <- 0

```

```

for (epoca in seq_len(epocas)) {

  for (i in seq_len(nrow(X))) {

    z <- sum(X[i, ] * pesos) + sesgo
    pred <- ifelse(z >= 0, 1, 0)
    error <- y[i] - pred

    pesos <- pesos + eta * error * X[i, ]
    sesgo <- sesgo + eta * error
  }
}

list(
  pesos = pesos,
  sesgo = sesgo
)
}

```

Ejemplo con la compuerta AND:

```

X_and <- data.frame(
  x1 = c(0, 0, 1, 1),
  x2 = c(0, 1, 0, 1)
)

y_and <- c(0, 0, 0, 1)

modelo_and <- entrenar_perceptron(
  X_and,
  y_and,
  eta = 0.1,
  epocas = 20
)

modelo_and

```

```

#> $pesos
#>  x1  x2
#> 0.2 0.1
#>
#> $sesgo
#> [1] -0.2

```

Función de predicción:

```

predecir_perceptron <- function(modelo, X) {

  X <- as.matrix(X)

  z <- as.numeric(
    X %*% modelo$pesos +
    modelo$sesgo
  )

  ifelse(z >= 0, 1, 0)
}

predecir_perceptron(
  modelo_and,
  X_and
)

```

```
#> [1] 0 0 0 1
```

12.15 Ejemplo de XOR

La compuerta XOR produce:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

No puede resolverse mediante una sola frontera lineal. Una capa oculta permite construir regiones más complejas.

12.16 Preparación de los datos

Antes de entrenar una red neuronal es recomendable:

- tratar valores faltantes;
- convertir categorías;
- separar entrenamiento, validación y prueba;
- normalizar variables numéricas;

- revisar desbalance;
- eliminar identificadores;
- evitar fuga de información.

12.17 Normalización

Las redes suelen entrenarse mejor cuando las variables tienen escalas comparables.

12.17.1 Estandarización

$$z = \frac{x - \bar{x}}{s}$$

12.17.2 Mínimo-máximo

$$x^* = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Los parámetros deben calcularse solo con entrenamiento.

```
set.seed(123)

indice <- sample(
  seq_len(nrow(iris)),
  size = round(0.70 * nrow(iris))
)

train <- iris[indice, ]
test <- iris[-indice, ]

medias <- sapply(train[, 1:4], mean)
desvios <- sapply(train[, 1:4], sd)

x_train <- scale(
  train[, 1:4],
  center = medias,
  scale = desvios
)

x_test <- scale(
  test[, 1:4],
  center = medias,
  scale = desvios
)
```

)

12.18 Clasificación binaria con neuralnet

i Ejecución de los ejemplos con neuralnet

Para que la generación del libro sea estable, los bloques que entrenan la red con `neuralnet` se muestran, pero no se ejecutan automáticamente durante el renderizado. Puedes ejecutarlos en RStudio después de instalar el paquete con `PREPARAR_PAQUETES_RED_NEURONAL.bat`.

Crearemos un problema con dos especies de `iris`.

```
iris_bin <- subset(
  iris,
  Species != "setosa"
)

iris_bin$clase <- ifelse(
  iris_bin$Species == "virginica",
  1,
  0
)

iris_bin$Species <- NULL
```

División:

```
set.seed(321)

idx <- sample(
  seq_len(nrow(iris_bin)),
  size = round(0.70 * nrow(iris_bin))
)

train_bin <- iris_bin[idx, ]
test_bin <- iris_bin[-idx, ]
```

Normalización segura:

```

variables <- setdiff(
  names(train_bin),
  "clase"
)

medias_bin <- sapply(
  train_bin[, variables],
  mean
)

desvios_bin <- sapply(
  train_bin[, variables],
  sd
)

train_bin[, variables] <- scale(
  train_bin[, variables],
  center = medias_bin,
  scale = desvios_bin
)

test_bin[, variables] <- scale(
  test_bin[, variables],
  center = medias_bin,
  scale = desvios_bin
)

```

Entrenamiento:

```

# Instalar una sola vez fuera del renderizado:
# install.packages("neuralnet", repos = "https://cloud.r-project.org")

if (!requireNamespace("neuralnet", quietly = TRUE)) {
  stop(
    "El paquete 'neuralnet' no está instalado. ",
    "Ejecute PREPARAR_PAQUETES_RED_NEURONAL.bat."
  )
}

formula_rna <- clase ~ .

modelo_rna <- neuralnet::neuralnet(
  formula = formula_rna,
  data = train_bin,
  hidden = c(5, 3),
  linear.output = FALSE,
  lifesign = "minimal",
  stepmax = 1e6
)

```

```
)
modelo_rna
```

12.19 Predicción con neuralnet

```
probabilidades <- neuralnet::compute(
  modelo_rna,
  test_bin[, variables]
)$net.result

prediccion <- ifelse(
  probabilidades >= 0.5,
  1,
  0
)

matriz <- table(
  Real = test_bin$clase,
  Predicho = prediccion
)

matriz
```

12.20 Métricas binarias

```
metricas_binarias <- function(real, predicho) {

  vp <- sum(real == 1 & predicho == 1)
  vn <- sum(real == 0 & predicho == 0)
  fp <- sum(real == 0 & predicho == 1)
  fn <- sum(real == 1 & predicho == 0)

  data.frame(
    exactitud = (vp + vn) /
      (vp + vn + fp + fn),

    sensibilidad = ifelse(
      vp + fn == 0,
      NA,
```



```

    vp / (vp + fn)
  ),

  especificidad = ifelse(
    vn + fp == 0,
    NA,
    vn / (vn + fp)
  ),

  precision = ifelse(
    vp + fp == 0,
    NA,
    vp / (vp + fp)
  )
)
}

metricas_binarias(
  test_bin$clase,
  prediccion
)

```

12.21 Selección del umbral

El umbral 0.5 no siempre es el mejor.

```

umbrales <- seq(0.10, 0.90, by = 0.05)

resultados_umbral <- data.frame(
  umbral = umbrales,
  exactitud = NA_real_,
  sensibilidad = NA_real_,
  especificidad = NA_real_
)

for (i in seq_along(umbrales)) {

  pred_i <- ifelse(
    probabilidades >= umbrales[i],
    1,
    0
  )

  met <- metricas_binarias(
    test_bin$clase,
    pred_i
  )
}

```

```
resultados_umbral[i, 2:4] <- met[  
  c(  
    "exactitud",  
    "sensibilidad",  
    "especificidad"  
  )  
]  
}  
  
resultados_umbral
```

12.22 Visualizador interactivo de una red neuronal

12.22.1 Visualizador disponible en la versión web

La versión web permite modificar el número de entradas, neuronas ocultas y salidas, y genera dinámicamente el dibujo de la red y el código equivalente para `neuralnet`.

12.23 Arquitectura de la red

Una arquitectura se define por:

- número de capas ocultas;
- neuronas por capa;
- activaciones;
- conexiones;
- regularización.

No existe una arquitectura universal.

12.23.1 Red muy pequeña

Puede producir subajuste.

12.23.2 Red demasiado grande

Puede memorizar entrenamiento y sobreajustar.

Una estrategia razonable es comenzar con una red pequeña y aumentar complejidad gradualmente.

12.24 Épocas, lotes y optimizadores

12.24.1 Época

Una época representa una pasada completa por entrenamiento.

12.24.2 Lote

Un lote es un subconjunto utilizado para calcular una actualización.

12.24.3 Descenso por lotes

Usa todo el conjunto.

12.24.4 Descenso estocástico

Actualiza con una observación.

12.24.5 Mini-batch

Usa pequeños grupos y es la opción más común.

12.24.6 Optimizadores

Entre los más conocidos se encuentran:

- SGD;
- Momentum;
- RMSprop;
- Adam.

12.25 Sobreajuste

Una red sobreajustada obtiene:

- error bajo en entrenamiento;
- error alto en validación o prueba.

Señales:

- la pérdida de entrenamiento sigue bajando;
- la pérdida de validación comienza a subir;
- crece la diferencia entre ambas.

12.26 Regularización

12.26.1 Regularización L2

Agrega una penalización:

$$L_{total} = L + \lambda \sum_j w_j^2$$

12.26.2 Regularización L1

$$L_{total} = L + \lambda \sum_j |w_j|$$

12.26.3 Dropout

Desactiva aleatoriamente una proporción de neuronas durante el entrenamiento.

12.26.4 Early stopping

Detiene el entrenamiento cuando la validación deja de mejorar.

12.27 Implementación moderna con keras

El paquete `keras` permite construir redes utilizando una interfaz de alto nivel.

i Entorno de ejecución

La instalación de `keras` y su motor puede variar según el equipo. Conviene ejecutar este ejemplo en un entorno configurado o en Google Colab.

```
library(keras)

x_train_keras <- as.matrix(x_train)
x_test_keras <- as.matrix(x_test)

y_train_keras <- as.integer(
  train$Species
) - 1

y_test_keras <- as.integer(
  test$Species
) - 1
```

Modelo:

```
modelo_keras <- keras_model_sequential() |>
  layer_dense(
    units = 16,
    activation = "relu",
    input_shape = ncol(x_train_keras)
  ) |>
  layer_dropout(rate = 0.20) |>
  layer_dense(
    units = 8,
    activation = "relu"
  ) |>
  layer_dense(
    units = 3,
    activation = "softmax"
  )
```

Compilación:

```
modelo_keras |>
  compile(
    optimizer = "adam",
    loss = "sparse_categorical_crossentropy",
    metrics = "accuracy"
  )
```

Entrenamiento:

```
historia <- modelo_keras |>
  fit(
    x = x_train_keras,
    y = y_train_keras,
    epochs = 100,
    batch_size = 16,
    validation_split = 0.20,
    verbose = 0
  )
```

Evaluación:

```
modelo_keras |>
  evaluate(
    x_test_keras,
    y_test_keras,
    verbose = 0
  )
```

12.28 Curvas de aprendizaje

Las curvas de pérdida y exactitud ayudan a diagnosticar el entrenamiento.

```
plot(historia)
```

Debe compararse:

- desempeño de entrenamiento;
- desempeño de validación;
- punto donde comienza el sobreajuste.

12.29 Clasificación multiclase

En un problema con K clases:

- la capa de salida suele tener K neuronas;
- se utiliza softmax;
- la clase predicha es la de mayor probabilidad.

$$\hat{y} = \arg \max_k P(y = k \mid \mathbf{x})$$

12.30 Codificación de la respuesta

Para clasificación multiclase pueden emplearse:

- enteros de 0 a $K - 1$;
- variables indicadoras one-hot.

Ejemplo one-hot:

Clase	Neurona 1	Neurona 2	Neurona 3
A	1	0	0
B	0	1	0
C	0	0	1

12.31 Inicialización de pesos

Todos los pesos no deben comenzar con el mismo valor, porque las neuronas aprenderían exactamente lo mismo.

Las inicializaciones modernas buscan:

- romper simetría;
- mantener estable la varianza;
- mejorar el flujo de gradientes.

12.32 Gradientes que desaparecen o explotan

En redes profundas, los gradientes pueden:

- aproximarse a cero;
- crecer excesivamente.

Consecuencias:

- aprendizaje muy lento;
- inestabilidad;
- pesos extremos.

Soluciones comunes:

- ReLU;
- inicialización adecuada;
- normalización;
- arquitecturas apropiadas;
- control de la tasa de aprendizaje.

12.33 Importancia de la reproducibilidad

Para reproducir resultados:

- fijar semillas;
- registrar arquitectura;
- guardar parámetros;
- documentar divisiones;
- conservar versiones;
- guardar modelos entrenados;
- anotar métricas y umbrales.

```
set.seed(2026)
```

12.34 Interpretabilidad

Las redes neuronales suelen considerarse menos interpretables que:

- regresión logística;
- árboles de decisión;
- modelos lineales.

Herramientas posibles:

- importancia por permutación;
- análisis de sensibilidad;
- gráficos de dependencia;
- SHAP;
- LIME;
- inspección de activaciones.

La interpretación debe acompañarse de validación y conocimiento del dominio.

12.35 Aplicación biomédica

Ejemplo: clasificar pacientes según riesgo de enfermedad.

Variables:

- edad;
- presión arterial;
- glucosa;
- colesterol;
- frecuencia cardiaca.

Respuesta:

- riesgo bajo;
- riesgo alto.

Debe prestarse especial atención a:

- sensibilidad;
- falsos negativos;
- calidad de los datos;
- validación externa;
- sesgo de selección;
- explicabilidad.

12.36 Aplicación ambiental

Ejemplo: clasificar niveles de contaminación.

Variables:

- PM10;
- ozono;
- temperatura;
- humedad;
- velocidad del viento;
- hora;
- estación.

Respuesta:

- aceptable;
- no aceptable.

La red puede capturar relaciones no lineales, pero debe compararse con modelos más simples.

12.37 Comparación con otros algoritmos

Aspecto	Red neuronal	Regresión logística	Árbol	Random Forest
No linealidad	Alta	Limitada	Alta	Alta
Interpretación	Baja	Alta	Alta	Media
Preparación	Alta	Media	Media	Media
Escalamiento	Importante	Recomendable	Poco importante	Poco importante
Datos requeridos	Frecuentemente muchos	Menos	Moderados	Moderados
Costo	Puede ser alto	Bajo	Bajo	Moderado

12.38 Ventajas

- modelan relaciones complejas;
- representan no linealidades;
- se adaptan a clasificación y regresión;
- pueden aprender características internas;
- escalan a problemas de gran dimensión;
- son base de aprendizaje profundo.

12.39 Limitaciones

- requieren preparación cuidadosa;
- pueden necesitar muchos datos;
- implican varios hiperparámetros;
- pueden sobreajustar;
- son menos interpretables;
- el entrenamiento puede ser costoso;
- los resultados dependen de arquitectura e inicialización.

12.40 Errores frecuentes

1. No normalizar variables.
2. Usar el conjunto de prueba para ajustar.
3. Entrenar demasiadas épocas sin validación.
4. Usar una red enorme para pocos datos.
5. Evaluar solo exactitud.
6. Ignorar el desbalance.

7. No fijar semilla.
8. No guardar la arquitectura.
9. Interpretar probabilidades como certezas.
10. Comparar modelos con particiones diferentes.
11. Dejar identificadores como predictores.
12. No comparar contra un modelo simple.

12.41 Flujo de trabajo recomendado

1. Definir el objetivo.
2. Preparar los datos.
3. Separar entrenamiento, validación y prueba.
4. Normalizar con entrenamiento.
5. Construir una arquitectura pequeña.
6. Entrenar y revisar curvas.
7. Ajustar hiperparámetros.
8. Aplicar regularización.
9. Evaluar en prueba una sola vez.
10. Comparar con modelos base.
11. Interpretar errores.
12. Documentar y guardar el modelo.

12.42 Actividad guiada

Utiliza `iris` para clasificación multiclase.

1. Divide los datos en entrenamiento y prueba.
2. Normaliza las cuatro variables.
3. Diseña una red con una capa oculta.
4. Entrena el modelo.
5. Obtén probabilidades.
6. Asigna la clase más probable.
7. Construye la matriz de confusión.
8. Calcula exactitud.
9. Repite con más neuronas.
10. Compara los resultados.
11. Describe si existe sobreajuste.
12. Compara contra k-NN o Random Forest.

12.43 Ejercicios

12.43.1 Ejercicio 1

Calcula manualmente la salida de una neurona sigmoide.

12.43.2 Ejercicio 2

Programa una neurona ReLU en R.

12.43.3 Ejercicio 3

Entrena un perceptrón para la compuerta OR.

12.43.4 Ejercicio 4

Explica por qué XOR requiere una capa oculta.

12.43.5 Ejercicio 5

Entrena una red binaria con `neuralnet`.

12.43.6 Ejercicio 6

Compara arquitecturas `hidden = 3`, `hidden = 5` y `hidden = c(5, 3)`.

12.43.7 Ejercicio 7

Modifica el umbral de clasificación y analiza sensibilidad y especificidad.

12.43.8 Ejercicio 8

Construye una red multiclase con `keras`.

12.43.9 Ejercicio 9

Compara la red con regresión logística.

12.43.10 Ejercicio 10

Describe tres medidas para reducir sobreajuste.

12.44 Preguntas de reflexión

- ¿Qué función cumple el sesgo?
- ¿Por qué se necesitan activaciones no lineales?
- ¿Qué diferencia existe entre propagación hacia adelante y retropropagación?
- ¿Por qué debe separarse validación de prueba?
- ¿Qué indica una pérdida de validación creciente?
- ¿Cuándo conviene usar una red neuronal?
- ¿Cuándo sería preferible un modelo más simple?
- ¿Por qué las probabilidades deben interpretarse con cautela?

12.45 Conclusión

Las redes neuronales artificiales combinan neuronas simples para aprender relaciones complejas.

Sus componentes fundamentales son:

- entradas;
- pesos;
- sesgos;
- funciones de activación;
- capas;
- función de pérdida;
- optimización;
- retropropagación.

Una red efectiva no depende solamente de aumentar capas o neuronas. También requiere:

- datos de calidad;
- normalización correcta;
- validación;
- regularización;
- selección cuidadosa de arquitectura;
- evaluación con métricas apropiadas;
- comparación con modelos más sencillos.

En el siguiente capítulo podremos abordar métodos no supervisados, comenzando con **agrupamiento k-means**.

Capítulo 13

Agrupamiento con k-means

13.1 Objetivos de aprendizaje

Al finalizar este capítulo podrás:

- distinguir aprendizaje supervisado y no supervisado;
- explicar intuitivamente el algoritmo k-means;
- calcular centroides y asignar observaciones a grupos;
- implementar k-means manualmente y con `kmeans()`;
- seleccionar el número de grupos mediante el método del codo y silhouette;
- interpretar centroides y perfiles de los grupos;
- reconocer las limitaciones y los errores frecuentes;
- aplicar el algoritmo a datos reales.

13.2 Del aprendizaje supervisado al no supervisado

En los capítulos anteriores conocíamos la variable de respuesta. En el aprendizaje no supervisado no existe una etiqueta conocida que indique el grupo correcto. El objetivo es descubrir estructura en los datos.

El **agrupamiento** o *clustering* reúne observaciones similares y separa observaciones diferentes.

13.3 Idea intuitiva de k-means

k-means busca dividir las observaciones en k grupos. Cada grupo se representa mediante un **centroide**, que es el promedio de sus observaciones.

El algoritmo repite dos operaciones:

1. asignar cada observación al centroide más cercano;
2. recalcular los centroides.

El proceso termina cuando las asignaciones dejan de cambiar o la mejora es muy pequeña.

13.4 Función objetivo

k-means minimiza la suma de cuadrados dentro de los grupos:

$$WSS = \sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \mu_j\|^2$$

Aquí, C_j es el grupo j y μ_j es su centroide.

13.5 Ejemplo manual

Considera seis puntos:

Punto	x	y
A	1	1
B	1.5	2
C	3	4
D	5	7
E	3.5	5
F	4.5	5

Si fijamos $k = 2$, elegimos dos centroides iniciales, asignamos cada punto al más cercano y recalculamos los promedios de ambos grupos. Repetimos hasta estabilizar.

13.6 Implementación básica en R

```
datos <- data.frame(
  x = c(1, 1.5, 3, 5, 3.5, 4.5),
  y = c(1, 2, 4, 7, 5, 5)
)

datos
```

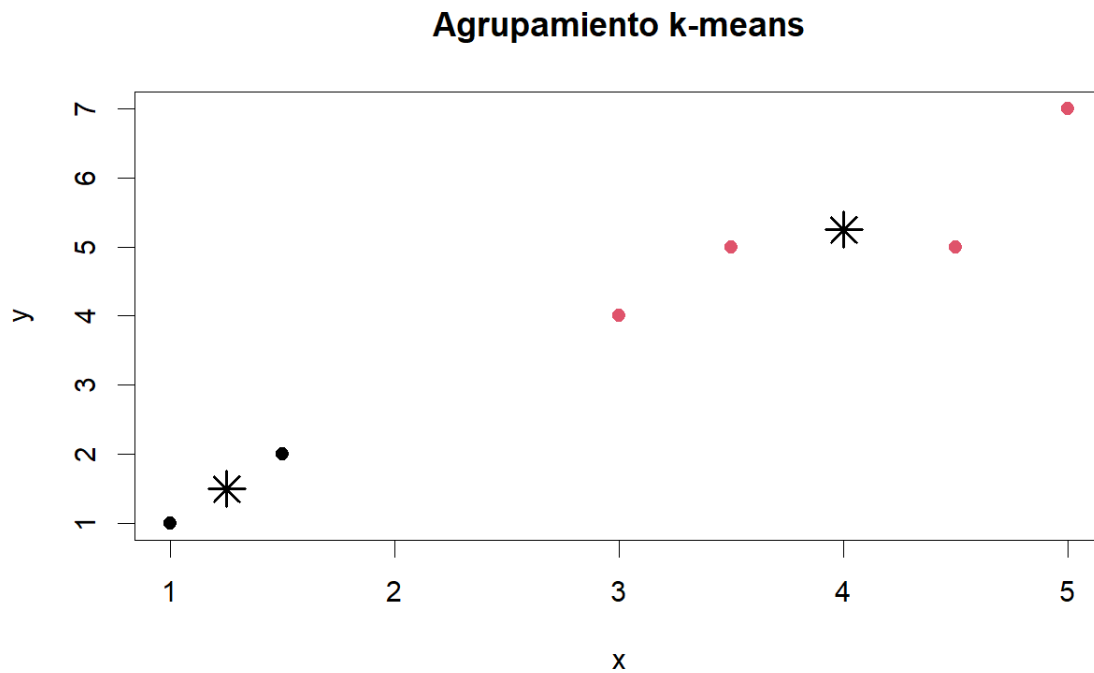
```
#>      x y
#> 1 1.0 1
#> 2 1.5 2
#> 3 3.0 4
```

```
#> 4 5.0 7
#> 5 3.5 5
#> 6 4.5 5
```

```
set.seed(123)
modelo_km <- kmeans(datos, centers = 2, nstart = 25)
modelo_km
```

```
#> K-means clustering with 2 clusters of sizes 2, 4
#>
#> Cluster means:
#>      x      y
#> 1 1.25 1.50
#> 2 4.00 5.25
#>
#> Clustering vector:
#> [1] 1 1 2 2 2 2
#>
#> Within cluster sum of squares by cluster:
#> [1] 0.625 7.250
#> (between_SS / total_SS =  78.5 %)
#>
#> Available components:
#>
#> [1] "cluster"      "centers"      "totss"      "withinss"    "tot.withinss"
#> [6] "betweenss"    "size"        "iter"      "ifault"
```

```
datos$cluster <- factor(modelo_km$cluster)
plot(
  datos$x, datos$y,
  col = datos$cluster,
  pch = 19,
  xlab = "x", ylab = "y",
  main = "Agrupamiento k-means"
)
points(modelo_km$centers, pch = 8, cex = 2, lwd = 2)
```

13.7 ¿Por qué usar `nstart`?

k-means depende de los centroides iniciales. `nstart = 25` ejecuta el algoritmo varias veces y conserva la solución con menor variación interna.

13.8 Escalamiento de variables

Si una variable está medida en miles y otra entre 0 y 10, la primera dominará las distancias. Por ello suele ser necesario estandarizar:

$$z = \frac{x - \bar{x}}{s}$$

```
X <- scale(iris[, 1:4])
head(X)
```

```
#>      Sepal.Length Sepal.Width Petal.Length Petal.Width
#> [1,]  -0.8976739   1.01560199  -1.335752   -1.311052
#> [2,]  -1.1392005  -0.13153881  -1.335752   -1.311052
#> [3,]  -1.3807271   0.32731751  -1.392399   -1.311052
```

```
#> [4,] -1.5014904 0.09788935 -1.279104 -1.311052
#> [5,] -1.0184372 1.24503015 -1.335752 -1.311052
#> [6,] -0.5353840 1.93331463 -1.165809 -1.048667
```

13.9 Aplicación con iris

```
set.seed(2026)
km_iris <- kmeans(X, centers = 3, nstart = 50)

table(
  Cluster = km_iris$cluster,
  Especie = iris$Species
)
```

```
#>      Especie
#> Cluster setosa versicolor virginica
#>      1      0          39          14
#>      2     50           0           0
#>      3      0          11          36
```

Los números de los clusters no tienen significado previo. El grupo 1 no es mejor ni peor que el grupo 2.

13.10 Interpretación de centroides

```
km_iris$centers
```

```
#>   Sepal.Length Sepal.Width Petal.Length Petal.Width
#> 1  -0.05005221 -0.88042696   0.3465767   0.2805873
#> 2  -1.01119138  0.85041372  -1.3006301  -1.2507035
#> 3   1.13217737  0.08812645   0.9928284   1.0141287
```

Cada fila representa un perfil promedio estandarizado. Valores positivos indican niveles superiores a la media; valores negativos, inferiores.

13.11 Método del codo

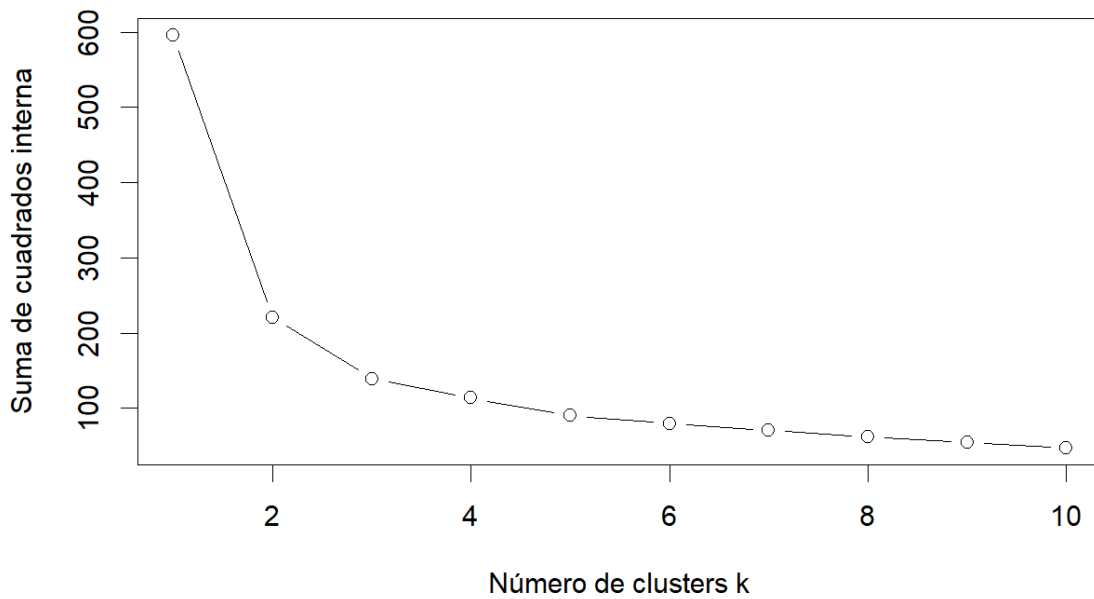
El método del codo compara la suma de cuadrados interna para distintos valores de k .

```
valores_k <- 1:10
wss <- numeric(length(valores_k))

for (i in seq_along(valores_k)) {
  set.seed(123)
  ajuste <- kmeans(X, centers = valores_k[i], nstart = 25)
  wss[i] <- ajuste$tot.withinss
}

plot(
  valores_k, wss,
  type = "b",
  xlab = "Número de clusters k",
  ylab = "Suma de cuadrados interna",
  main = "Método del codo"
)
```

Método del codo



Se busca un punto a partir del cual añadir grupos produce mejoras pequeñas.

13.12 Coeficiente silhouette

Silhouette compara cohesión interna y separación externa. Sus valores se encuentran entre -1 y 1:

- cerca de 1: agrupamiento claro;
- cerca de 0: observación fronteriza;
- negativo: posible asignación incorrecta.

```
if (!requireNamespace("cluster", quietly = TRUE)) {  
  install.packages("cluster", repos = "https://cloud.r-project.org")  
}  
  
sil <- cluster::silhouette(km_iris$cluster, dist(X))  
mean(sil[, "sil_width"])  
plot(sil)
```

13.13 Laboratorio interactivo k-means

El laboratorio interactivo de k-means está disponible en la versión web del capítulo. Permite modificar k , el tamaño de la muestra y la dispersión de los grupos.

13.14 Ventajas

- sencillo e intuitivo;
- rápido en conjuntos medianos y grandes;
- fácil de visualizar;
- útil para segmentación y exploración;
- disponible en R base.

13.15 Limitaciones

- requiere fijar k ;
- depende de la inicialización;
- es sensible a escalas y valores atípicos;
- favorece grupos aproximadamente esféricos;
- no maneja bien clusters con densidades muy diferentes;
- solo trabaja naturalmente con variables numéricas.

13.16 Errores frecuentes

1. No estandarizar variables.
2. Elegir k únicamente porque coincide con una expectativa previa.
3. Ejecutar una sola inicialización.
4. Interpretar los números de cluster como categorías ordenadas.
5. Incluir identificadores.
6. Ignorar valores atípicos.
7. Confundir clusters con clases verdaderas.
8. Describir grupos sin revisar sus centroides.

13.17 Aplicación con datos reales

En accidentes de tránsito pueden agruparse municipios, horarios o tipos de accidente usando variables como número de víctimas, frecuencia, hora, zona y severidad. El objetivo no es predecir una etiqueta, sino descubrir perfiles semejantes.

13.18 Actividad guiada

1. Selecciona cuatro variables numéricas de un conjunto real.
2. Trata valores faltantes.
3. Estandariza los datos.
4. Evalúa valores de k entre 2 y 10.
5. Usa el método del codo.
6. Calcula silhouette.
7. Entrena el modelo final con `nstart = 50`.
8. Interpreta los centroides.
9. Asigna nombres descriptivos a los perfiles.
10. Explica las limitaciones del análisis.

13.19 Ejercicios

1. Implementa una iteración manual de k-means.
2. Compara resultados con y sin estandarización.
3. Cambia la semilla y analiza la estabilidad.
4. Agrega valores atípicos.
5. Compara $k = 2$, $k = 3$ y $k = 5$.
6. Contrasta el método del codo y silhouette.
7. Aplica k-means a datos ambientales o de accidentes.

13.20 Conclusión

k-means descubre grupos mediante distancias y centroides. Su utilidad depende de una preparación adecuada, una selección razonada de k y una interpretación basada en los perfiles de cada cluster.

En el siguiente capítulo estudiaremos el **Análisis de Componentes Principales (PCA)** como técnica de reducción de dimensión.

Referencias

- Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.
- Burges, Christopher J. C. 1998. «A Tutorial on Support Vector Machines for Pattern Recognition». *Data Mining and Knowledge Discovery* 2: 121-67. <https://doi.org/10.1023/A:1009715923555>.
- Cortes, Corinna, y Vladimir Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20: 273-97. <https://doi.org/10.1007/BF00994018>.
- Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. <https://doi.org/10.1109/TIT.1967.1053964>.
- Cox, David R. 1958. «The Regression Analysis of Binary Sequences». *Journal of the Royal Statistical Society: Series B (Methodological)* 20 (2): 215-42. <https://doi.org/10.1111/j.2517-6161.1958.tb00292.x>.
- Domingos, Pedro, y Michael Pazzani. 1997. «On the Optimality of the Simple Bayesian Classifier under Zero-One Loss». *Machine Learning* 29: 103-30. <https://doi.org/10.1023/A:1007413511361>.
- Fawcett, Tom. 2006. «An Introduction to ROC Analysis». *Pattern Recognition Letters* 27 (8): 861-74. <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fix, Evelyn, y Joseph L. Hodges. 1951. *Discriminatory Analysis: Nonparametric Discrimination: Consistency Properties*. Project 21-49-004, Report 4. USAF School of Aviation Medicine.
- Hand, David J., y Keming Yu. 2001. «Idiot's Bayes—Not So Stupid after All?» *International Statistical Review* 69 (3): 385-98. <https://doi.org/10.1111/j.1751-5823.2001.tb00465.x>.
- Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.^a ed. Wiley. <https://doi.org/10.1002/9781118548387>.
- Instituto Nacional de Estadística y Geografía. 2025. «Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS), 2024». Instituto Nacional de Estadística y Geografía. <https://www.inegi.org.mx/programas/accidentes/>.

Instituto Nacional de Estadística y Geografía. 2026. «Términos de libre uso de la información del INEGI». <https://www.inegi.org.mx/inegi/terminos.html>.

James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning: with Applications in R*. 2.^a ed. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.

Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence* 2: 1137-45.

Montgomery, Douglas C., Elizabeth A. Peck, y G. Geoffrey Vining. 2021. *Introduction to Linear Regression Analysis*. 6.^a ed. Wiley.

Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1: 81-106. <https://doi.org/10.1007/BF00116251>.

Reconocimiento de la fuente de datos

Los datos de accidentes empleados en este libro proceden de la **Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS)** del Instituto Nacional de Estadística y Geografía (Instituto Nacional de Estadística y Geografía 2025).

Los datos originales pertenecen al INEGI. La limpieza, transformación, selección de variables, gráficas, modelos de aprendizaje automático e interpretaciones son elaboración del autor y no constituyen resultados oficiales ni implican el aval del Instituto.